

J Donadelli

10 de Novembro de 2026 11:29:35 ADT

Conteúdo

Conteúdo ii

1 Fundamentos 1
- 1.1 Definições iniciais . . . 1
- 1.2 Subgrafo . . . 2
- 1.3 Passeios, caminhos e circuitos . . . 3
- 1.4 Outras noções de grafos . . . 7

2 Representação 14
- 2.1 Isomorfismo . . . 14
  - 2.1.1 Invariantes de grafos . . . 17
- 2.2 Representações computacionais . . . 18
  - 2.2.1 Matriz de adjacência de G . . . 19
  - 2.2.2 Lista de adjacência de G . . . 19
- 2.3 Complexidade de algoritmos em grafos . . . 20

3 Buscas em grafos 28
- 3.1 Busca genérica . . . 28
- 3.2 Algoritmos de busca . . . 31
  - 3.2.1 Busca em Largura . . . 32
  - 3.2.2 Busca em Profundidade . . . 32
  - 3.2.3 Distâncias em grafos por busca em largura . . . 33

4 Árvores e Florestas 39
- 4.1 Árvores geradoras de custo mínimo em grafos com pesos nas arestas . . . 45
  - 4.1.1 Algoritmo de Dijkstra–Jarník–Prim para árvore geradora mínima . . . 45
  - 4.1.2 Algoritmo de Kruskal para árvore geradora mínima . . . 51
  - 4.1.3 Estruturas de dados para representar conjuntos disjuntos . . . 55

5 Grafos dirigidos 66
- 5.1 Representação computacional e percurso . . . 66
  - 5.1.1 Busca em profundidade . . . 67
- 5.2 Grafos dirigidos acíclicos e ordenação topológica . . . 69
  - 5.2.1 Ordenação topológica por busca em profundidade . . . 71
- 5.3 Componentes fortemente conexas . . . 76

6 Caminhos mínimos 86
- 6.1 Algoritmo de Dijkstra . . . 87
- 6.2 Algoritmo de Bellman–Ford . . . 91
- 6.3 Algoritmo de Floyd–Warshall . . . 94

1 Fundamentos

1.1 Definições iniciais

Seja V um conjunto. Definimos

$$\binom{V}{2} = \big\{\{u, v\} \subseteq V : u \neq v\big\}$$

por exemplo

$$\binom{\{1,2,3\}}{2} = \big\{\{1,2\},\{1,3\},\{2,3\}\big\}.$$

Definição 1.1: grafo

Um ***grafo*** é uma estrutura matemática definida por um par ordenado de conjuntos finitos (V, E) tal que $E \subseteq \binom{V}{2}$.
Os elementos de V são chamados ***vértices***.
Os elementos de E são chamados ***arestas***.

Se G denota o grafo que é definido pelo par (V, E) então escrevemos $G := (V, E)$.

Além disso, se G é um grafo e não especificamos o conjunto dos vértices e o conjunto das arestas que definem G, esses passam a ser referidos como $V(G)$ e $E(G)$, respectivamente.

Exemplo 1.1

Os conjuntos

$$V = \{1,2,3,4,5,6,7,8\}$$
$$E = \big\{\{1,2\},\{1,5\},\{2,5\},\{3,4\},\{6,7\}\big\}$$

definem um grafo.

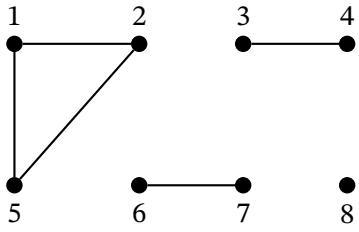


Figura 1.1: diagrama do grafo do Exemplo 1.1.

Todo grafo pode ser representado geometricamente por um ***diagrama***: no plano, desenhamos um ponto para cada vértice e um segmento de curva ligando cada par de vértices que forma uma aresta. Um diagrama de um grafo não é único, mas cada diagrama descreve um único grafo. Um diagrama do grafo do Exemplo 1.1 é apresentado na figura ao lado.

- Os vértices u e v de uma grafo G são ***adjacentes*** se $\{u, v\}$ é uma aresta de G. Também dizemos que u e v são os ***extremos*** da aresta.
- Duas arestas e e f de uma grafo G são ***arestas adjacentes*** se elas têm um vértice em comum, ou seja, $|\, e \cap f \,| = 1$.

- A aresta e ***incide*** no vértice v, em um grafo G, se $v \in e$. A quantidade de arestas de G que incidem em v é o ***grau de*** v em G, $\mathrm{d}_G(v)$:

$$\mathrm{d}_G(v) \coloneqq \Big|\big\{ w \in V(G) : \{v, w\} \in E(G) \big\}\Big|.$$

- A ***vizinhança*** do vértice v no grafo G, denotada $N_G(v)$, é formada pelos outros extremos das arestas que incidem em v, isto é,

$$N_G(v) \coloneqq \{ u \in V(G) : \{u, v\} \in E(G) \}.$$

  Alternativamente, temos $\mathrm{d}_G(v) = |N_G(v)|$.

Teorema 1.1

Para todo grafo G vale que

$$\sum_{u \in V(G)} \mathrm{d}_G(u) = 2|E(G)|. \tag{1.1}$$

Demonstração. Seja $G = (V, E)$ um grafo e defina o conjunto

$$X \coloneqq \{ (u, e) \in V \times E : u \in e \}.$$

Vamos contar seu número de elementos de duas maneiras distintas.

Primeiro, cada vértice $u \in V$ participa de $\mathrm{d}_G(u)$ elementos de X, portanto

$$|X| = \sum_{u \in V} d(u). \tag{1.2}$$

Depois, cada aresta e está presente em dois elementos de X, logo

$$|X| = 2|E|. \tag{1.3}$$

De (1.2) e (1.3) temos (1.1). □

1.2 Subgrafo

Definição 1.2: subgrafo

Dizemos que o grafo H é um ***subgrafo*** do grafo G se, e somente se,

$$V(H) \subseteq V(G) \text{ e } E(H) \subseteq E(G)$$

e, nesse caso, escrevemos $H \subseteq G$ para indicar que H é subgrafo de G.

Exemplo 1.2

Considerando o grafo G do Exemplo 1.1 temos que

$$\begin{aligned} G' &:= (\{1,2,5\},\{\{1,5\},\{5,2\},\{1,2\}\}) \quad \text{e} \\ G'' &:= (\{3,5,6\},\varnothing) \end{aligned}$$

são subgrafos de G, enquanto que

$$\begin{aligned} H &:= (\{1,2,3\},\{\{1,2\},\{3,4\}\}), \\ I &:= (\{1,2,3,4,9\},\{\{1,2\},\{3,4\}\}) \quad \text{e} \\ J &:= (\{1,2,3,4,8\},\{\{1,2\},\{3,4\},\{1,8\}\}) \end{aligned}$$

não são subgrafos de G pois: H não é grafo, em I não vale $V(I) \subseteq V(G)$ e em J não vale $E(J) \subseteq E(G)$.

Dados um grafo G e um subconjunto de vértices $U \subseteq V(G)$, o ***subgrafo induzido por*** U é o subgrafo $G[U]$ dado por

$$\left(U, E(G) \cap \binom{U}{2}\right).$$

Analogamente, definimos subgrafo induzido por um subconjunto de arestas. Se

$$M = \{e_1, e_2, \ldots, e_m\} \subseteq E(G),$$

então o ***subgrafo induzido por*** M é o subgrafo $G[M]$ dado por

$$\left(\bigcup_{i=1}^{m} e_i, M\right).$$

Exemplo 1.3

Dos grafos G, H e I cujos diagramas são dados na Figura 1.2 podemos dizer que H é um subgrafo induzido de G enquanto que I é um subgrafo mas não é induzido.

Um subgrafo $H \subseteq G$ onde $V(H) = V(G)$ é chamado de ***subgrafo gerador***. No exemplo acima o grafo I é subgrafo gerador de G, enquanto que H não é subgrafo gerador de G.

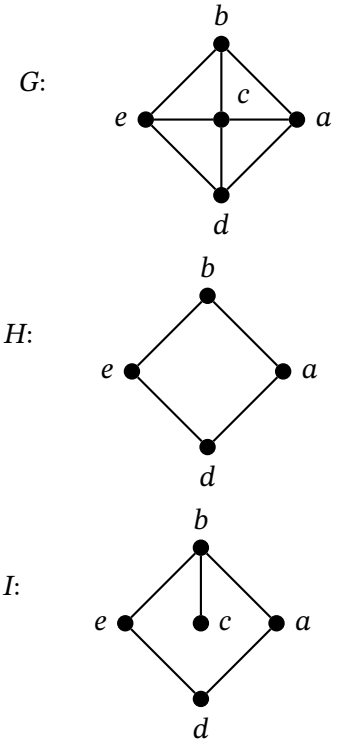


Figura 1.2: diagrama dos grafos G, H e I.

1.3 Passeios, caminhos e circuitos

Uma sequência W de vértices de um grafo G

$$W = v_1, v_2, \ldots, v_k \qquad (k \geq 1) \tag{1.4}$$

é chamada de ***passeio*** em G se v_i e v_{i+1} são adjacentes, para todo $i \in \{1, 2, \ldots, m-1\}$.

Por exemplo, no grafo do Exemplo 1.1 a sequência $1, 2, 5, 2, 1, 5$ define um passeio. Também a sequência $1, 2, 5, 2, 1$ define um passeio que, nesse caso, chamamos de ***passeio fechado***, ou seja, um passeio como W na equação (1.4) acima com $v_1 = v_k$.

No grafo G do Exemplo 1.2 temos o passeio fechado a, c, b, a, c, d, a no qual os vértices c e a ocorrem mais de uma vez. Também o par $\{a, c\}$ é uma aresta *percorrida duas vezes*. Restringido a possibilidade de repetição de vértices e arestas temos tipos especiais de subgrafos, que merecem destaque e, portanto, um nome especial.

Definição 1.3: caminho em G

Um ***caminho*** $P \subseteq G$ é um passeio

$$P = v_1, v_2, \ldots, v_k \qquad (k \geq 1)$$

em G tal que $v_i \neq v_j$, sempre que $i \neq j$, ou seja, vértices não são repetidos.

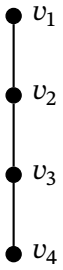


Figura 1.3: caminho.

Denotamos por P^k um caminho com k vértices.

Exemplo 1.4

Para o grafo G representado no diagrama da Figura 1.4, o passeio $P^4 = a, b, f, i$ é um caminho no grafo G. Ainda, esse mesmo caminho é definido pelo subgrafo induzido $G[\{a, b, f, i\}]$.
Também o subgrafo definido pelo par de subconjuntos $\big(\{a, d, c, h, i\}, \big\{\{a, d\}, \{d, c\}, \{c, h\}, \{h, i\}\big\}\big)$ é um caminho em G.
O subgrafo induzido pelo conjunto de vértices $\{m\}$ é um caminho P^1 e o subgrafo induzido pela aresta $\{j, l\}$ é um caminho P^2.

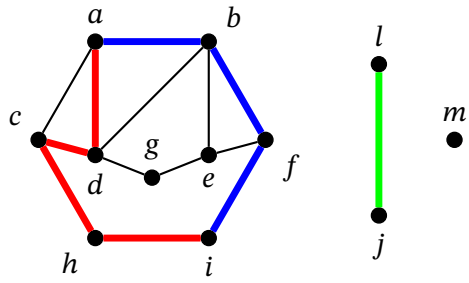


Figura 1.4: exemplos de caminhos em um grafo.

Num caminho $v_1, \ldots, v_k$, dizemos que os vértices v_1 e v_k são os seus ***extremos*** e dizemos que os vértices v_i, $1 < i < k$, são os seus ***vértices internos***.

Definição 1.4: circuito em G

Um ***circuito*** $C \subseteq G$ é um passeio

$$C = v_1, v_2, \ldots, v_k, v_1 \qquad (k \geq 3)$$

em G tal que $v_i \neq v_j$, sempre que $i \neq j$, ou seja, vértices não são repetidos exceto nos extremos do passeio.

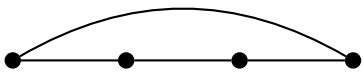


Figura 1.5: um circuito.

Exemplo 1.5: o grafo caminho e o grafo circuito

O grafo (V, E) dado por

$$V = \{1, \dots, k\}$$
$$E = \big\{\{i, i+1\} : 1 \leq i \leq k-1\big\}$$

para algum $k \geq 1$, é um caminho com k vértices (e $k-1$ arestas).

Analogamente, o grafo (V, E) dado por

$$V = \{1, \dots, k\}$$
$$E = \big\{\{i, i+1\} : 1 \leq i \leq k-1\big\} \cup \big\{\{k, 1\}\big\},$$

para algum $k \geq 3$, é um circuito com k vértices (e k arestas).

Um circuito com quatro vértices é representado pelo diagrama da Figura 1.5.

O número de arestas (e, portanto, o número de vértices) em um circuito é o ***comprimento*** desse circuito e denotamos um circuito de comprimento k por C^k. Por exemplo, o grafo G definido a partir da Figura 1.4; nele temos que $C^4 = G[\{b, d, e, g\}]$ é um circuito, bem como o subgrafo $C^5 = (V, E)$ dado por $V = \{a, b, d, e, g\}$ e $E = \{\{a, b\}, \{b, e\}, \{e, g\}, \{g, d\}, \{d, a\}\}$. Também é um circuito o subgrafo induzido pelas arestas cuja cor seja ou azul ou vermelha.

Denotamos por $\delta(G)$ o menor grau de um vértice de G, isto é,

$$\delta(G) = \min_{v \in V(G)} d_G(v)$$

chamado ***grau mínimo*** de G.

Proposição 1.1

Todo grafo G contém um caminho de comprimento pelo menos $\delta(G)$ e, se $\delta(G) \geq 2$, contém um circuito de comprimento pelo menos $\delta(G) + 1$.

Demonstração. Dado um grafo G, vamos exibir um caminho com pelo menos $\delta(G)$ arestas e um circuito com pelo menos $\delta(G) + 1$ arestas nos casos em que $\delta(G) \geq 2$.

Seja $P = x_0, x_1, \dots, x_k$ um caminho de comprimento máximo em G. De P ter comprimento máximo em G temos que $N_G(x_0) \subsetneq V(P)$, caso

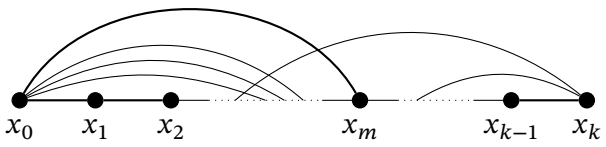


Figura 1.6: se $P = x_0, \dots, x_k$ é de comprimento máximo em G então seus extremos devem ter a vizinhança em $V(P)$.

contrário, haveria caminho mais longo que P em G. Logo, $|N_G(x_0)| \leq |V(P)| - 1$ e, como $|N_G(x_0)| \geq \delta(G)$ e $|E(P)| = |V(P)| - 1$, segue que $|E(P)| \geq \delta(G)$, isto é, P é um caminho de comprimento pelo menos $\delta(G)$.

Agora, tratando o caso de circuito, vamos supor que $\delta(G) \geq 2$. Se o grau mínimo é pelo menos dois, então o vértice x_0 tem um vizinho em P diferente de x_1 e assim está bem definido o número

$$m := \max\{\, j \in \{2, \dots, k\} : \; x_j \in N_G(x_0) \,\},$$

que é o maior índice em $\{2, \dots, k\}$ de um vértice vizinho de x_0 em P.

Basta notarmos que o circuito $x_0, x_1, \dots, x_m, x_0$ tem comprimento $m+1$ e que $N_G(x_0) \subseteq \{x_1, x_2, \dots, x_m\}$, logo $m \geq |N_G(x_0)|$, portanto,

$$m \geq |N_G(x_0)| \geq \delta(G).$$

Portanto, $m + 1 \geq \delta + 1$, ou seja, o circuito C tem comprimento pelo menos $\delta(G) + 1$. □

Distância

Uma consequência importante da definição de caminho num grafo é que, com ela, temos uma definição de distância entre vértices do grafo, definida pelo caminho mais curto. Vamos formalizar essa definição.

O ***comprimento*** de um caminho P é número de arestas no caminho. Um caminho com n vértices tem $n-1$ arestas, portanto, comprimento $n-1$.

Definição 1.5: distância em G

Em um grafo $G = (V, E)$, a ***distância*** entre dois vértices quaisquer $u, v \in V$, denotada por $\operatorname{dist}_G(u, v)$, é definida como o comprimento do menor caminho P com extremos u e v, quando existe algum caminho

$$\operatorname{dist}_G(u, v) = \min_{k \in \mathbb{Z}_+} \{\, G \supseteq P^k \text{ caminho com extremos } u, v \,\}.$$

Se u e v não são extremos de algum caminho em G, então convencionamos

$$\operatorname{dist}_G(u, v) = \infty.$$

Convencionamos, como é usual, que ∞ satisfaz

$$n + \infty = \infty \quad \text{e} \quad n < \infty,$$

para todo $n \in \mathbb{N}$.

No Exemplo 1.4, temos $\operatorname{dist}_G(a, j) = \infty$, $\operatorname{dist}_G(a, b) = 1$, $\operatorname{dist}_G(a, i) = 3$, $\operatorname{dist}_G(m, j) = \infty$.

Definição 1.6: grafo conexo

Um grafo G é ***conexo*** se para todos $u, v \in V(G)$ vale que

$$\text{dist}_G(u, v) < \infty.$$

Dados vértices $u, v \in V(G)$, dizemos que v ***alcança*** u em G se, e somente se, existe um caminho com extremos u e v em G. Assim, "alcança" é uma relação de equivalência em $V(G)$.

As classes de equivalência dessa relação são os subconjuntos dos vértices alcançáveis entre si, chamadas de ***componentes conexas*** do grafo. Equivalentemente, um grafo é conexo se e somente se tem uma única classe de equivalência pela relação "alcança".

Proposição 1.2: $(V(G), \text{dist}_G)$ é espaço métrico

A função $\text{dist}_G : V(G) \times V(G) \to \mathbb{N}$ é uma ***métrica*** em $V(G)$ pois satisfaz

1 $\text{dist}_G(u, v) = \text{dist}_G(v, u) \geq 0$ para todos $u, v \in V(G)$;
2 $\text{dist}_G(v, u) = 0$ se e somente se $u = v$;
3 a ***desigualdade triangular***

$$\text{dist}_G(u, w) \leq \text{dist}_G(u, v) + \text{dist}_G(v, w) \tag{1.6}$$

para quaisquer $u, v, w \in V(G)$.

Proposição 1.3

Se (V, E) é conexo então $(V, E \setminus \{e\})$ é conexo para toda aresta e que pertence a algum circuito de G.

Demonstração. Seja G um grafo conexo e e uma aresta de algum circuito $C \subseteq G$. Consideremos dois vértices quaisquer $u,\ v \in V$ e P um caminho $u = u_1, u_2, \ldots, u_m = v$ em G com extremos u e v.

Se $e \notin E(P)$ então P é um caminho em $(V, E \setminus \{e\})$.

Se $e \in E(P)$, então $e = \{u_i, u_{i+1}\}$ para algum i e a sequência $u_1, \ldots, u_{i-1}, C - e, u_{i+2}, \ldots, u_m$ é um passeio em $(V, E \setminus \{e\})$ que contém os vértices u e v. Logo $(V, E \setminus \{e\})$ é conexo. □

1.4 Outras noções de grafos

Em algumas situações precisamos de um modelo para um problema a ser resolvido e esse modelo seria um grafo se desconsiderássemos algumas peculiaridades da situação. Por exemplo, um mapa rodoviário pode ser modelado definindo-se um vértice para cada cidade e duas cidades formam uma aresta no grafo (modelo) se existe rodovia ligando as cidades correspondentes aos vértices. Normalmente, distância é um parâmetro importante nesses mapas e assim as arestas devem ter um

comprimento associado a elas, entretanto, "comprimento de aresta" não faz parte da definição de um grafo. Num outro exemplo, se estamos interessados em rotas de tráfego dentro de uma cidade podemos definir um vértice por esquina e duas esquinas consecutivas numa mesma rua formam uma aresta. Nesse caso, as ruas têm sentido (mão e contramão) e as arestas também deveriam ter mas, novamente, essa característica não faz parte da definição de grafos. Esses problemas e muitos outros podem ser modelados com "outros tipos" de grafos. Alguns desses outros tipos são

- ***grafo com peso nas arestas*** é definido por um tripla (V, E, ρ) de modo que (V, E) é um grafo e $\rho : E \to R$ é uma função que assume valores em R; usualmente $R \subseteq \mathbb{R}$;
- ***grafo orientado*** têm orientação nas arestas, de modo que as arestas, ou ***arcos***, agora são pares ordenados. Um grafo orientado fica definido por um par (V, E) com $E \subseteq V \times V$ sendo uma relação irreflexiva e assimétrica;
- ***grafo dirigido ou digrafo*** é dado por um par (V, E) onde $E \subseteq V \times V$;
- ***multigrafo*** é dado por um conjunto V de multigrafo vértices, um conjunto E de arestas e uma função de E em $\binom{V}{2}$ que diz quem são os extremos de cada aresta; nesse caso podemos ter mais de uma aresta com os mesmos extremos.

Em cada um dos casos acima, podemos tomar o grafo naturalmente definido por essas estruturas, chamado ***grafo subjacente***. Outros tipos podem ser derivados combinando esses tipos de grafos, por exemplo, grafo dirigido com pesos nas arestas. Um exemplo de diagrama de grafo orientado com pesos nas arestas é dado na figura abaixo.

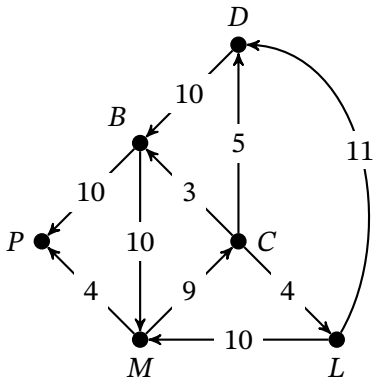


Figura 1.7: exemplo de grafo orientado.

Usamos  para representar a aresta orientada (a, c).

Exercícios

Exercício 1. Demonstre que em todo grafo o número de vértices com grau ímpar é par.

Exercício 2. Um químico deseja embarcar os produtos P, Q, R, S, T, O, X usando o menor número de caixas. Alguns produtos não podem ser colocados numa mesma caixa porque reagem. Os produtos P, Q, R, X reagem entre si; P reage com O e com S e vice-versa; T também reage com O e com S e vice-versa. Descreva o grafo que modela essa situação, mostre um diagrama desse grafo e use-o para descobrir o menor número de caixas necessárias para embarcar os produtos com segurança.

Exercício 3. Adaltina esperava as amigas Brandelina, Clodina, Dejaina e Edina para um lanche em sua casa. Enquanto esperava preparou os lanches: Bauru, Misto quente, Misto frio e X-salada. Brandelina gosta de Misto frio e de X-salada; Clodina de Bauru e X-salada; Dejaina gosta de Misto quente e Misto frio; Edina gosta de de Bauru e Misto quente. Descreva o grafo que modela essa situação, mostre um diagrama desse grafo e use-o para descobrir se é possível que cada amiga de Adaltina tenha o lanche que gosta. Se possível, determine o número de soluções.

Exercício 4. Defina todos os grafos sobre o conjunto de vértices $\{1, 2, 3, 4\}$ e tamanho 6.

Exercício 5. Para todo $n \in \mathbb{N}$, qual é o número máximo de arestas que pode ter um grafo com n vértices?

Exercício 6. O ***complemento*** de um grafo G, denotado por $G^{\complement}$, é o grafo que tem o mesmo conjunto de vértices de G e dois vértices formam uma aresta em $G^{\complement}$ se e somente se ***não*** formam uma aresta de G:

$$\begin{aligned} V(G^{\complement}) &= V(G), \\ E(G^{\complement}) &= \binom{V(G)}{2} \setminus E(G) = \big\{\{u, v\} \subset V(G) : \{u, v\} \notin E(G)\big\}. \end{aligned}$$

Dê o complemento dos seguintes grafos

(i) G dado por

$$\begin{aligned} V(G) &= \{1, 2, 3, 4, 5\}, \\ E(G) &= \{\{1, 2\}, \{1, 3\}, \{1, 4\}, \{1, 5\}, \{2, 3\}, \{2, 4\}, \{2, 5\}, \{3, 4\}, \{3, 5\}, \{4, 5\}\}. \end{aligned}$$

(ii) H dado por

$$\begin{aligned} V(H) &= \{1, 2, 3, 4, 5, 6, 7\}, \\ E(H) &= \{\{1, 2\}, \{1, 3\}, \{2, 4\}, \{2, 5\}, \{3, 6\}, \{3, 7\}\}. \end{aligned}$$

(iii) B dado por

$$\begin{aligned} V(B) &= \{a, b, c, 1, 2, 3\}, \\ E(B) &= \{\{a, 1\}, \{a, 2\}, \{a, 3\}, \{b, 1\}, \{b, 2\}, \{b, 3\}, \{c, 1\}, \{c, 2\}, \{c, 3\}\}. \end{aligned}$$

Exercício 7. Se definirmos a ***união*** dos grafos G e H por $G \cup H := \big(V(G) \cup V(H), E(G) \cup E(H)\big)$, a união $G \cup H$ é um grafo?

Exercício 8. Se definirmos ***intersecção*** dos grafos G e H por $G \cap H := \big(V(G) \cap V(H), E(G) \cap E(H)\big)$, a intersecção $G \cap H$ é um grafo?

Exercício 9. Um grafo é chamado de ***completo*** sobre V se todo par de vértices de V é uma aresta do grafo, ou seja $E = \binom{V}{2}$. Um grafo completo com n vértices é denotado por K^n. Prove que para qualquer grafo G vale que $G \cup G^{\complement} = K^n$.

Exercício 10. Considere o caso geral do exercício 2: Um químico deseja embarcar os produtos $p_1, p_2, \dots, p_n$ usando o menor número de caixas. Alguns produtos não podem ser colocados numa mesma caixa porque reagem e o químico conhece uma listagem com m pares de produtos que reagem. Seja G o grafo que modela esse problema, onde vértices são produtos e arestas os pares que reagem. Denotemos por $\chi(G)$ o número de mínimo de caixas de modo que seja possível encaixotar os produtos com segurança. Prove que

$$\chi(G) \leq \frac{1}{2} + \sqrt{2m + \frac{1}{4}}.$$

(Dica: em uma distribuição mínima de caixas, a cada duas caixas, precisa existir pelo menos um produto em uma caixa reagindo com um produto da outra caixa. Assim podemos garantir um número mínimo m de arestas para o grafo.)

Exercício 11. Vimos que $\delta(G)$ denota o grau mínimo de G. Analogamente, definimos $\Delta(G)$, o ***grau máximo*** de G, por

$$\Delta(G) = \max_{v \in V(G)} \mathrm{d}_G(v)$$

e definimos $\mathrm{d}(G)$, o ***grau médio*** de G, por

$$\mathrm{d}(G) = \frac{1}{|\,V(G)\,|} \sum_{v \in V(G)} \mathrm{d}_G(v)$$

para todo G com pelo menos 1 vértice, caso contrário $\mathrm{d}(G) := 0$.

Demonstre que $\delta(G) \leq \mathrm{d}(G) \leq \Delta(G)$ para todo grafo G.

Exercício 12. Decida se pode existir um grafo G com vértices que têm graus $2, 3, 3, 4, 4, 5$, respectivamente. E graus $2, 3, 4, 4, 5$? Se sim, descreva-os.

Exercício 13. Seja G um grafo com 14 vértices e 25 arestas. Se todo vértice de G tem grau 3 ou 5, quantos vértices de grau 3 o grafo G possui?

Exercício 14. Chico e sua esposa foram a uma festa com três outros casais. No encontro deles houveram vários apertos de mão. Ninguém apertou a própria mão ou a mão da(o) esposa(o), e ninguém apertou a mão da mesma pessoa mais que uma vez. Após os cumprimentos Chico perguntou para todos, inclusive para a esposa, quantas mãos cada um apertou e recebeu de cada pessoa uma resposta diferente. Quantas mãos Chico apertou?

Exercício 15. Prove que existem pelo menos dois vértices com o mesmo grau em todo grafo de ordem maior ou igual a dois.

Exercício 16. Para um número natural r, um grafo é r-***regular*** se todos os vértices têm grau r. Para um grafo r-regular com n vértices e m arestas, expresse m em função de n e r. Dê exemplo de um grafo 3-regular que não é completo.

Exercício 17. Demonstre que em todo grafo G com $\delta(G) > 0$ e $|E(G)| < |V(G)|$ existem pelo menos dois vértices de grau 1.

Exercício 18. Quantos subgrafos tem o grafo $\big(\{1,2,3,4,5,6\},\{\{1,2\}\}\big)$?

Exercício 19. Quantos subgrafos completos tem o grafo completo de ordem n?

Exercício 20. Sejam G um grafo e $M \subseteq E(G)$. Tome o subconjunto

$$U = \bigcup_{e \in M} e$$

de vértices de G. Prove ou dê um contraexemplo para $G[U] = G[M]$.

Exercício 21. Descubra um subgrafo induzido de

$$\begin{aligned} V(G) &= \{1,2,3,4,5,6,7,8\} \text{ e} \\ E(G) &= \big\{\{1,2\},\{1,3\},\{2,3\},\{2,5\},\{3,6\},\{8,5\},\{8,6\},\{5,6\},\{3,4\},\{5,7\}\big\} \end{aligned}$$

que seja 1-regular e tenha o maior número possível de arestas. (Qual a relação com a resolução do exercício 3?)

Exercício 22. Mostre que em qualquer coloração das arestas do grafo K^6 com as cores preta e branca vale: ou K^6 contém um k^3 com todas as arestas pretas ou K^6 contém um k^3 com todas as arestas brancas.

Exercício 23. Dados um grafo G e $U \subseteq V(G)$, dizemos que U é um ***conjunto independente***, ou ***conjunto estável***, em G se $E\big(G[U]\big) = \varnothing$, ou seja, não há par de vértices adjacentes ambos pertencentes a U.

Denotamos por $\alpha(G)$ a cardinalidade do maior conjunto independente em G,

$$\alpha(G) = \max\big\{\, |A| : A \subset V(G) \text{ é um conjunto independente} \,\big\}.$$

Demonstre que se $d(G) > \alpha(G)$ então G contém triângulo.

Exercício 24. Dados um grafo G e $U \subseteq V(G)$, dizemos que U é um ***clique*** em G se $G[U]$ é completo.

Denotamos por $\omega(G)$ a cardinalidade do maior clique em G

$$\omega(G) = \max\big\{\, |A| : A \subset V(G) \text{ é um clique} \,\big\}.$$

Demonstre que $\omega(G) = \alpha(G^{\complement})$.

Exercício 25. Demonstre que as desigualdades abaixo valem para todo grafo G

(i) $\alpha(G) \geq |V(G)|/(\Delta(G)+1)$;
(ii) $\alpha(G) \leq |E(G)|/\delta(G)$, se $\delta(G) \neq 0$;
(iii) $\omega(G) \leq \Delta(G)+1$.

Exercício 26. Suponha $H \subseteq G$. Prove ou refute as desigualdades:

(i) $\alpha(H) \leq \alpha(G)$;
(ii) $\alpha(G) \leq \alpha(H)$;
(iii) $\omega(G) \leq \omega(H)$;
(iv) $\omega(H) \leq \omega(G)$.

Exercício 27. É verdade que todo grafo com pelo menos três vértices, exatamente dois vértices de grau 1 e os demais vértices com grau 2 é um caminho?

Exercício 28. Escreva um algoritmo que recebe uma grafo G e decide se G é um caminho. Determine a complexidade do algoritmo.

Exercício 29. Sejam $P = u_1, u_2, \ldots, u_k$ e $Q = v_1, v_2, \ldots, v_\ell$ duas sequências de vértices de um grafo. Definimos a ***concatenação*** dessas sequências como a sequência dos vértices de P seguidos dos vértices de Q, denotada $P, Q = u_1, u_2, \ldots, u_k, v_1, v_2, \ldots, v_\ell$. Supondo que P e Q sejam caminhos no grafo, sob que condições a sequência P, Q é caminho?

Exercício 30. Prove que se $u_1, u_2, \ldots, u_k$ é um passeio em G então existem índices $i_1, i_2, \ldots, i_t$ tais que a subsequência $u_1, u_{i_1}, u_{i_2}, \ldots, u_{i_t}, u_k$ é um caminho em G. Também, prove que se $u_1, v_2, \ldots, v_{k-1}, u_k$ é um passeio distinto do anterior, então G contém circuito.

Exercício 31. Prove a desigualdade triangular (equação (1.6)).

Exercício 32. Mostre que a relação $e \sim f$ em $E(G)$ dada por $e = f$ ou existe um circuito $C \subset G$ tal que e e f pertencem a $E(C)$ é uma relação de equivalência.

Exercício 33. Seja k um número natural. O ***k-cubo*** é o grafo cujo conjunto de vértices são as sequências binárias de k bits e dois vértices são adjacentes se e somente se as k-tuplas correspondentes diferem exatamente em uma posição. Determine o número de vértices e de arestas do k-cubo. Determine os parâmetros α (conjunto independente máximo) e ω (clique máximo) do k-cubo.

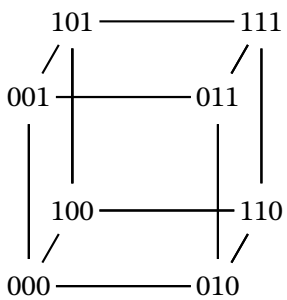


Figura 1.8: um diagrama do 3-cubo.

Exercício 34. Definimos o ***diâmetro*** do grafo G como a maior distância entre dois vértices quaisquer de G, ou seja

$$\operatorname{diam}(G) = \max_{u,v \in V(G)} \operatorname{dist}_G(u,v).$$

Naturalmente, $\operatorname{diam}(G) = \infty$ se existirem dois vértices no grafo G que não são extremos de um caminho.

Determine o diâmetro do k-cubo.

Exercício 35. Definimos a ***cintura*** do grafo G como o comprimento do menor circuito contido em G, ou seja

$$\operatorname{cin}(G) = \min\{|E(C)| : \text{ existe um circuito } C \subseteq G\},$$

quando existe um circuito, caso contrário convencionamos que $\operatorname{cin}(G) = \infty$.

Determine a cintura do k-cubo.

Exercício 36. Seja k um número natural. O ***grafo de Fibonacci*** F_k é o grafo cujo conjunto de vértices são as sequências binárias de tamanho k sem ocorrências de 1 consecutivos; dois vértices são adjacentes se e somente se as k-tuplas correspondentes diferem exatamente em uma posição.

Desenhe um diagrama do F_3.

Determine o número de vértices, arestas, o diâmetro e a cintura do F_k. Determine os parâmetros α (conjunto independente máximo) e ω (clique máximo) do grafo F_k.

Exercício 37. Seja G um grafo conexo. Mostre que se existem $w \in V(G)$ e $\{u,v\} \in E(G)$ tais que $\operatorname{dist}(w,u) = \operatorname{dist}(w,v)$ então $\operatorname{cin}(G) \leq \operatorname{dist}(w,u) + \operatorname{dist}(w,v) + 1$.

Exercício 38. Seja p um número primo e tome $V = \{0,1,\ldots,p-1\}$. Defina o grafo 3-regular $G = (A \cup B, E)$ com $A = V \times \{a\}$ e $B = V \times \{b\}$, ou seja, o que queremos é que A e B sejam duas cópias disjuntas de V.

Cada vértice $(x,a) \in A$ é adjacente aos vértices (x,b), $(x+1,b)$ e $(x+y,b)$ de B onde $y \neq 0,1$ é fixo e a soma é feita módulo p.

Demonstre que qualquer subgrafo induzido de G de ordem $p+1$ contém um caminho de comprimento 2 quando $y = 2, (p+1)/2, p-1$. O mesmo vale para qualquer $y \neq 0,1$ de V?

Representação 2

2.1 Isomorfismo

Dois grafos são isomorfos quando têm exatamente a mesma estrutura de conexões, embora seus vértices possam estar nomeados de maneiras diferentes. Ou seja, quando há uma relação 1-1 entre seus vértices que preserva a relação de adjacência. A definição forma é a seguinte.

Definição 2.1: grafos isomorfos

Dizemos que os grafos G e H são ***isomorfos*** se existe uma função bijetora

$$f : V(G) \to V(H) \tag{2.1}$$

tal que

$$\{u, v\} \in E(G) \text{ se e somente se } \{f(u), f(v)\} \in E(H)$$

para todos $u, v \in V(G)$. Nesse caso, escrevemos $G \simeq H$.

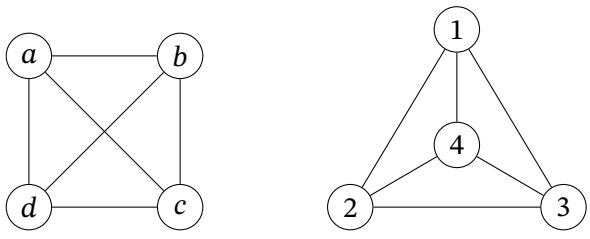
Grafos isomorfos diferem apenas na maneira como sua estrutura é representada, não na estrutura propriamente dita.

Uma função f como (2.1) acima é chamada de ***isomorfismo*** e, por abuso de notação, escrevemos

$$f : G \to H$$

para um isomorfismo como definido acima.

Por exemplo, são (trivialmente) isomorfos os grafos definidos pelas representações:



e um isomorfismo é a função f dada por

$$f(a) = 1,\ f(b) = 2,\ f(c) = 3 \text{ e } f(d) = 4.$$

Observe que há outros isomorfismos possíveis.

Notemos que quaisquer dois grafos completos G e H com n vértices são isomorfos. Mais que isso, entre grafos completos, qualquer uma das $n!$ bijeções entre $V(G)$ e $V(H)$ define um isomorfismo entre eles. Nesse caso, dizemos que o grafo completo é ***único a menos de***

isomorfismos. Ingenuamente, isso quer dizer que o grafo é único a menos dos nomes dados aos vértices. Esse fato justifica usarmos a mesma notação para todos eles: K^n para os grafos completos com n vértices.

Exemplo 2.1

Há oito grafos distintos sobre o conjunto de vértices $\{1, 2, 3\}$, eles estão descritos nas representações da Figura 2.1 abaixo.

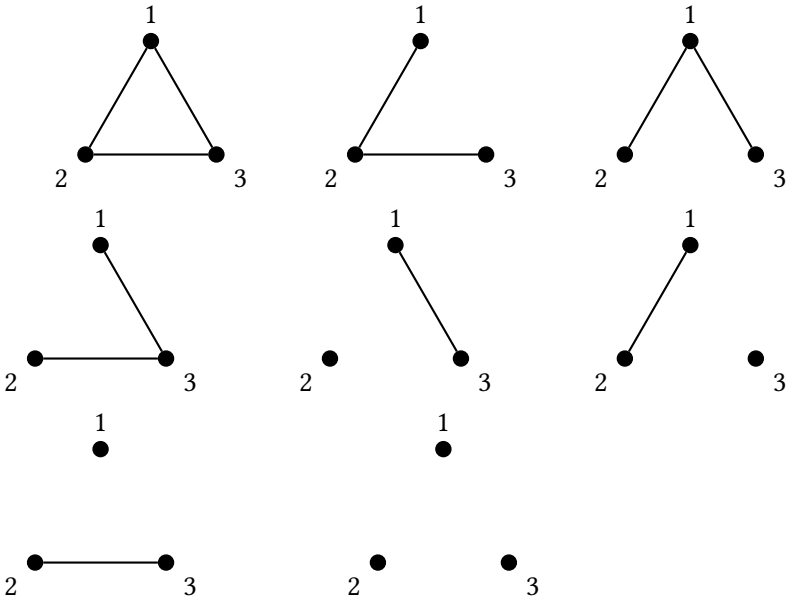


Figura 2.1: grafos distintos de ordem 3.

Exemplo 2.2

Dentre os 8 grafos distintos do exemplo anterior, há apenas 4 grafos não-isomorfos com três vértices, representados pelos diagramas da Figura 2.2, sem rótulos nos vértices (porque aqui não importa). Nesse caso dizemos que

a menos de isomorfismos

há 4 grafos de ordem 3.

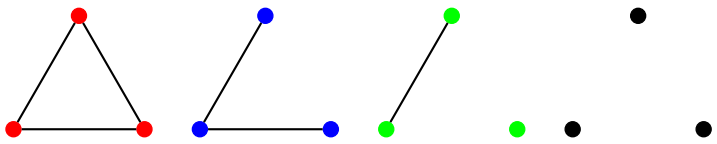
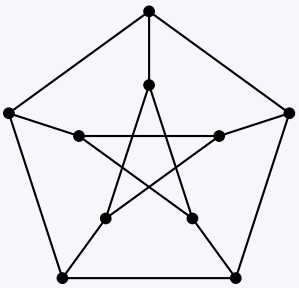


Figura 2.2: os 4 "tipos de grafos" não-isomorfos de ordem 3. A cores diferenciam "os grafos".

Exemplo 2.3: Grafo de Petersen

Os grafos representados na Figura 2.3 são isomorfos pelo isomorfismo na Tabela 2.1. Qualquer grafo isomorfo a eles é chamado de ***Grafo de Petersen***, é um dos grafos mais conhecidos

na teoria dos grafos e tem a seguinte estrutura de adjacências



A sua fama se deve ao fato dele ser o menor exemplo ou o menor contraexemplo para muitos problemas em grafos. Júlio Petersen (1839–1910) foi um matemático dinamarquês que por volta de 1898 construiu o grafo que leva seu nome como o menor contraexemplo para afirmação: ***todo grafo conexo sem ponte admite uma 3-coloração própria das arestas***. Voltaremos a essa afirmação quando tivermos conhecimento dos termos técnicos que ela contém. O grafo de Petersen é retratado nas capas de vários periódicos científicos e livros sobre teoria dos grafos e matemática discreta.

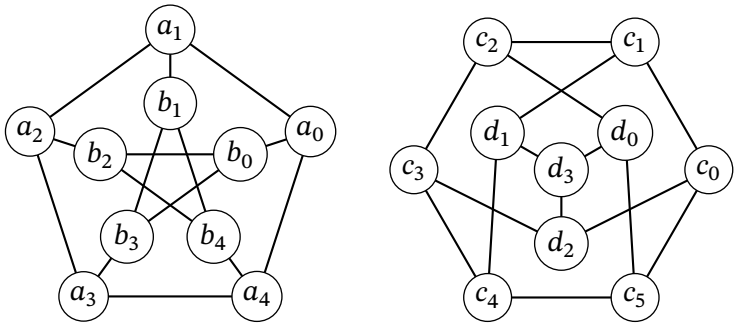


Figura 2.3: grafos isomorfos (grafo de Petersen).

v	$f(v)$	v	$f(v)$
a_0	c_3	b_0	c_4
a_1	d_2	b_1	d_3
a_2	c_0	b_2	c_5
a_3	c_1	b_3	d_1
a_4	c_2	b_4	d_0

Tabela 2.1: isomorfismo entre os grafos da Figura 2.3.

Não isomorfismo

Nenhum par dentre os grafos G, H e K representados na Figura 2.4 são isomorfos.

Temos que G não é isomorfo a H porque G não tem um vértice de grau quatro enquanto que o vértice 5 em H tem grau quatro, portanto não há como haver uma bijeção entre os vértices desse grafo que preserve as adjacências.

Pelo mesmo motivo H não é isomorfo a K.

Já G não é isomorfo a K. De fato, o vértice 1 é o único de grau um em ambos os grafos, de modo que um isomorfismo $f : G \to K$ teria $f(1) = 1$ e, portanto, $f(2) = 5$, pois 2 e 5 são os vizinhos de 1. Os outros vizinhos de 2 seriam levados nos outros vizinhos de 5, ou seja, $f(\{3, 6\}) = \{4, 6\}$. Assim, $f(\{2, 3, 6\}) = \{5, 4, 6\}$. Mas $\{2, 3, 6\}$ induz um triângulo em G e $\{4, 5, 6\}$ não induz triângulo em K (veja o

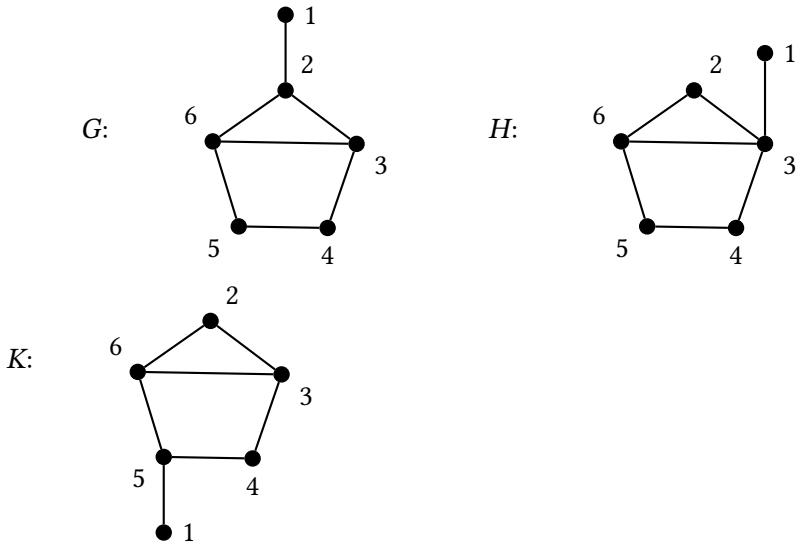


Figura 2.4: grafos não-isomorfos.

Exercício 43), uma contradição.

2.1.1 Invariantes de grafos

Nos exemplos da subseção anterior, concluímos que dois grafos não são isomorfos porque uma característica de um deles não se transfere, via isomorfismo, ao outro. Um ***invariante de grafos*** é uma característica que pode ser calculada a partir do grafo e que tem o mesmo valor em grafos isomorfos. Por exemplo, a propriedade "existe um vértice adjacente a todos os demais" é um invariante.

Definição 2.2

Um ***invariante de grafos*** é uma função f definida sobre grafos tal que $f(G) = f(H)$ sempre que $G \simeq H$.

O valor de f pode ser um número, uma sequência, um polinômio ou simplesmente sim/não. São exemplos de invariantes:

- o número de vértices $|V(G)|$ e o número de arestas $|E(G)|$;
- a ***sequência de graus***, isto é, a lista dos graus dos vértices de G em ordem não crescente;
- o número de componentes conexas;
- o número cromático $\chi(G)$, o tamanho $\omega(G)$ de uma clique máxima e o tamanho $\alpha(G)$ de um conjunto independente máximo;
- o comprimento do menor circuito (a ***cintura*** de G) e a distância máxima entre dois vértices (o ***diâmetro*** de G);
- a propriedade "G é bipartido" (ou planar, ou euleriano).

Por outro lado, "o vértice 1 tem grau 3" ***não*** é invariante: renomear os vértices pode mudar a resposta, ao contrário de "existe um vértice de grau 3", que é.

Se f é um invariante e $f(G) \neq f(H)$, então $G \not\simeq H$. Isso decorre da definição. Porém, a recíproca não vale, $f(G) = f(H)$ não implica

$G \simeq H$. Por exemplo, existem grafos não isomorfos com a mesma sequência de graus como vimos na seção anterior.

2.2 Representações computacionais

Em problemas computacionais envolvendo grafos, é conveniente supor que o conjunto de vértices é

$$V = [n] = \{1, 2, \ldots, n\}.$$

Essa convenção não restringe a classe de grafos que podemos considerar, pois a escolha dos nomes dos vértices é irrelevante quando estamos interessados em propriedades que dependem apenas da estrutura do grafo.

Essa observação é especialmente importante do ponto de vista computacional. Uma vez que estamos interessados em propriedades invariantes por isomorfismo, podemos substituir qualquer grafo por um representante isomorfo cujos vértices sejam numerados de 1 a n. Com isso facilitamos o acesso às informações usando estruturas de dados indexadas disponíveis nas linguagens de programação estruturadas.

Dado um grafo $G = (V, E)$, podemos substituir sua representação por uma representação isomorfa na qual os vértices são inteiros. Mais precisamente, se $|V| = n$, escolhemos uma bijeção $f : V \to [n]$ e definimos o grafo $G' = ([n], E')$, onde

$$E' = \big\{\{f(u), f(v)\} : \{u, v\} \in E\big\}.$$

A função f é, por construção, um isomorfismo entre G e G'.

Para os problemas computacionais considerados nesta disciplina, essa substituição não altera o problema: as propriedades de interesse são invariantes por isomorfismo. Portanto, se G é a entrada de um problema, podemos primeiro construir sua cópia isomorfa G' e executar o algoritmo sobre G'. Uma vez obtida a solução para G', podemos transportá-la de volta para G usando o isomorfismo f^{-1}.

Por exemplo, se o algoritmo produz um caminho $(v_1, v_2, \ldots, v_k)$ em G', então $(f^{-1}(v_1), f^{-1}(v_2), \ldots, f^{-1}(v_k))$ é o caminho correspondente em G. O mesmo princípio se aplica a árvores, ciclos, conjuntos de arestas, fluxos e outras estruturas produzidas pelos algoritmos.

Na implementação, a bijeção f e sua inversa podem ser mantidas por estruturas de dados que associem os rótulos originais aos inteiros $1, \ldots, n$. Por exemplo, uma tabela de espalhamento (*hash table*) permite, em geral, realizar essas consultas em tempo esperado constante, enquanto uma árvore binária de busca permite realizá-las em tempo logarítmico quando balanceada. Não trataremos dos detalhes dessa conversão.

Essa convenção é apenas uma escolha de representação. Ela não

restringe os grafos considerados, pois todo grafo com n vértices é isomorfo a um grafo com esse conjunto de vértices.

No que segue, sempre que tratamos problemas computacionais sobre grafos estamos assumindo que os grafos são definidos sobre o conjunto de vértices $[n] = \{1, 2, \ldots, n\}$, para algum $n \in \mathbb{N}$.

2.2.1 Matriz de adjacência de G

A matriz de adjacência de $G = (V, E)$ é a matriz $|V| \times |V|$ denotada por A_G, ou A simplesmente, definida por

$$A(i,j) = \begin{cases} 1, & \text{se } \{i,j\} \in E(G) \\ 0, & \text{caso contrário.} \end{cases}$$

Para o grafo G representado na Figura 2.4, a matriz de adjacência é

$$A = \begin{bmatrix} 0 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & 1 & 0 \end{bmatrix}.$$

2.2.2 Lista de adjacência de G

Outro modo, menos óbvio, usa o conceito de lista ligada, ou lista encadeada. Uma lista ligada é uma coleção de elementos com uma ordem dada pelas "ligações" chamadas ponteiros. Cada elemento dessa coleção, chamado nó, tem uma informação e uma referência para o próximo elemento. Na Figura 2.6 abaixo representamos num diagrama uma lista ligada para o conjunto $\{12, 37, 99\}$. A ideia agora



Figura 2.5: lista ligada.

é usar uma lista para a vizinhança de cada vértice de um grafo.

Lista de adjacência é um vetor $N[\ ]$ de listas ligadas. A lista $N[i]$ contém os vizinhos do vértice i e cada nó dessa lista é composto por uma variável que armazena um vértice e um ponteiro que aponta para o próximo nó da lista.

A representação de uma grafo por lista de adjacência não é única, pois qualquer permutação dos nós em $N[i]$ define uma lista válida. Uma lista de adjacência para o grafo G representado na Figura 2.4 é representada na Figura 2.6.

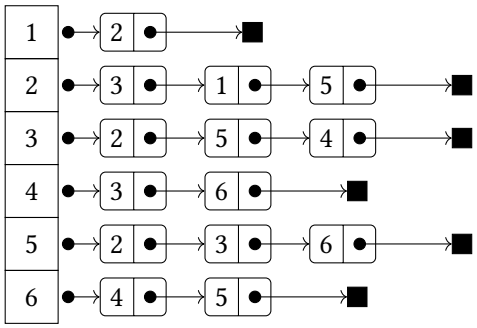


Figura 2.6: Lista de adjacência do grafo G na Figura 2.4.

2.3 Complexidade de algoritmos em grafos

A expressão que estabelece a complexidade de um algoritmo é usualmente escrita, notação assintótica, como função do ***tamanho da entrada***, isto é, o tamanho da representação de uma entrada para o algoritmo.

Se um algoritmo recebe um grafo (V, E) como entrada, e nada mais, então o parâmetro "tamanho da entrada" é o ***tamanho da representação*** do grafo. No caso de matriz de adjacência é razoável admitirmos que o tamanho da representação é $|V|^2$; no caso de lista de adjacência, que é $|V| + |E|$.

Neste texto entendemos por tempo de execução do algoritmo a ***complexidade de tempo no pior caso***, isto é, uma estimativa para o número máximo, dentre todas as entradas do mesmo tamanho, de instruções executadas e que é expressa em função do tamanho n da representação, tipicamente por meio de uma cota assintótica para $T(n)$:

$$T(n) = \max\{T(I) : I \text{ tem representação de tamanho } n\}$$

onde $T(I)$ denota o número de instruções executadas pelo algoritmo com entrada I.

Exemplo 2.4

Sejam A e B dois algoritmos distintos que resolvem o mesmo problema sobre um grafo $G = (V, E)$. O algoritmo A usa, no pior caso, exatos $|V|^2$ passos para executar a tarefa e o algoritmo B usa exatos $(|V| + |E|)\lfloor \log |E| \rfloor$ passos no pior caso. Para grafos com $|V|^2/4$ arestas o primeiro algoritmo é sempre melhor enquanto que para grafos com $1.000|V|$ arestas o segundo algoritmo é ***assintoticamente*** melhor (melhor para todo grafo com $|V| \geq 16.645$ vértices; o primeiro é melhor para $|V| \leq 16.644$).

Definição 2.3: notação O

Sejam f e g duas funções definidas nos naturais e que assumem valores naturais (ou, reais positivos), então

$$f(n) = O(g(n)), \quad \text{para } n \text{ suficientemente grande}$$

significa que *existem* constantes positivas c e n_0 tais que *para todo* $n \geq n_0$ vale

$$f(n) \leq c \cdot g(n).$$

Vejamos a complexidade de alguns algoritmos que executam tarefas simples comparando cada uma das representações dadas acima para um grafo fixo $G = (V, E)$

Tarefa	Matriz de adjacência	Lista de adjacência
$\{i, j\} \in E$?	$O(1)$	$O(1 + \min\{\mathrm{d}(i), \mathrm{d}(j)\}) = O(\lvert V\rvert)$
Computar $\mathrm{d}(i)$	$O(\mathrm{d}(i)) = O(\lvert V\rvert)$	$O(\lvert V\rvert)$
Escrever E	$O(\lvert V\rvert^2)$	$O(\lvert E\rvert)$
Inserir $\{i, j\}$	$O(1)$	$O(1)$
Remover $\{i, j\}$	$O(1)$	$O(\mathrm{d}(i) + \mathrm{d}(j)) = O(\lvert V\rvert)$

Um algoritmo em grafo é de ***complexidade de tempo polinomial*** se o tempo de execução no pior caso é expressa por uma função polinomial no tamanho da representação. É importante notar que "ter complexidade polinomial" é independente da representação do grafo ser por matriz ou lista de adjacência pois essas representações são ***polinomialmente relacionadas*** (isto é, uma pode ser obtida da outra por algoritmo de tempo polinomial). Ademais, entendemos por ***algoritmo eficiente*** um algoritmo com complexidade polinomial.

Notação assintótica

As estimativas para o custo de um algoritmo são expressas usando notação assintótica. A análise assintótica foca no comportamento do algoritmo para entradas "suficientemente grandes" e simplifica a expressão da complexidade de tempo ao ignorar constantes multiplicativas e termos aditivos de ordem inferior, uma vez que essas diferenças não alteram a ordem de grandeza do crescimento da função. A análise assintótica permite uma avaliação de desempenho representativa, independente de tecnologia ou detalhes de implementação, tornando mais simples a comparação direta entre algoritmos.

Sejam $f, g : \mathbb{N} \to \mathbb{N}$ com $g(n) > 0$ para todo n suficientemente grande. Dizemos que f é ***assintoticamente muito menor*** que g e escrevemos

$$f(n) = o(g(n)) \text{ quando } n \to \infty \tag{2.2}$$

se, e só se,

$$\lim_{n\to\infty} \frac{f(n)}{g(n)} = 0.$$

Por exemplo,

1. $1 = o(\log(\log(n)))$.
2. $\log(\log(n)) = o(\log(n))$.
3. $\log(n) = o(n^\varepsilon)$ para todo $\varepsilon > 0$.
4. $n^\varepsilon = o(n^c)$ para quaisquer $0 < \varepsilon < 1 \leq c$.
5. $n^c = o(n^{\log n})$ para todo $1 \leq c$.
6. $n^{\log n} = o(\exp(n))$.
7. $\exp(n) = o(n^n)$.
8. $n^n = o(\exp(\exp(n)))$.

Também usamos a notação $f \ll g$ o que nos permite escrever, a partir do exemplo acima, a sequência monótona

$$1 \ll \log(\log(n)) \ll \log(n) \ll n^\varepsilon \ll n^c \ll n^{\log n} \ll e^n \ll n^n \ll e^{e^n}.$$

Dizemos que f ***assintoticamente menor*** que g se

$$f(n) = O(g(n)). \tag{2.3}$$

Na definições dadas nas equações (2.2) e (2.3) o símbolo "=" não é a igualdade no sentido usual, é um abuso da notação que permitimos em troca de algumas conveniências. Temos que $n = O(n^2)$ e $n^2+2n+1 = O(n^2)$ mas $n \neq n^2 + 2n + 1$. Quando usamos essas definições.

Proposição 2.1

Se $f(n) = o(g(n))$ então $f(n) = O(g(n))$.

A prova é imediata da definição de limite. A recíproca da Proposição 2.1 não vale, como pode ser visto tomando-se $f(n) = g(n) = n^2$.

Proposição 2.2

Se $f_1(n) = O(g_1(n))$ e $f_2(n) = O(g_2(n))$ então

1. $f_1(n) + f_2(n) = O(\max\{\, g_1(n), g_2(n)\,\})$.
2. $f_1(n) \cdot f_2(n) = O(g_1(n) \cdot g_2(n))$.
3. $a \cdot f_1(n) = O(g_1(n))$ para toda constante $a \in \mathbb{R}$.

Demonstração. Vamos provar o item 1. Digamos que $f_1(n) \leq c_1 g_1(n)$ para todo $n \geq n_1$ e que $f_2(n) \leq c_2 g_2(n)$ para todo $n \geq n_2$, onde n_1, n_2, c_1, c_2 são as constantes dadas pela definição de notação O. Então

$$\begin{aligned} f_1(n) + f_2(n) &\leq \max\{\, c_1, c_2\,\}(g_1(n) + g_2(n)) \text{ com } c = \max\{\, c_1, c_2\,\} \\ &\leq c(2 \cdot \max\{\, g_1(n), g_2(n)\,\}) \end{aligned}$$

para todo $n \geq \max\{ n_1, n_2 \}$, o que prova a afirmação.

As outras propriedades são deduzidas de modo análogo e são deixadas como exercício. □

É preciso observamos alguns cuidados com o teorema acima. Para inteiros positivos n e k não vale que $1^k + 2^k + \cdots + (n-1)^k + n^k = O(\max\{ 1^k, 2^k, \ldots, (n-1)^k, n^k \}) = O(n^k)$. O problema aqui é que o máximo só pode ser tomado sobre um número de termos que não dependa de n. De fato, temos que $1^k + \cdots + n^k = O(n^{k+1})$ e que $1^k + \cdots + n^k \neq O(n^k)$.

Fica como exercício a verificação do seguinte resultado.

Proposição 2.3: transitividade

Se $f(n) = O(g(n))$ e $g(n) = O(h(n))$ então $f(n) = O(h(n))$.

Mais alguns exemplos são dados a seguir.

1. $an^2 + bn + c = O(n^2)$ para toda constante $a > 0$.
   Primeiro, observamos que, para n suficientemente grande, $|an^2 + bn + c| \leq |a|n^2 + |b|n + |c|$ e agora usamos a Proposição 2.2 em cada operando das somas, de $an^2 = O(n^2)$, $|b|n = O(n)$ e $|c| = O(1)$ temos $an^2 + |b|n + |c| = O(\max\{ n^2, n, 1 \}) = O(n^2)$. Analogamente, para todo $k \in \mathbb{N}$

$$\sum_{i=0}^{k} a_i n^i = O(n^k).$$

2. $n \log(n!) = O(n^2 \log n)$.
   Primeiro, temos $n = O(n)$. Depois, $n! = \prod_{i=1}^{n} i < \prod_{i=1}^{n} n = n^n$. Como log é crescente $\log(n!) < \log(n^n) = n \log(n)$, portanto $\log(n!) = O(n \log(n))$. Pela Proposição 2.2 $n \log(n!) = O(n^2 \log n)$.
3. Para toda constante $a > 1$, $\log_a(n) = O(\log(n))$. De fato,

$$\log_a(n) = \frac{1}{\log a} \log n$$

   porém $\frac{1}{\log a} = O(1)$ e $\log n = O(\log n)$ e pela Proposição 2.2 $\log_a(n) = O(\log(n))$.

Um algoritmo eficiente com respeito ao tempo de execução é um algoritmo que nas instâncias de tamanho n tem tempo de execução $O(n^k)$ para algum inteiro positivo k fixo.

Convenções de uso da notação assintótica

Ao usar notação assintótica nós desconsideramos os coeficientes, por exemplo, usamos $O(n^2)$ ao invés de $O(3n^2)$ e $O(1)$ ao invés de $O(1024)$ ainda que, como classes de funções, $O(n^2) = O(3n^2)$

e $O(1) = O(1024)$.

Escrevemos no argumento de $O(\cdot)$ somente o termo mais significativo, por exemplo, usamos $O(n^2)$ ao invés de $O(2n^2 + 5n \log n + 4)$. Nesse caso, da Proposição 2.2 vale que $2n^2 + 5n \log n + 4 = O(\max\{ n^2, n \log n, 1 \}) = O(n^2)$.

Quando notação assintótica aparece em equações, na forma "expressão 1 = expressão 2" onde "expressão" são expressões algébricas que envolvem notação assintótica, os termos assintóticos em "expressão 1" são quantificados universalmente, enquanto que os termos assintóticos em "expressão 2" são quantificados existencialmente. Por exemplo, em $n^3 + O(n^2) = O(n^3) + n^2 + n$ entendemos como: existe um $n_0 > 0$ tal que para todo $n \geq n_0$

para todo $f(n)$ em $O(n^2)$, existe $g(n)$ em $O(n^3)$ tal que $n^3 + f(n) = g(n) + n^2 + n$.

Notação Ω e Θ

A notação Ω adaptada por Donald Knuth da notação introduzida por outros matemáticos é definida por

$$f(n) = \Omega(g(n)) \text{ se e somente se } g(n) = O(f(n)).$$

Escrevemos $f(n) = \Theta(g(n))$ se e somente se $f(n) = O(g(n))$ e $g(n) = O(f(n))$.

P×NP

Atualmente não sabemos se existe algoritmo eficiente para vários problemas em grafos.

Por exemplo, não sabemos se existe algoritmo eficiente que receba dois grafos e decida se eles são isomorfos.

Problema 2.1: O problema do isomorfismo de grafos

Dados os grafos $G = (V, E)$ e $H = (V, E')$, decidir se eles são isomorfos.

Entretanto, não é difícil projetar um algoritmo de tempo polinomial que recebe a terna (G, H, f), formada por dois grafos e uma função $f : V(G) \to V(H)$ e devolve ***sim*** caso G e H são isomorfos e f é o isomorfismo, caso contrário devolve ***não***.

Em linguagem técnica, da Teoria da Computação, dizemos que ***o problema do isomorfismo de grafos está na classe NP de complexidade de problemas computacionais.*** Não é sabido se esse problema é NP-difícil, caso seja, um algoritmo eficiente para esse problema implica na existência de algoritmo eficiente para todo problema computacional em NP. O problema complementar está em coNP:

Problema 2.2: O problema do não-isomorfismo de grafos

Dados os grafos $G = (V, E)$ e $H = (V, E')$, decidir se eles são não-isomorfos.

Também não se conhece algoritmo de tempo polinomial no tamanho dos grafos para decidir se dois grafos não são isomorfos. Mais do que isso, não se conhece um algoritmo de tempo polinomial no tamanho dos grafos que receba como entrada uma terna (G, H, P) onde P é uma "prova curta" (i.e., de tamanho polinomial em $|V| + |E| + |E'|$) e que decida se P é uma prova do não-isomorfismo entre G e H. Em linguagem técnica dizemos que *não se sabe se o problema do não-isomorfismo de grafos está na classe* **NP** *de complexidade computacional.*

Em 2015, Babai apresentou um algoritmo para o problema de isomorfismo de grafos com tempo de execução ***quase-polinomial***. Assim, embora não se saiba se existe um algoritmo de tempo polinomial, sabemos que o problema de isomorfismo pode ser resolvido em tempo $\exp((\log n)^c)$, onde n representa o tamanho da entrada e $c > 0$ constante. O o mesmo resultado fornece um algoritmo quase-polinomial para GNI.

Assim, a dificuldade não está simplesmente em encontrar *algum* argumento para provar que dois grafos não são isomorfos. A questão mais profunda é determinar um procedimento geral e eficiente que, dados dois grafos, decida se eles pertencem à mesma classe de isomorfismo. É justamente essa questão que dá origem ao problema de isomorfismo de grafos e à sua interessante relação com a teoria da complexidade computacional.

Exercícios

Exercício 39. Determine quais pares dentre os grafos abaixo são isomorfos.

(i) G_1 tal que $V(G_1) = \{v_1, u_1, w_1, x_1, y_1, z_1\}$ e $E(G_1) = \{\{u_1, v_1\}, \{u_1, w_1\}, \{v_1, w_1\}, \{v_1, x_1\}, \{w_1, y_1\}, \{x_1, y_1\}, \{x_1, z_1\}\}$;

(ii) G_2 tal que $V(G_2) = \{v_2, u_2, w_2, x_2, y_2, z_2\}$ e $E(G_2) = \{\{u_2, v_2\}, \{u_2, w_2\}, \{v_2, w_2\}, \{v_2, x_2\}, \{w_2, y_2\}, \{x_2, y_2\}, \{y_2, z_2\}\}$;

(iii) G_3 tal que $V(G_3) = \{v_3, u_3, w_3, x_3, y_3, z_3\}$ e $E(G_3) = \{\{u_3, v_3\}, \{u_3, w_3\}, \{v_3, w_3\}, \{v_3, x_3\}, \{w_3, y_3\}, \{x_3, y_3\}, \{u_3, z_3\}\}$.

Exercício 40. Decida se o grafo representado pelo diagrama na Figura 2.7 abaixo é o grafo de Petersen.

Exercício 41. Decida se os dois grafos 4-regular representados pelos dois diagramas da Figura 2.8 abaixo são isomorfos.

Exercício 42. Mostre que existem 11 grafos não-isomorfos com 4 vértices.

Exercício 43. Sejam G e H grafos isomorfos e $f : G \to H$ um isomorfismo. É verdade que $G[U]$ é isomorfo a $H[f(U)]$ para todo

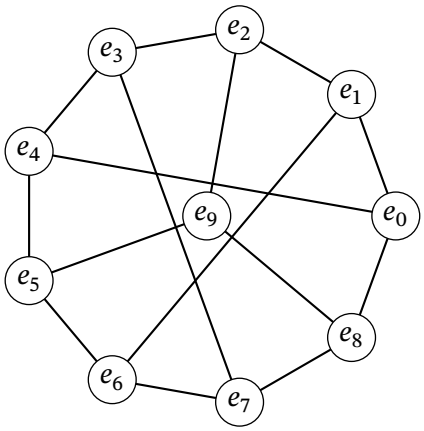


Figura 2.7: esse é um grafo de Petersen?

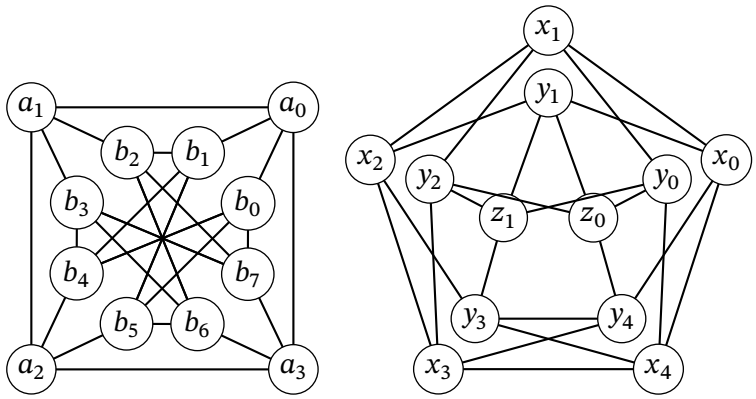


Figura 2.8: esses grafos são isomorfos?

$U \subseteq V(G)$? Justifique.

Exercício 44. Um ***automorfismo*** de um grafo é um isomorfismo do grafo sobre ele mesmo. Quantos automorfismos tem um grafo completo?

Exercício 45. Mostre que o conjunto de automorfismos de um grafo com a operação de composição de funções definem um grupo.

Exercício 46. Qual o número de grafos distintos sobre um conjunto de vértices V de tamanho n?

Exercício 47. Prove que há pelo menos $2^{\binom{n}{2}}(n!)^{-1}$ grafos não isomorfos sobre um conjunto de vértices V de tamanho n.

Exercício 48. Um grafo G é ***vértice-transitivo*** se para quaisquer $u, v \in V(G)$ existe um automorfismo f de G com $f(v) = u$. Analogamente, G é ***aresta-transitivo*** se para quaisquer arestas $\{x, y\}, \{z, w\} \in E(G)$ existe um automorfismo f de G tal que $\{f(x), f(y)\} = \{z, w\}$.

Dê um exemplo de grafo vértice-transitivo. Dê um exemplo de grafo aresta-transitivo. Dê um exemplo de grafo aresta-transitivo mas não vértice-transitivo.

Exercício 49. Considere o grafo $G = (V, E)$, com $V = [n]$ e

$$E = \{\{1,2\},\{2,3\},\{2,5\},\{3,4\},\{3,5\},\{4,6\},\{5,6\}\}.$$

(a) Escreva a matriz de adjacência A_G.

(b) Escreva uma representação de G por lista de adjacência, usando uma lista ligada para cada vértice. A ordem dos vizinhos em cada lista é livre.
(c) Quantos nós possuem, no total, as lista de adjacência? Relacione sua resposta com $|E|$.
(d) Suponha que se queira determinar se $i, j \in E$. Explique como realizar essa consulta nas duas representações e justifique as complexidades $O(1)$ e $O(|V|)$ apresentadas no texto.

Exercício 50. Seja $G = (V, E)$ um grafo com $|V| = n$ e $|E| = m$.

(a) Mostre que uma representação de G por matriz de adjacência utiliza $\Theta(n^2)$ posições, enquanto uma representação por lista de adjacência utiliza $\Theta(n + m)$ posições.
(b) Para cada uma das situações abaixo, indique qual representação é mais adequada, matriz ou lista de adjacência, e justifique sua escolha comparando o custo das operações relevantes.

- $m = \Theta(n)$ e o algoritmo precisa percorrer todos os vizinhos de cada vértice.
- $m = \Theta(n^2)$ e o algoritmo realiza muitas consultas do tipo "$i, j \in E$?".
- $m = \Theta(n)$ e o algoritmo precisa apenas escrever todas as arestas.
- $m = \Theta(n^2)$ e o algoritmo precisa escrever todas as arestas.

(c) Explique por que, apesar das diferenças de eficiência, as duas representações não alteram a classificação de um algoritmo como sendo de tempo polinomial.

Exercício 51. Sejam G um grafo sobre o conjunto de vértices $[n]$, A a sua matriz de adjacência e $b(i, j)$ as entradas da matriz A^2. Que parâmetro de G está associado ao número $b(i, i)$, para cada $i \in V(G)$? Qual a relação entre $b(i, j)$ e, $N_G(i)$ e $N_G(j)$ quando $i \neq j$?

Exercício 52. Seja G um grafo qualquer sobre o conjunto de vértices $[n]$ e A sua matriz de adjacência. Como é possível determinar o número de triângulos de um grafo G qualquer a partir de A^3?

Exercício 53. O ***traço*** de uma matriz A, denotado por $\text{tr}(A)$, é a soma dos elementos na diagonal da matriz. Quanto vale o traço de uma matriz de adjacência? Quanto vale $\text{tr}(A^2)$ em função de $E(G)$?

Exercício 54. Por A ser uma matriz simétrica, sabemos da Álgebra Linear que todos os seus autovalores são números reais. Determine os autovalores da matriz de adjacência do grafo completo K^n sobre o conjunto de vértices $[n]$. Mostre que se G é um grafo r-regular sobre o conjunto de vértices $[n]$, então r é uma autovalor de A.

Exercício 55. Sejam G um grafo sobre o conjunto de vértices $[n]$ e A sua matriz de adjacência. Prove que se os autovalores de A são distintos então o grupo dos automorfismos (Exercício 45) é comutativo.

3 Buscas em grafos

Dado um grafo G e um vértice $s \in V(G)$, *quais vértices são alcançáveis a partir de s*? Os vértices alcançáveis a partir de s são precisamente aqueles que pertencem à mesma componente que s.

3.1 Busca genérica

Seja G um grafo. Uma busca é um algoritmo que visita vértices de grafo G. Em qualquer instante durante a execução de uma busca em G vale que cada vértice $u \in V(G)$ está em um de dois estados possíveis: ***não-visitado*** ou ***visitado***.

O algoritmo abaixo, que chamaremos de Visite (G, s), recebe um grafo G, um vértice inicial $s \in V(G)$ e *visita todos os vértices alcançáveis a partir de s*.

De início temos duas estruturas de dados L e p vazias, um grafo G dado por matriz ou lista de adjacência e todos os vértices de G estão marcados *não-visitado*.

Para gerenciar as visitas, assumimos que Visite (G, s) usa um "depósito" L no qual a busca, inserção e remoção de elementos (vértices) é feita de maneira *arbitrária* e com custo fixo. Além disso, o teste "$L = \emptyset$?" é feito em tempo constante. Usamos vetor p para armazenar o vértice que "descobriu" um novo vértice não-visitado.

Algoritmo 1: Visite (G, s).

Dado : um grafo G com vértices marcados *não-visitados* e um vértice s.

Devolve : visita todos os vértices de G alcançáveis a partir de s.

```
1 insira s em L;
2 p[s] ← nil;
3 enquanto L ≠ ∅ faça
4     u ← remova um elemento de L;
5     se u é não-visitado então
6         marque u visitado;
7         para cada w ∈ N_G(u) não-visitado faça
8             p[w] ← u;
9             insira w em L;
```

No que segue faremos uma análise do algoritmo Visite(G, s) para determinar sua correção e sua complexidade.

Análise do algoritmo.

Fixe um grafo $G = (V, E)$ e um vértice $s \in V(G)$, com todos os vértices inicialmente *não-visitados*. Podemos e ***vamos assumir que G é conexo.***

Proposição 3.1: Finitude

O algoritmo Visite (G, s) executa um número finito de iterações do laço **enquanto** da linha 3 e, portanto, termina.

Demonstração. Seja I o número total de inserções em L ao longo da execução. Cada vértice u dispara o laço **para cada** da linha 7 no máximo uma vez, precisamente quando u é removido e encontrado *não-visitado* pela primeira vez (linha 5), instante em que u é marcado *visitado* e permanece assim para sempre. Nesse laço, no máximo $\mathrm{d}_G(u)$ inserções ocorrem (uma por vizinho $w \in N_G(u)$ que satisfizer a condição de ser não-visitado; podem ocorrer menos, se algum vizinho já estiver visitado nesse instante, mas não mais). Logo

$$I \;\leq\; 1 + \sum_{u \text{ é visitado}} \mathrm{d}_G(u) \;\leq\; 1 + \sum_{u \in V(G)} \mathrm{d}_G(u) \;=\; 1 + 2|E|,$$

onde o 1 inicial é a inserção de s na linha 1. Assim I é finito. Como cada iteração do laço (linha 3) remove exatamente um elemento de L (linha 4) e L começa vazio, o número de iterações é igual ao número de remoções, que não pode exceder I; logo o laço termina após no máximo $I \leq 1 + 2|E|$ iterações com $L = \varnothing$. □

Lema 3.1: Mecanismo de visita

Se um vértice $x \in V(G)$ é inserido em L ao menos uma vez durante a execução de Visite (G, s), então x é marcado *visitado* em algum momento dessa execução.

Demonstração. Pela Proposição 3.1, o algoritmo termina com $L = \varnothing$; terminar vazio significa que toda ocorrência de todo vértice inserido em L, ao longo da execução, é removido (linha 4) em algum momento.

Em particular se x foi inserido, será removido. Considere a *primeira* dessas remoções de uma ocorrência de x, na ordem temporal de execução.

Como x só pode se tornar *visitado* imediatamente após ser removido e passar pelo teste da linha 5 (não há nenhuma outra instrução no algoritmo que marque um vértice como visitado), no instante dessa primeira remoção x ainda está *não-visitado*. Logo a condição da linha 5 é satisfeita, e x é marcado *visitado* nesse instante. □

Teorema 3.1: Corretude

Ao final de uma execução de Visite (G, s), todo vértice $w \in V$ foi marcado *visitado*.

Demonstração. Por indução na distância $\text{dist}_G(s, w)$, mostraremos que todo w com é visitado ao final da execução. De G conexo, $\text{dist}_G(s, w)$ é finita.

Base. $\text{dist}_G(s, w) = 0$, i.e. $w = s$. O vértice s é inserido em L na linha 1; pelo Lema 3.1, s é visitado.

Passo indutivo. Suponha que todo vértice u com $\text{dist}_G(s, u) \leq k$ é visitado em algum momento da execução. Seja w com $\text{dist}_G(s, w) = k + 1$, e tome um caminho

$$s = v_0, v_1, \ldots, v_k, v_{k+1} = w$$

que realiza a distância $\text{dist}_G(s, w) = k + 1$.

Então, $v_0, v_1, \ldots, v_k$ também é um caminho de comprimento mínimo (justifique), logo $\text{dist}_G(s, v_k) = k$ e, pela hipótese de indução, v_k é visitado em algum instante t da execução.

Há dois casos, dependendo do estado de w no instante t:

1. Se w já estava *visitado* antes de t, então w é visitado, não há mais nada a provar.
2. Se w estava *não-visitado* no instante t: como $\{v_k, w\} \in E(G)$, temos $w \in N_G(v_k)$; no instante t, ao percorrer $N_G(v_k)$, a condição "w não-visitado" vale, logo w é inserido em L (linha 9). Pelo Lema 3.1, w é visitado em algum momento posterior da execução.

Em ambos os casos w é visitado. Por indução, todo vértice com $\text{dist}_G(s, w)$ finito é visitado ao final da execução de Visite (G, s). □

Teorema 3.2: Complexidade do "Visite"

Suponha que a busca, a inserção e a remoção de um elemento de L sejam feitas em tempo $O(T)$, sem nenhuma política específica de gerenciamento, e que o teste e a marcação do estado (*visitado*/*não-visitado*) de um vértice sejam $O(1)$. Então Visite (G, s) tem complexidade de tempo $O(|E|\, T)$.

Demonstração. Pela Proposição 3.1, o número total de inserções em L ao longo da execução é $I \leq 1 + 2|E| = O(|E|)$, e o número de remoções é $R = I = O(|E|)$ (cada iteração do laço remove exatamente um elemento). Logo o número total de operações sobre L é $O(|E|)$, cada uma custando $O(T)$, o que contribui $O(|E|\, T)$ ao tempo total, essa parcela cobre as linhas 1, 4 e 9, e também o teste "$L \neq \varnothing$" da linha 3, executado uma vez por iteração.

Resta o custo dos laços **para cada**. Quando u é removido e encontrado *não-visitado* (linha 5), o algoritmo o marca *visitado* — o que só acontece uma única vez para cada vértice, já que um vértice *visitado* nunca retorna a *não-visitado*, e percorre $N_G(u)$: para cada $w \in N_G(u)$, o teste "w é não-visitado" custa $O(1)$, e quando positivo a atribuição $p[w] \leftarrow u$ custa mais $O(1)$ (a inserção de w em L já foi contabilizada acima, no total de operações sobre L). Logo esse laço custa $O(\mathrm{d}_G(u))$, e como cada vértice o dispara no máximo uma vez,

$$\sum_{u \text{ visitado}} O\big(\mathrm{d}_G(u)\big) \;\leq\; \sum_{u \in V(G)} O\big(\mathrm{d}_G(u)\big) \;=\; O(|E|).$$

Somando as parcelas: operações em L + laços **para cada**

$$O(|E|\,T) + O(|E|) = O\big(|E|\,T\big),$$

para $T \geq 1$ (com a resalve de que todo modelo razoável de custo por operação tem $T \geq 1$). □

Para visitar os vértices de uma grafo G qualquer, não necessariamente conexo, repetimos o Visite(G, s) em cada componente conexa:

Algoritmo 2: Busca(G).
Dado : um grafo G.
Devolve : visita aos vértices de G.

```
1 marque todos os elementos de V(G) não-visitado;
2 enquanto há vértice não-visitado faça
3     escolha v não-visitado;
4     Visite( G, v).
```

A inicialização tem custo $O(|V(G)|)$. Se $C_1, \ldots, C_k$ são as componentes conexas de G, o tempo de cada visite é $O(|E(C_i)|\,T)$ e $\sum_i |E(C_i)| = |E(G)|$, portanto, o tempo total é $O(|V(G)| + |E(G)|\,T)$.

Teorema 3.3: Complexidade de "Busca"

Suponha que a busca, a inserção e a remoção de um elemento de L sejam feitas em tempo $O(T)$, sem nenhuma política específica de gerenciamento, e que o teste e a marcação do estado (*visitado*/*não-visitado*) de um vértice sejam $O(1)$. Então Busca(G) visita todos os vértices de um grafo (V, E) em tempo $O(|V| + |E|\,T)$.

3.2 Algoritmos de busca

Há, principalmente, dois casos particulares do algoritmo 1 visto acima. Cada caso é obtido quando definimos uma política de gerenciamento

para L.

3.2.1 Busca em Largura

Se a estrutura de dados L é uma ***fila***, com inserção feita no fim da fila e remoção no inícios da fila, então essas instruções, inserções (*push*) e remoções (*pull*), podem ser realizadas em tempo $O(1)$ cada, portanto o algoritmo de Busca em Largura é executado em tempo $O(V + E)$.

3.2.2 Busca em Profundidade

Se a estrutura de dados L é uma ***pilha***, com inserção e remoção feitas no topo da pilha, então essas instruções, inserções (*push*) e remoções (*pop*), podem ser realizadas em tempo $O(1)$ cada, portanto o algoritmo de Busca em Largura é executado em tempo $O(V + E)$.

Busca em Profundidade rotulada

A seguinte versão recursiva para a busca em profundidade, que chamamos de ***busca em profundidade rotulada***, será usada adiante nesse texto; no início temos

$$\text{cont} = \text{sai}[v] = \text{chega}[v] = 0 \quad \text{e} \quad \texttt{p}[v] = \texttt{nil},$$

para todo vértice v de G. A ideia é manter uma variável contadora das operações de inserção e remoção na pilha L. Esse contador é usado para rotular cada vértice com dois valores inteiros: o valor do contador quando o vértice v entrou na pilha, chega$[v]$, e o valor do contador quando o vértice v saiu da pilha, sai$[v]$.

Algoritmo 3: BP(G, w).

Dado : um grafo G e um vértice w.
Devolve : busca em profundidade em G a partir de w com rótulos *chega*, *sai* e p em cada vértice.

```
1 cont ← cont + 1;
2 chega[w] ← cont;
3 para cada u ∈ N(w) faça
4     se chega[u] = 0 então
5         p[u] ← w;
6         BP(G, u);
7 cont ← cont + 1;
8 sai[w] ← cont.
```

Ainda, o algoritmo usa um vetor $p[\ ]$ indexado pelos vértices. No final da execução $p[v]$ tem o vértice que foi empilhado antes de v

durante a execução, em outras palavras, foi o vértice que descobriu v.

Proposição 3.2: Rotulação na Busca em Profundidade

Após uma execução de BP (G, w), para quaisquer $u, v \in V(G)$ alcançáveis a partir de w, um e somente um dos itens abaixo vale para esse par de vértices

1. $\text{chega}[u] < \text{sai}[u] < \text{chega}[v] < \text{sai}[v]$;
2. $\text{chega}[v] < \text{sai}[v] < \text{chega}[u] < \text{sai}[u]$;
3. $\text{chega}[u] < \text{chega}[v] < \text{sai}[v] < \text{sai}[u]$;
4. $\text{chega}[v] < \text{chega}[u] < \text{sai}[u] < \text{sai}[v]$.

Demonstração. A prova é deixada como exercício. □

3.2.3 Distâncias em grafos por busca em largura

Nesta seção adaptamos a busca em largura para calcular, a partir de um vértice fixado s, a distância até todos os demais vértices do grafo. A ideia é simples: em vez de apenas marcar cada vértice como *visitado*, usamos o próprio valor da distância como marcador de estado (um vértice ainda não foi descoberto enquanto $\mathtt{d}[\cdot]$ vale ∞) e atribuímos a cada vizinho recém-descoberto a distância do vértice que o descobriu, mais um.

Algoritmo 4: Distância(G, s).

Dado : um grafo G e um vértice $s \in V(G)$.
Devolve : $\text{dist}_G(s, v)$ para todo $v \in V(G)$, e um vetor p de predecessores.

```
1  para cada v ∈ V(G) faça d[v] ← ∞;
2  d[s] ← 0;
3  p[s] ← nil;
4  insira s na fila L;
5  enquanto L ≠ ∅ faça
6      u ← remova da fila L;
7      para cada v ∈ N_G[u] tal que d[v] = ∞ faça
8          d[v] ← d[u] + 1;
9          p[v] ← u;
10         insira v na fila L;
```

Fixe um grafo $G = (V, E)$ e um vértice $s \in V(G)$. Para todo $v \in V(G)$, denotamos por $\text{dist}_G(s, v)$ a distância verdadeira entre s e v em G (o número de arestas de um caminho mais curto), com a convenção $\text{dist}_G(s, v) = \infty$ se v não é alcançável a partir de s. Ao longo desta seção, $\mathtt{d}[v]$ denota o valor armazenado na variável de

mesmo nome pelo algoritmo Distância(G, s) em um dado instante da execução (o contexto deixa claro qual instante).

Usaremos duas observações imediatas sobre o algoritmo. Primeiro, um vértice $v \neq s$ só recebe valores de $\mathrm{d}[v]$ e de $p[v]$ no momento em que é inserido em L, e a condição do laço **para cada** exige $\mathrm{d}[v] = \infty$; logo isso acontece no máximo uma vez por vértice, e *depois que* $\mathrm{d}[v]$ *se torna finito, seu valor nunca mais muda.* Segundo, o valor atribuído a v é $\mathrm{d}[u]+1$, em que $u = p[v]$ é o vértice que estava sendo processado.

O primeiro resultado mostra que o vetor de predecessores $p[\,]$ sempre codifica um caminho válido até s, cujo comprimento é o valor $\mathrm{d}[v]$.

Lema 3.2: Caminho via p

Se, ao final da execução de Distância(G, s), $\mathrm{d}[v] = k$ para algum k finito, então existe uma sequência de vértices *distintos* $v = v_k, v_{k-1}, \ldots, v_1, v_0 = s$ tal que $v_{i-1} = p[v_i]$ e $\{v_{i-1}, v_i\} \in E(G)$, para $i = 1, \ldots, k$. Em particular, $\mathrm{dist}_G(s, v) \leq \mathrm{d}[v]$.

Demonstração. Por indução em $k = \mathrm{d}[v]$.

Base ($k = 0$). Então $v = s$: todo vértice diferente de s que recebe valor recebe $\mathrm{d}[u] + 1 \geq 1$. A sequência é a trivial $v_0 = s$.

Passo indutivo. Seja $k \geq 1$ e suponha o resultado válido para $k - 1$. Seja $v \neq s$ com $\mathrm{d}[v] = k$. O valor de $\mathrm{d}[v]$ foi atribuído quando v foi descoberto a partir de $u = p[v]$, com $v \in N_G(u)$ (logo $\{u, v\} \in E(G)$) e $\mathrm{d}[u] = k - 1$ (valor que não muda mais).

Por hipótese de indução, existe uma sequência de vértices distintos $u = v_{k-1}, \ldots, v_0 = s$ com as propriedades desejadas. Acrescentando $v_k := v$ obtemos $v_k, v_{k-1}, \ldots, v_0$.

Essa sequência é um caminho de s a v com $k = \mathrm{d}[v]$ arestas. Como $\mathrm{dist}_G(s, v)$ é o comprimento *mínimo* entre os caminhos de s a v, segue $\mathrm{dist}_G(s, v) \leq \mathrm{d}[v]$. □

Falta mostrar que os valores $\mathrm{d}[v]$ são de fato as distâncias verdadeiras, e é aqui que o fato de L ser uma *fila* (e não uma pilha, nem uma política arbitrária) é essencial. A demonstração usa a estrutura do algoritmo em "camadas", uma para cada valor de distância.

Para $k \geq 0$, seja

$$V_k := \{v \in V(G) : \ \mathrm{dist}_G(s, v) = k\}.$$

Usaremos dois fatos elementares sobre distâncias:

(i) se $\{x, y\} \in E(G)$ e $\mathrm{dist}_G(s, x)$ é finito, então $\mathrm{dist}_G(s, y) \leq \mathrm{dist}_G(s, x) + 1$ (um caminho mais curto até x, seguido da aresta $\{x, y\}$, é um passeio de comprimento $\mathrm{dist}_G(s, x) + 1$ até y);

(ii) todo $v \in V_{k+1}$ tem um vizinho em V_k (o penúltimo vértice de um caminho mais curto de s a v).

Em particular, se $V_k \neq \emptyset$ então $V_j \neq \emptyset$ para todo $j \leq k$.

Lema 3.3: Estrutura em camadas da fila

Para todo $k \geq 0$ tal que $V_k \neq \emptyset$, a execução atinge um instante (o *instante* k) no qual:

(a) $L = V_k$;
(b) os vértices v com $\mathrm{d}[v] < \infty$ são exatamente os de $V_0 \cup \cdots \cup V_k$, e todos eles satisfazem $\mathrm{d}[v] = \mathrm{dist}_G(s, v)$.

Demonstração. Por indução em k.

Base ($k = 0$). Tome como instante 0 o momento logo após a linha 4. Nele $L = \{s\} = V_0$ e s é o único vértice com valor finito, $\mathrm{d}[s] = 0 = \mathrm{dist}_G(s, s)$.

Passo indutivo. Suponha $V_{k+1} \neq \emptyset$ (o que implica $V_k \neq \emptyset$) e que o instante k existe. Como L é uma fila e $L = V_k$ nesse instante, as próximas $|V_k|$ remoções retiram exatamente os vértices de V_k, e tudo o que é inserido nesse intervalo fica atrás deles. Considere o processamento de um vértice $u \in V_k$ (cujo valor é $\mathrm{d}[u] = k$, por (b)) e um vizinho $v \in N_G(u)$. Pelo fato (i), $\mathrm{dist}_G(s, v) \leq k + 1$.

- Se $\mathrm{dist}_G(s, v) \leq k$, então $v \in V_0 \cup \cdots \cup V_k$, que por (b) já tem valor finito no instante k; logo v é ignorado.
- Se $\mathrm{dist}_G(s, v) = k + 1$, então $v \notin V_0 \cup \cdots \cup V_k$ e $\mathrm{d}[v] = \infty$ no instante k. O primeiro vértice de V_k adjacente a v a ser processado insere v com $\mathrm{d}[v] \leftarrow k + 1$; os seguintes o ignoram.

Portanto, durante o processamento de V_k são inseridos apenas vértices de V_{k+1}, cada um uma única vez e com o valor correto $k + 1$; e, pelo fato (ii), todo vértice de V_{k+1} é inserido, pois tem algum vizinho em V_k. Logo, no instante $k + 1$, imediatamente após o último vértice de V_k ser processado, temos $L = V_{k+1}$ e os vértices com valor finito são exatamente os de $V_0 \cup \cdots \cup V_{k+1}$, todos com valor correto. □

Combinando os dois lemas, obtemos a corretude do algoritmo.

Teorema 3.4: Corretude

Ao final da execução de Distância(G, s), vale $\mathrm{d}[v] = \mathrm{dist}_G(s, v)$ para todo $v \in V(G)$.

Demonstração. Se $\mathrm{dist}_G(s, v) = k$ é finito, então $V_k \neq \emptyset$ e, pelo item (b) do Lema 3.3 no instante k, temos $\mathrm{d}[v] = k$; como esse valor nunca mais muda, ele é o valor final.

Se $\mathrm{dist}_G(s, v) = \infty$, suponha por absurdo que $\mathrm{d}[v]$ seja finito ao final da execução. Pelo Lema 3.2, existiria um caminho de s a v em G, contradizendo $\mathrm{dist}_G(s, v) = \infty$. Logo $\mathrm{d}[v] = \infty$. □

Como consequência imediata, o vetor $p[\,]$ não serve apenas para certificar uma distância: ele permite reconstruir explicitamente um

caminho mais curto.

Corolário 3.1: Reconstrução de caminho mínimo via p

Para todo $v \in V(G)$ com $\text{dist}_G(s, v)$ finito, a sequência $v = v_k, v_{k-1}, \ldots, v_0 = s$ (com $k = \text{dist}_G(s, v)$) obtida seguindo os ponteiros $v_{i-1} = p[v_i]$ é um caminho *mais curto* de s a v em G.

Demonstração. Pelo Teorema 3.4, $\mathrm{d}[v] = \text{dist}_G(s, v) = k$. Pelo Lema 3.2, a sequência é um caminho (de vértices distintos) de s a v com k arestas, que é o menor comprimento possível. Portanto é um caminho mais curto. □

Observação 3.1

O fato de L ser gerenciada como *fila* é usado de forma essencial na demonstração do Lema 3.3: é ele que garante que toda a camada V_k é processada antes de qualquer vértice de V_{k+1}. Com outra política (por exemplo, uma pilha), o algoritmo continua visitando todos os vértices alcançáveis a partir de s, mas $\mathrm{d}[v]$ passa a registrar apenas o comprimento de *algum* caminho até v (o encontrado pela política), não necessariamente o mais curto.

Quanto à complexidade, cada vértice é inserido em L no máximo uma vez (a condição $\mathrm{d}[v] = \infty$ no laço **para cada** garante isso), e portanto o mesmo argumento usado para o algoritmo genérico Visite(G, s) se aplica sem modificações: com listas de adjacência e supondo $O(1)$ por operação em L, o algoritmo Distância(G, s) tem complexidade de tempo $O(|V| + |E|)$.

Exercícios

Exercício 56. Escreva uma versão não recursiva da busca em profundidade rotulada e analise sua complexidade.

Exercício 57. Prove a Proposição 3.2.

Exercício 58. Considere uma execução da busca rotulada BP(G, w) e defina, para todo $t \in \mathbb{N}$, a t-ésima iteração do vetor p, denotada por $p^{(t)}$, recursivamente, da seguinte maneira

$$\mathrm{p}^{(t)}[v] = \begin{cases} \texttt{nil} & \text{se } t = 0 \\ \mathrm{p}^{(t-1)}[\mathrm{p}[v]] & \text{se } t > 0. \end{cases}$$

Prove as seguintes equivalências: para quaisquer $u, v \in V(G)$

1. $[\text{chega}[v], \text{sai}[v]] \cap [\text{chega}[u], \text{sai}[u]] = \varnothing$ se e somente se não existe $t \in \mathbb{N}$ tal que $\mathrm{p}^{(t)}[u] = v$ e não existe $t' \in \mathbb{N}$ tal que $\mathrm{p}^{(t')}[v] = u$;

2. $\big[\text{chega}[v], \text{sai}[v]\big] \subset \big[\text{chega}[u], \text{sai}[u]\big]$ se e somente se existe $t \in \mathbb{N}$ tal que $p^{(t)}[v] = u$;
3. $\big[\text{chega}[v], \text{sai}[v]\big] \supset \big[\text{chega}[u], \text{sai}[u]\big]$ se e somente se existe $t \in \mathbb{N}$ tal que $p^{(t)}[u] = v$.

Exercício 59. Sejam $G = (V, E)$ um grafo e $L \subseteq V$ um conjunto de vértices. O conjunto L define um ***corte de arestas*** em G, denotado $\nabla_G(L)$ e dado por

$$\nabla_G(L) = \{\{u, v\} \in E : \ u \in L, \ v \in \overline{L}\}$$

onde $\overline{L} = V \setminus L$.

Considere o seguinte algoritmo sobre um grafo conexo G

Algoritmo 5: Visite_vértices (G).

Dado : um grafo conexo G.
Devolve : visita aos vértices de G.

```
1  L, M = ∅;
2  marque todos os elementos de V(G) não-visitado;
3  escolha um vértice v não-visitado;
4  insira v em L e marque v visitado;
5  enquanto ∇_G(L) ≠ ∅ faça
6      escolha {u, w} ∈ ∇(L);
7      insira {u, w} em M;
8      v ← o extremo de {u, w} em L̄
9      insira v em L;
10     marque v visitado.
```

1. O algoritmo visita todos os vértices de G? (Pode-se assumir que $\nabla(L) = \emptyset$ se, e só se, $L = V(G)$ – dado que $L \neq \emptyset$ ao longo de toda a execução – e esse fato será provado em um capítulo adiante).
2. Prove que, no final de execução do algoritmo, M contém $|V(G)|-1$ arestas.
3. Determine a complexidade do algoritmo, propondo uma estrutura de dados que permita testar $\nabla(L) \neq \emptyset$ e escolher uma aresta de $\nabla_G(L)$ eficientemente. Pode-se assumir que as demais operações em L e em M são feitas em tempo constante.
4. Prove que $G[M]$ não contém circuito (dica: indução no número de iterações: cada aresta inserida em M conecta um vértice já em L a um vértice novo (que nunca tocou M antes, pois estava em $\overline{L}$)).

Exercício 60. Um grafo é dito ***acíclico*** se não contém circuitos. Verifique se o seguinte algoritmo está correto independentemente da política para L:

Algoritmo 6: Acíclico(G).

Dado : um grafo simples G.

Devolve : ***verdadeiro*** se G é acíclico, ***falso*** caso contrário.

```
1  marque todos os vértices de G não-visitado;
2  para cada s ∈ V(G) não-visitado faça
3      insira s em L;
4      p[s] ← nil;
5      marque s visitado;
6      enquanto L ≠ ∅ faça
7          u ← remova um elemento de L;
8          para cada v ∈ N_G(u) faça
9              se v é não-visitado então
10                 p[v] ← u;
11                 marque v visitado;
12                 insira v em L;
13             senão
14                 se v ≠ p[u] então retorne falso ;
15 retorne verdadeiro
```

4 Árvores e Florestas

Chamamos de ***acíclico*** qualquer grafo que não contenha circuito. Uma ***floresta*** é um grafo acíclico e os componentes conexos de uma floresta são árvores, ou seja, uma ***árvore*** é um grafo conexo e acíclico.

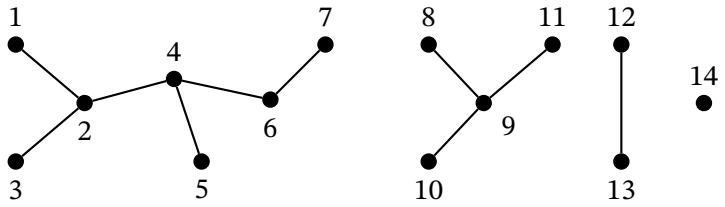


Figura 4.1: Uma floresta com 4 árvores.

Notemos que todo grafo acíclico G com pelo menos dois vértices tem pelo menos dois vértices de grau 1, basta tomarmos os extremos de um caminho de comprimento máximo (veja a Figura 1.6, pág. 5).

Chamamos de ***folha*** todo vértice de grau 1 em uma árvore.

Proposição 4.1

Toda árvore com n vértices tem $n - 1$ arestas.

Demonstração. Vamos provar por indução em $n \geq 1$ que a seguinte afirmação vale: *se um grafo com n vértices é uma árvore então o número de arestas é $n - 1$.*

Se o grafo é trivial, isto é $n = 1$, então o número de arestas é 0. Para $n \geq 2$, vamos assumir que toda árvore com $n - 1$ vértices tem $n - 2$ arestas e seja G uma árvore com n vértices.

Tome $v \in V(G)$ uma folha de G. O grafo $G - v$, dado por

$\nabla_G(U)$, o corte definido por U, foi definido no Exercício 59

$$G - v \coloneqq \big(V(G) \setminus \{v\},\, E \setminus \nabla_G(\{v\})\big)$$

é uma árvore (justifique) com $n - 1$ vértices e pela hipótese indutiva $|E(G - v)| = n - 2$. Como $|E(v)| = 1$ e $|E(G - v)| = |E(G) \setminus E(v)|$, temos que $|E(G)| = n - 1$. Portanto, pelo Princípio de Indução Finita, a afirmação vale para todo $n \geq 1$. □

Por outro lado, grafos conexos com uma aresta a menos que o número de vértices são árvores. Proveremos esse fato mais adiante.

Definição 4.1: Remoção e adição de aresta

Se $G = (V, E)$ é um grafo e e uma aresta de G, então definimos o grafo obtido pela remoção de e, denotado por $G - e$, por

$$G - e \coloneqq \big(V, E \setminus \{e\}\big).$$

Sejam $x, y \in V(G)$ vértices distintos de G. Definimos o grafo $G + xy$ por

$$G + xy := \Big(V(G), E(G) \cup \big\{\{x, y\}\big\}\Big).$$

Note que, sendo x e y adjacentes, $G + xy = G$.

Teorema 4.1: caracterização de árvore

As seguintes afirmações são equivalentes para todo grafo $G = (V, E)$:

(1) G é árvore;
(2) para quaisquer $x, y \in V$ existe um único caminho em G com extremos x e y;
(3) G é *conexo minimal*: G é conexo e $G - e$ é desconexo, para qualquer $e \in E$;
(4) G é *acíclico maximal*: G é acíclico e $G + xy$ contém circuito, para quaisquer $x, y \in V$ não adjacentes em G.

Demonstração. Para G trivial o teorema vale como pode-se verificar facilmente. Se G tem ordem 2, então $G = K^2$ e a verificação também é fácil, deixa-mo-lá para o leitor. Vamos supor que G tem pelo menos 3 vértices e mostraremos (1)⇒(2)⇒(3)⇒(4)⇒(1).

(1) ⇒ (2): Seja G uma árvore e sejam $x, y \in V$. Se $x = y$, o único caminho é o trivial. Suponha $x \neq y$. Como G é conexo, existe um caminho de x a y. Suponha, por absurdo, que $P = x_0, x_1, \dots, x_{n+1}$ e $Q = y_0, y_1, \dots, y_{m+1}$ sejam caminhos distintos com $x_0 = y_0 = x$ e $x_{n+1} = y_{m+1} = y$. Seja

$$p := \max\{i \geq 0 : \ x_k = y_k \text{ para todo } 0 \leq k \leq i\}.$$

Como y ocorre em Q apenas na última posição (e em P também), $p = n+1$ ou $p = m+1$ forçaria $n = m$ e $P = Q$. Logo $p \leq \min\{n, m\}$, os vértices x_{p+1} e y_{p+1} existem e $x_{p+1} \neq y_{p+1}$. Seja

$$q := \min\{j : \ j > p \text{ e } x_j = y_\ell \text{ para algum } \ell > p\},$$

e fixe $\ell > p$ com $x_q = y_\ell$. Este q existe, pois $j = n + 1$ serve (com $\ell = m + 1$). Considere

$$C = x_p, x_{p+1}, \dots, x_q = y_\ell, y_{\ell-1}, \dots, y_{p+1}, y_p = x_p.$$

Seus vértices são distintos: P e Q são caminhos, e, pela minimalidade de q, nenhum x_j com $p < j < q$ aparece em $y_{p+1}, \dots, y_m$. Além disso, C tem $(q - p) + (\ell - p)$ arestas, com ambas as parcelas ≥ 1 e não ambas iguais a 1 (senão $x_{p+1} = y_{p+1}$); logo C tem comprimento ≥ 3 e é um circuito em G, contradição.

(2) ⇒ (3): Seja G tal que (2) vale. Por hipótese G é conexo. Para toda aresta $e = \{u, v\} \in E$ temos que $P = u, v$ é o único caminho com extremos u e v, portanto, $G - e$ é desconexo.

(3) ⇒ (4): Seja G conexo minimal. Se G contém circuito, então para qualquer $e \in E(G)$ que pertença a um circuito temos que $G - e$ é conexo (Proposição 1.3), uma contradição. Agora, sejam $x, y \in V$ distintos e não adjacentes em G; como G é conexo existe um caminho, digamos $P = x, v_1, \ldots, v_k, y$, com extremos x e y. Em $G + xy$ temos o circuito $x, v_1, \ldots, v_k, y, x$ pois $k \geq 1$.

(4) ⇒ (1): Seja G acíclico maximal; resta provar que G é conexo. Suponha que não: existem $x \neq y$ sem caminho entre eles em G. Em particular $\{x, y\} \notin E(G)$, e por (4) o grafo $G + xy$ tem um circuito C. Como G é acíclico, $\{x, y\} \in E(C)$, e então $C - xy$ é um caminho de x a y em G, contradição. □

Como consequência do Teorema 4.1 podemos concluir que *uma árvore com n vértices é um grafo conexo com o menor número possível de arestas dentre todos os grafos conexos com n vértices.*

Subárvores geradoras

Lembremos que $H \subseteq G$ é um subgrafo ***gerador*** se $V(H) = V(G)$.

Definição 4.2: árvore geradora

Se uma árvore $T \subseteq G$ é um subgrafo gerador de G, dizemos que T é uma ***árvore geradora*** de G.

Todo grafo conexo tem uma árvore geradora dele: seja G um grafo conexo e seja $T \subseteq G$ um subgrafo gerador de G, conexo e com o menor número de arestas possível. Se T contém circuito então para qualquer aresta e de um circuito $T - e$ é conexo (Prop. 1.3) e tem menos arestas que T, uma contradição. Desse fato, podemos concluir que a recíproca do lema 4.1 do seguinte modo: suponha que G é conexo, com n vértices e $n - 1$ arestas. Seja $T \subseteq G$ uma árvore geradora de G. Como acabamos de mostrar T tem $n - 1$ arestas, logo $T = G$, portanto G é uma árvore.

Teorema 4.2

Um grafo com n vértices é uma árvore se, e somente se, é conexo e o número de arestas é $n - 1$. □

Um algoritmo que recebe um grafo conexo e determina uma árvore geradora do grafo conexo é fácil e rápido e já sabemos como fazer: é o subgrafo induzido pelo conjunto de arestas $M = \{\{v, p[v]\} : v \in V(G) \text{ e } p[v] \neq \text{nil}\}$, onde o vetor $p[\ ]$ é determinado pela execução de uma algoritmo de busca (e.g., Algoritmo 3).

O exercício 59 item 4), baseado no próximo resultado, também nos dá um modo de construir uma árvore geradora de G: começando com $U = \{v\}$, tomamos sucessivamente um extremo fora de U de uma aresta em $\nabla_G(U)$ e acrescentamos o a U.

Proposição 4.2

Um grafo G é conexo se e somente se $\nabla_G(U)) \neq \emptyset$ para todo subconjunto próprio e não-vazio de vértices $U \subsetneq V(G)$.

Demonstração. Seja G um grafo conexo. Se G é conexo e admite um subconjunto próprio e não-vazio $U \subset V(G)$, então existe em G um caminho

$$u = x_0, x_1, \ldots, x_k = w$$

para algum $u \in U$ e algum $w \in \overline{U}$. Tome $m \coloneqq \max\{j : x_j \in U\}$ e temos $0 \leq m < k$ pela escolha de u e de w, logo $\{x_m, x_{m+1}\} \in \nabla_G(U)$.

Para a recíproca, considere um grafo G tal que para todo subconjunto próprio e não-vazio de vértices $U \subsetneq V(G)$ vale $\nabla_G(U)) \neq \emptyset$.

Se G não é conexo, então existem vértices u e w de G sem nenhum caminho entre eles. Defina U como a componente conexa de u. Logo $U \neq \emptyset$, e $w \notin U$, ou seja, $U \subsetneq V(G)$. Pela hipótese, $\nabla_G(U) \neq \emptyset$, e portanto existe uma aresta $\{a, b\} \in E(G)$ com $a \in U$ e $b \in \overline{U}$. Como $a \in U$, existe um caminho P de u a a, portanto, P, b é um caminho de u a b, logo $b \in U$. Assim, $b \in U$ e $b \in \overline{U}$ é uma contradição. Logo G é conexo. □

Exercícios

Exercício 61. Determine o número de arestas de uma floresta com n vértices e com k componentes conexas, em função de n e k.

Exercício 62. Desenhe todas as árvores não-isomorfas com 5 vértices e todas as árvores não-isomorfas com 7 vértices e com grau máximo pelo menos 4.

Exercício 63. Mostre que toda árvore T tem pelo menos $\Delta(T)$ folhas.

Exercício 64. Prove que o conjunto de arestas $\{p[v], v\}$ definido por uma busca em profundidade rotulada induz uma árvore.

Exercício 65. Seja G um grafo. Mostre que as seguintes afirmações são equivalentes:

(a) G é conexo e $|E(G)| = |V(G)| - 1$;

(b) $|E(G)| = |V(G)| - 1$ e G não contém circuito.

(c) G é uma árvore;

Exercício 66. Considere uma floresta F com n vértices e m arestas.

(a) Qual é o número máximo de vértices num componente conexo de F?

(b) Mostre que há pelo menos $\max\{n-2m, 0\}$ vértices de grau zero.

(c) Mostre que se $m < n/2$ então resta pelo menos um vértice de grau zero.

Exercício 67. Seja T uma árvore, u e v dois vértices não adjacentes em T e $a \in E(T)$ uma aresta do circuito em $T + uv$. Mostre que

$$T + uv - a := \big(V(T), E(T) \cup \{\{u,v\}\} \setminus \{a\}\big)$$

é uma árvore. Dizemos que essa árvore foi obtida por uma ***operação elementar*** a partir da árvore T.

Exercício 68. Sejam $T_1, T_2 \subset G$ árvores geradoras distintas de um grafo conexo G. Mostre que T_2 pode ser obtida a partir de T_1 através de uma sequência de operações elementares.

Exercício 69. Em um grafo conexo G dizemos que uma aresta e é uma ***ponte*** se $G - e$ é desconexo. Prove uma ponte pertence a todas as árvores geradoras de G.

Exercício 70. Seja T uma árvore de ordem t. Mostre que qualquer grafo com grau mínimo pelo menos $t-1$ contém um subgrafo isomorfo a T.

Exercício 71 (*precisa de álgebra linear*). Seja G um grafo conexo com n vértices e m arestas, $m \geq n-1$, e seja $T \subset G$ uma árvore geradora de G. Como $|E(T)| = n-1$, há exatamente $m-n+1$ arestas em $E(G) \setminus E(T)$; denote-as por $e_1, \dots, e_{m-n+1}$. Para cada i, o grafo $T + e_i$ contém um *único* circuito C_i, formado por e_i e pelo único caminho em T entre as pontas de e_i. Chamamos $C_1, \dots, C_{m-n+1}$ de ***circuitos fundamentais de G com respeito a T***.

Dados $A, B \subseteq E(G)$, denote por $A \Delta B$ a diferença simétrica. Considere o conjunto $\wp(E(G))$ das partes de $E(G)$ sobre o corpo $\mathbb{F}_2 = \{0,1\}$, com *soma* $A \Delta B$ e *produto por escalar* $0 \cdot A = \varnothing$, $1 \cdot A = A$.

1. Verifique que $\wp(E(G))$ é um espaço vetorial sobre $\mathbb{F}_2$.
2. Prove que para todo circuito $C \subset G$ existem $b_1, \dots, b_{m-n+1} \in \mathbb{F}_2$ tais que $E(C) = b_1 \cdot E(C_1) \Delta \cdots \Delta\, b_{m-n+1} \cdot E(C_{m-n+1})$.
3. Seja $\mathcal{C}(G)$ o subespaço gerado pelos conjuntos de arestas dos circuitos de G. Prove que $E(C_1), \dots, E(C_{m-n+1})$ formam uma base de $\mathcal{C}(G)$ e conclua que $\dim \mathcal{C}(G) = m-n+1$.

Exercício 72 (*precisa de álgebra linear*). Seja G um grafo conexo com n vértices e m arestas, e seja $T \subset G$ uma árvore geradora de G. Sejam $f_1, \dots, f_{n-1}$ as arestas de T. Para cada j, o grafo $T - f_j$ tem exatamente duas componentes; seja S_j o conjunto de vértices de uma delas e defina

$$D_j = \nabla(S_j).$$

Observe que $D_j \cap E(T) = \{f_j\}$, pois f_j é a única aresta de T entre S_j e $V(G) \setminus S_j$. Chamamos $D_1, \dots, D_{n-1}$ de ***cortes fundamentais de G com respeito a T***.

Considere o espaço vetorial $\wp(E(G))$ sobre $\mathbb{F}_2$ com a soma $A \nabla B$ (exercício anterior).

1. Prove que $\nabla(S)\Delta\nabla(S') = \nabla(S\Delta S')$ para quaisquer $S, S' \subseteq V(G)$. Conclua que o conjunto $\mathcal{D}(G)$ dos cortes de G (incluindo $\nabla(\varnothing) = \varnothing$) é um subespaço de $\wp(E(G))$, chamado ***espaço de cortes*** de G.
2. Prove que para todo corte $D = \nabla(S)$ de G existem $b_1, \dots, b_{n-1} \in \mathbb{F}_2$ tais que $D = b_1 \cdot D_1 \nabla \cdots \nabla\, b_{n-1} \cdot D_{n-1}$.
3. Prove que $D_1, \dots, D_{n-1}$ formam uma base de $\mathcal{D}(G)$ e conclua que $\dim \mathcal{D}(G) = n - 1$.
4. Mostre que $\dim \mathcal{C}(G) + \dim \mathcal{D}(G) = m$, onde $\mathcal{C}(G)$ é o espaço de circuitos do exercício anterior. *Dica*: mostre que, para $e_i \notin E(T)$, vale $f_j \in E(C_i)$ se e somente se $e_i \in D_j$, e que todo circuito intersecta todo corte em um número par de arestas.

Exercício 73 (***Shannon switching game*** *(1952)*). O seguinte jogo foi proposto por Claude Shannon para ser jogado numa grade retangular; descrevemos a versão em um grafo qualquer. O jogo é jogado em um grafo finito G com dois vértices distinguidos, s e t, por dois jogadores, o Positivo (*Short*) e o Negativo (*Cut*), que jogam alternadamente.

No início todas as arestas estão *livres*. Na sua vez, o Positivo *fixa* uma aresta livre e, na sua vez, o Negativo *remove* uma aresta livre; uma aresta fixada ou removida permanece assim até o fim do jogo. O Positivo vence se, em algum momento, existe um caminho de s a t formado por arestas fixadas; caso contrário, isto é, se todas as arestas forem marcadas sem que tal caminho exista, vence o Negativo. Em particular, não há empates. Dizemos que o jogo (G, s, t) é

positivo se o Positivo tem estratégia vencedora, não importando quem começa;

negativo se o Negativo tem estratégia vencedora, não importando quem começa;

neutro se vence quem joga primeiro.

Observe que um jogador nunca é prejudicado por uma jogada extra; logo, se um jogador vence jogando em segundo, vence também jogando em primeiro. Disso decorre que todo jogo é positivo, negativo ou neutro. Prove que se G tem um subgrafo H com $s, t \in V(H)$ que admite duas árvores geradoras (de H) disjuntas nas arestas, então o jogo (G, s, t) é positivo. **Dica.** Seja (T_1, T_2) um par de árvores geradoras de H. Esse par é dito *bom* se num dado momento do jogo se nenhuma aresta de $T_1 \cup T_2$ foi removida e toda aresta de $E(T_1) \cap E(T_2)$ já foi fixada. O Positivo deve manter um par bom antes de cada jogada do Negativo. Se o Negativo remove uma aresta $f \in E(T_1)$, o grafo $T_1 - f$ tem 2 componentes e, como T_2 é conexo, a Prop. 4.2 fornece uma aresta $g \in E(T_2)$ ligando as duas componentes; o Positivo fixa g. Cuide dos casos em que f está fora de $T_1 \cup T_2$, em que g já está fixada, e de quem começa o jogo. Ao final, explique por que o Positivo tem um caminho de s a t de arestas fixadas.

Lehman (1964) de forma teórica e genérica, na teoria de matroides, provou que uma estratégia vencedora para o Positivo existe; o jogo de Shannon depende da estrutura de independência linear dos seus circuitos. Bruno e Weinberg (1970) simplificaram e traduziram os conceitos abstratos de matroides em estruturas puras de teoria dos grafos, oferecendo um algoritmo eficiente (de tempo polinomial) para determinar quem vence. A metodologia impactou o projeto de circuitos elétricos e redes de comutação além de servir de base para algoritmos de otimização de fluxos em redes e conectividade em grafos.

4.1 Árvores geradoras de custo mínimo em grafos com pesos nas arestas

Seja $G = (V, E, \rho)$ um grafo conexo com pesos $\rho : E \to \mathbb{R}$ nas arestas. Para $F \subseteq E$ definimos o ***custo*** de F por

$$c(F) = \sum_{e \in F} \rho(e),$$

e o custo de um subgrafo $H \subseteq G$ é $c(H) := c(E(H))$. O problema que nos interessa a partir de agora é: dado $G = (V, E, \rho)$, determinar uma árvore geradora T de G tal que

$$c(T) = \min\{c(T') : T' \text{ é árvore geradora de } G\}.$$

Uma tal árvore é chamada de ***árvore geradora mínima*** de G. Ela existe, pois G é conexo (logo tem alguma árvore geradora) e tem apenas um número finito delas; porém pode não ser única.

Observação 4.1: um cuidado com pesos negativos

Se $\rho \geq 0$, o subgrafo gerador conexo de menor custo pode ser tomado como uma árvore: se ele contém um circuito, removemos uma aresta do circuito, o que mantém a conexidade e não aumenta o custo. Com pesos negativos isso falha: num triângulo com pesos -1, o triângulo inteiro custa -3, mais barato que cada árvore geradora cujo custos é -2.

Problema 4.1: Árvore Geradora Mínima (AGM)

Dado $G = (V, E, \rho)$ conexo, computar uma subárvore geradora de G de custo mínimo.

A seguir apresentamos os algoritmos de Dijkstra–Jarník–Prim[1] e de Kruskal para determinar uma árvore geradora mínima. Ambos são algoritmos gulosos: a cada passo escolhem, dentre as opções disponíveis, a que parece melhor naquele momento (ótimo local), sem reconsiderar escolhas anteriores. Em geral essa estratégia não produz uma solução ótima (ótimo global), mas para árvores geradoras mínimas ela produz, e é isso que provaremos.

1: O algoritmo de Dijkstra–Jarník–Prim foi descoberto mais de uma vez, em contextos distintos. Foi descrito primeiro por Vojtěch Jarník (1930), que considerou complicada a solução de Otakar Borůvka (1926), motivada pela eletrificação rural da Morávia. Robert Prim o redescobriu em 1957 nos Bell Labs, ao tratar de redes de telefonia, e Edsger Dijkstra em 1959, a pedido de engenheiros do Mathematisch Centrum, em Amsterdã, interessados em reduzir a fiação de um computador.

4.1.1 Algoritmo de Dijkstra–Jarník–Prim para árvore geradora mínima

O algoritmo de Dijkstra–Jarník–Prim recebe um grafo conexo $G = (V, E, \rho)$ e devolve $S \subseteq E$ tal que $G[S]$ é uma árvore geradora mínima de G. O algoritmo parte de um conjunto unitário, digamos $U = \{v\}$ para algum $v \in V$, e a cada rodada o algoritmo acrescenta a U o vértice de $\overline{U}$ que é extremo de uma aresta de menor custo no corte $\nabla_G(U)$ (que

existe pela Proposição 4.2 e empates são desfeitos arbitrariamente), até que $U = V$. No final, as arestas escolhidas induzem uma árvore geradora mínima.

Algoritmo 7: Dijkstra–Jarník–Prim(G)

Dado : um grafo conexo G com pesos nas arestas.
Devolve : uma árvore geradora mínima.

1 $U, S = \emptyset$;
2 escolha $v \in V(G)$ arbitrário;
3 $U \leftarrow \{v\}$;
4 **enquanto** $U \neq V(G)$ **faça**
5 escolha $\{u, w\} \in \nabla_G(U)$ de peso mínimo;
6 insira $\{u, w\}$ em S;
7 $v \leftarrow \{u, w\} \cap \overline{U}$;
8 insira v em U;
9 **retorne** (V, S).

Notemos que esse algoritmo é uma ligeira modificação do Algoritmo 5 (pela Proposição 4.2 os laços desses dois algoritmos têm a mesma condição).

Análise e correção do algoritmo de Dijkstra–Jarník–Prim.

Usaremos a técnica do invariante do laço.

Proposição 4.3

Seja $n = |V(G)|$ e denote por U_k e S_k os valores de U e S após a k-ésima iteração do `enquanto` da linha 4 (com $U_0 = \{v\}$ e $S_0 = \emptyset$). Para $0 \leq k \leq n - 1$, o par $T_k := (U_k, S_k)$ é uma árvore e é subgrafo de alguma árvore geradora mínima de G.

Demonstração. Por indução em k. Para $k = 0$, T_0 é a árvore trivial com o vértice v, subgrafo de toda árvore geradora de G; em particular de uma mínima, que existe pois G é conexo e tem finitas árvores geradoras.

Seja $1 \leq k \leq n - 1$ e suponha que T_{k-1} é uma árvore contida em uma árvore geradora mínima T. Na k-ésima iteração o algoritmo escolheu (linha 5) uma aresta $\{u, w\} \in \nabla_G(U_{k-1})$ de peso mínimo no corte, com $u \in U_{k-1}$ e $w \notin U_{k-1}$, e fez $U_k = U_{k-1} \cup \{w\}$ e $S_k = S_{k-1} \cup \{\{u, w\}\}$. Como w é um vértice novo ligado a T_{k-1} apenas por $\{u, w\}$, T_k é conexo e nenhum circuito passa por w (que tem grau 1); logo T_k é uma árvore.

Se $\{u, w\} \in E(T)$, então $T_k \subseteq T$ e nada há a provar. Suponha $\{u, w\} \notin E(T)$. Pelo Teorema 4.1, item (2), existe um único caminho P em T de u a w. Como $u \in U_{k-1}$ e $w \notin U_{k-1}$, o argumento da primeira

metade da prova da Proposição 4.2 mostra que P contém uma aresta $e \in \nabla_T(U_{k-1})$. Temos $e \neq \{u, w\}$ (pois $\{u, w\} \notin E(T)$) e

(a) $T' := T - e + uw$ é uma árvore geradora de G: como $e \in E(P)$, os vértices u e w estão em componentes distintas de $T - e$, que $\{u, w\}$ religa; T' é conexo e tem $n - 1$ arestas.

(b) $e \notin S_{k-1}$: toda aresta de S_{k-1} tem as duas pontas em U_{k-1}, e e tem uma ponta fora de U_{k-1}. Assim $S_{k-1} \subseteq E(T) \setminus \{e\} \subseteq E(T')$ e, como $\{u, w\} \in E(T')$, $T_k \subseteq T'$.

(c) Como $e \in \nabla_G(U_{k-1})$ e $\{u, w\}$ tem peso mínimo nesse corte, $\rho(\{u, w\}) \leq \rho(e)$, logo $c(T') = c(T) - \rho(e) + \rho(\{u, w\}) \leq c(T)$. Pela minimalidade de T, T' também é uma árvore geradora mínima.

Assim T_k é subgrafo da árvore geradora mínima T'. A proposição segue do Princípio da Indução Finita. □

Corolário 4.1

O algoritmo de Dijkstra–Jarník–Prim devolve o conjunto S de arestas de uma árvore geradora mínima de G.

Demonstração. Cada iteração acrescenta um vértice a U, logo o laço termina após $n - 1$ iterações com $U = V$. Pela proposição, (V, S) é uma árvore contida em uma árvore geradora mínima T. Ambas têm $n - 1$ arestas, portanto $(V, S) = T$. □

A complexidade de tempo do algoritmo de Dijkstra–Jarník–Prim depende da implementação do passo 5. Mantendo um vetor booleano que diz quais vértices estão em U, podemos testar em $O(1)$ se uma aresta pertence a $\nabla_G(U)$ e, a cada iteração, percorrer todas as arestas para encontrar uma de menor peso nesse corte, o que custa $O(|E|)$. Como há $|V| - 1$ iterações, o tempo total é $O(|V||E|)$. (Ordenar as arestas antes não melhora essa cota, pois a busca da primeira aresta do corte ainda pode percorrer todas elas.) Com mais cuidado, podemos melhorar o tempo para grafos esparsos: uma implementação com *heap* binária tem complexidade $O(|E| \log |V|)$.

Heap binária e análise de complexidade do algoritmo de Prim

Uma ***heap*** binária para mínimo é uma estrutura de dados que mantém um conjunto de elementos, cada um com uma *chave*, e permite acessar rapidamente o elemento de menor chave. Ela implementa uma ***fila de prioridade*** quando a menor chave é a de maior prioridade.

Formalmente, é uma árvore binária *quase completa*, isto é, com todos os níveis cheios, exceto possivelmente o último, que é preenchido da esquerda para a direita, e que satisfaz a ***propriedade de heap***: *a chave no nó pai é menor ou igual à chave nos seus filhos.* Em consequência, a raiz contém sempre o elemento de menor chave.

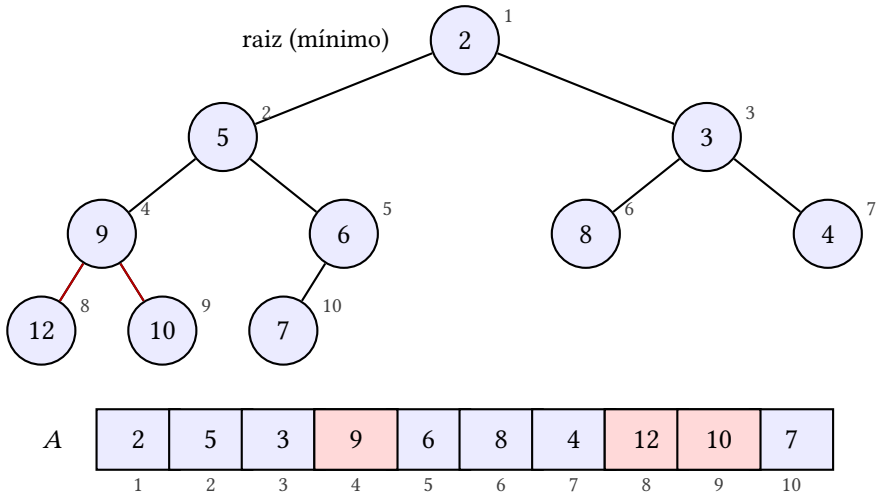


Figura 4.2: *Heap* com 10 elementos e sua representação em vetor. Para o nó na posição i, o pai está em $\lfloor i/2 \rfloor$ e os filhos em $2i$ e $2i+1$. Em destaque, o nó $i = 4$ e seus filhos 8 e 9.

Por ser quase completa, a árvore é armazenada em um vetor A, sem ponteiros. Para o nó na posição i:

- pai: $\lfloor i/2 \rfloor$;
- filho esquerdo: $2i$;
- filho direito: $2i + 1$.

A altura da árvore é $\lfloor \log_2 n \rfloor$, que dá custo logarítmico das operações (veja a Tabela 4.1).

Tabela 4.1: Operações do *Heap* Mínimo e seus custos. Depois de uma operação pode ser necessário trocar chaves de posição para refazer a propriedade de *heap*. Isso significa *subir* uma chave trocando de posição como o pai ou *descer* uma chave trocando de posição como o menor filho. Para que `diminuir-chave` seja eficiente, é preciso saber em que posição do vetor está cada elemento. Isso se resolve mantendo um vetor auxiliar pos$[v]$, atualizado a cada troca.

Operação	Descrição	Custo
`min`	devolve $A[1]$	$O(1)$
`inserir`(x)	coloca x na última posição e refaz *heap*	$O(\log n)$
`extrair-min`	remove $A[1]$, move o último elemento para a raiz e refaz *heap*	$O(\log n)$
`diminuir-chave`(i, k)	reduz a chave da posição i para k e refaz *heap*	$O(\log n)$
`construir-heap`	organiza um vetor arbitrário em forma de *heap*	$O(n)$

As operações preservam a propriedade de *heap* da seguinte forma.

- *Subir uma chave na árvore* — ocorre nas operações `inserir` e `diminuir-chave`: antes da operação a propriedade de *heap* vale; depois, a única violação possível é entre o nó x modificado e seu pai. Se tal violação ocorre, trocamos os dois. Após a troca, x é menor que o antigo pai, que era menor ou igual ao irmão de x e aos filhos de x, de modo que a única violação possível é, de novo, entre x e seu novo pai, um nível acima. O processo termina na raiz, ou quando não há mais violação, em no máximo $\lfloor \log_2 n \rfloor$ passos.

▶ *Descer uma chave na árvore* — ocorre nas operações `extrair-min` e `construir-heap`): aqui supomos que as subárvores dos filhos de x já são *heap*; então a única violação possível é entre um nó x e seus filhos. Se x é maior que algum filho, trocamos x com o *menor* dos filhos c; assim, o novo pai c é menor que x e menor ou igual ao outro filho. A única violação possível passa a ser entre x e seus novos filhos, um nível abaixo. O processo termina quando não há violação ou ao chegar em uma folha. Depois dele, a subárvore de x é uma *heap*.

Dado um vetor com n elementos em ordem qualquer, construímos nesse vetor uma *heap* organizando-o de baixo para cima. Os nós internos ocupam as posições $1, \ldots, \lfloor n/2 \rfloor$ e as folhas ocupam as demais, que já são *heaps*. Processamos cada $i = \lfloor n/2 \rfloor, \ldots, 1$, nessa ordem, aplicando "*descer*" ao nó i; ao processá-lo, as subárvores de $2i$ e $2i+1$ já são *heap*, e depois a subárvore de i também é. No final, o vetor satisfaz a propriedade de *heap*.

A ***altura de um nó*** é o maior número de arestas de um caminho descendente dele até uma folha. Descer um nó de altura h custa $O(h)$ e há no máximo $\lceil n/2^{h+1} \rceil \le n/2^h$ nós de altura h (veja Cormen et al. (2012); a desigualdade vale para $h \le \lfloor \log_2 n \rfloor$, pois $n/2^{h+1} \ge 1/2$). Como $\sum_{h \ge 0} h/2^h = 2$, o custo total para organizar um vetor qualquer numa *heap* é[2]

$$\le \sum_{h=0}^{\lfloor \log_2 n \rfloor} \frac{n}{2^h} O(h) = O\left(n \sum_{h \ge 0} \frac{h}{2^h} \right) = O(n).$$

2: A série $\sum_{h=0}^{\infty} x^h$ é o limite, quando $n \to \infty$, das somas parciais $S_n = \sum_{h=0}^{n} x^h$, que são somas de P.G. Se $|x| < 1$, o limite de S_n existe e é $(1-x)^{-1}$. Como séries de potências podem ser derivadas termo a termo dentro do raio de convergência, obtemos $\sum_{h=1}^{\infty} h\, x^{h-1} = (1-x)^{-2}$ e, multiplicando por x, $\sum_{h=0}^{\infty} h\, x^h = x\,(1-x)^{-2}$.

No algoritmo de Dijkstra–Jarník–Prim, o vetor A da *heap* guarda vértices: $A[i]$ é o vértice na posição i da árvore, e a prioridade é dada pelo vetor chave, que satisfaz a propriedade de *heap*, isto é,

$$\text{chave}[A[\,\lfloor i/2 \rfloor\,]] \le \text{chave}[A[i]].$$

Assim, `extrair-min`(A) devolve $A[1]$, o vértice de menor chave, move o último elemento para a raiz, desce e faz $\text{pos}[w] \leftarrow$ `nil`. Nas trocas de posições (subir ou descer) atualiza-se pos dos dois vértices. O vetor auxiliar pos satisfaz $A[\text{pos}[z]] = z$ para $z \in A$ e $\text{pos}[z] =$ `nil` para os vértices já extraídos $z \notin A$; ele permite testar $z \in A$ e localizar z em $O(1)$.

Para `diminuir-chave`(A, z, k) e para o teste "$z \in A$" é preciso ir do vértice à sua posição. Para isso, o vetor auxiliar pos satisfaz $A[\text{pos}[z]] = z$ para $z \in A$ e $\text{pos}[z] =$ `nil` para os vértices já extraídos $z \notin A$; ele permite testar $z \in A$ e localizar z em $O(1)$.

O ponto crítico do algoritmo de Dijkstra–Jarník–Prim é encontrar, a cada iteração, a aresta mais barata que sai de U. Para isso, guardamos os vértices fora de U em uma *heap* A ordenada por `chave`, isto é,

$U = V \setminus A$, de modo que, para $z \in A$,

$$\text{chave}[z] = \min\{\rho(\{u,z\}) : u \in U,\ \{u,z\} \in E(G)\}$$

(ou ∞ se z não tem vizinho em U) e $\text{pai}[z]$ é um vértice $u \in U$ que realiza esse mínimo. Esse invariante vale após cada iteração do laço da linha 8 do Algoritmo 8, a partir da primeira (que extrai o vértice inicial v, de chave 0 por convenção, e faz $U = \{v\}$). De fato, quando w entra em U só mudam as chaves dos vizinhos $z \in A$ de w, e o teste $\rho(\{w,z\}) < \text{chave}[z]$ as atualiza para o mínimo. Consequentemente, a partir da segunda iteração, seja U o conjunto antes da extração; como G é conexo e $\varnothing \neq U \subsetneq V$, a Proposição 4.2 dá $\nabla_G(U) \neq \varnothing$, e o vértice w extraído tem $\text{chave}[w] < \infty$: w é o extremo em $\overline{U}$ de uma aresta $\{\text{pai}[w], w\}$ de menor peso em $\nabla_G(U)$, que é exatamente o passo da linha 5.

Algoritmo 8: Dijkstra–Jarník–Prim—heap(G)

Dado : um grafo conexo $G = (V, E, \rho)$ com pesos nas arestas.

Devolve : uma árvore geradora mínima.

```
1  S ← ∅;
2  escolha v ∈ V arbitrário;
3  para cada z ∈ V faça
4  |  chave[z] ← ∞;
5  |  pai[z] ← nil;
6  chave[v] ← 0;
7  A ← construir-heap(V)
8  enquanto A ≠ ∅ faça
9  |  w ← extrair-min(A)
10 |  se pai[w] ≠ nil então insira {pai[w], w} em S ;
11 |  para cada z ∈ N_G(w) faça
12 |  |  se z ∈ A e ρ({w,z}) < chave[z] então
13 |  |  |  pai[z] ← w;
14 |  |  |  diminuir-chave(A, z, ρ({w,z}));
15 retorne (V, S)
```

Supomos o *grafo dado por listas de adjacência* e mantemos $\text{pos}[z]$ (com `nil` para os vértices já extraídos), o que permite testar $z \in A$ em $O(1)$. A inicialização e a construção da *heap* custam $O(|V|)$. As $|V|$ chamadas a `extrair-min` custam $O(|V| \log |V|)$. Cada aresta $\{w,z\}$ é examinada duas vezes (a partir de cada extremo), mas só provoca uma chamada a `diminuir-chave` quando o outro extremo ainda está em A, ou seja, no máximo uma vez, o que dá $O(|E|)$ chamadas de custo $O(\log |V|)$ cada. Como G é conexo, $|V| \leq 2|E|$ (para $|V| \geq 2$), e todos os termos ficam absorvidos por

$$O(|E| \log |V|).$$

Para comparação, se a *heap* for substituída por um vetor simples, `extrair-min` custa $O(|V|)$ e `diminuir-chave` custa $O(1)$, resultando em $O(|V|^2)$. Essa versão é melhor em grafos densos, ou seja, grafos tais que $|E| = \Theta(|V|^2)$, enquanto a *heap* binária é melhor em grafos esparsos.

Teorema 4.3

O algoritmo Dijkstra–Jarník–Prim—heap(G) computa uma árvore geradora mínima de um grafo conexo (V, E) com pesos nas arestas em tempo $O(|E| \log |V|)$.

4.1.2 Algoritmo de Kruskal para árvore geradora mínima

O algoritmo de Kruskal, proposto por Joseph Kruskal em 1956, também é bastante simples: a cada passo escolhe-se a aresta mais barata dentre as que ainda não foram examinadas, desde que ela não forme circuito com as arestas já escolhidas. Arestas de mesmo peso podem ser ordenadas arbitrariamente.

Algoritmo 9: Kruskal(G)

Dado : um grafo conexo $G = (V, E)$ com pesos nas arestas.

Devolve : árvore geradora de custo mínimo.

```
1 S = ∅;
2 F ← lista das arestas de G em ordem crescente de peso;
3 para cada e ∈ F faça
4     se (V, S ∪ {e}) é acíclico então insira e em S ;
5 retorne (V, S);
```

Correção do algoritmo de Kruskal.

Vamos provar que o algoritmo de Kruskal determina uma árvore geradora de custo mínimo de um grafo conexo e com pesos nas arestas. A prova vale para qualquer ordem de desempate entre arestas de mesmo peso.

Seja $G = (V, E, \rho)$ conexo, S o conjunto devolvido e $(e_1, \dots, e_k)$ as arestas de S na ordem em que foram inseridas. Como S começa vazio e uma aresta só é inserida se S continua acíclico (linha 4), o grafo (V, S) é acíclico. Como G é conexo e tem apenas um número finito de árvores geradoras, existe uma árvore geradora mínima; seja T uma delas com o maior número possível de arestas em comum com S. Mostraremos que $E(T) = S$.

Vamos provar que $S \subseteq E(T)$. Suponha que não e seja $m := \min\{j : e_j \in S \setminus E(T)\}$. Então $e_1, \dots, e_{m-1} \in E(T)$. Pelo Teorema 4.1, $T + e_m$ contém um circuito C, que contém e_m, e $C - e_m$ é o único

caminho P de T entre as pontas de e_m. Como (V, S) é acíclico, C tem uma aresta $f \notin S$; em particular $f \neq e_m$, logo $f \in E(P) \subseteq E(T)$. Considere

$$R \coloneqq \{e_1, \dots, e_{m-1}\} \cup \{f\} \subseteq E(T),$$

que, por estar contido em uma árvore, é acíclico.

Afirmamos que f vem depois de e_m na fila F. Se viesse antes, no momento em que f foi examinada o conjunto S' corrente estaria contido em $\{e_1, \dots, e_{m-1}\}$, e como $S' \cup \{f\} \subseteq R$ é acíclico, f teria sido escolhida. Como F está em ordem crescente de peso, $\rho(f) \geq \rho(e_m)$.

Como $f \in E(P)$, as pontas de e_m estão em componentes distintas de $T - f$, e portanto $T' \coloneqq T - f + e_m$ é uma árvore geradora de G, com

$$c(T') = c(T) - \rho(f) + \rho(e_m) \leq c(T).$$

Pela minimalidade de T, $c(T') = c(T)$, e T' também é mínima. Como $f \notin S$ e $e_m \in S \setminus E(T)$, T' tem uma aresta a mais em comum com S do que T, contrariando a escolha de T. Portanto $S \subseteq E(T)$.

Vamos provar que $E(T) \subseteq S$. Suponha $e \in E(T) \setminus S$. No momento em que e foi examinada, o conjunto S' corrente satisfazia $S' \subseteq S \subseteq E(T)$, logo $S' \cup \{e\} \subseteq E(T)$ é acíclico e o algoritmo teria inserido e em S, uma contradição.

Assim $S = E(T)$, isto é, $(V, S) = T$ é uma árvore geradora mínima. Com isso provamos

Teorema 4.4: correção do algoritmo de Kruskal

Para todo grafo conexo $G = (V, E, \rho)$, o algoritmo de Kruskal devolve (V, S) tal que (V, S) é uma árvore geradora mínima de G. □

Análise da complexidade do algoritmo de Kruskal

Notemos que enquanto o conjunto S evolui de $\varnothing$ até definir uma árvore geradora ele sempre define uma subfloresta geradora de G, e seus componentes formam uma partição de V. Para $e = \{x, y\} \notin S$ vale

$$(V, S \cup \{e\}) \text{ acíclico} \iff x \text{ e } y \text{ estão em componentes distintos de } (V, S).$$

De fato, se estão no mesmo componente, um caminho de x a y em (V, S) junto com e é um circuito. Se estão em componentes distintos e houvesse um circuito em $(V, S \cup \{e\})$, ele conteria e (pois (V, S) é acíclico) e o restante dele seria um caminho de x a y em (V, S), absurdo.

O teste na linha 4 acima verifica se a aresta $e = \{x, y\}$ não liga vértices do mesmo conjunto (caso contrário, teremos um circuito) e, nesse caso, a inserção na linha 4 passa a vir acompanhada da união

do conjunto onde está o vértice x com o conjunto do vértice y.

Assim, precisamos de estruturas de dados que, dinamicamente, representem e manipulem conjuntos disjuntos de modo eficiente. Estruturas como essa são conhecidas na literatura como estruturas para união-e-busca (*union-find*) ou *estruturas de dados para conjuntos disjuntos*.

Essas estruturas mantêm uma família de subconjuntos disjuntos com um elemento de cada subconjunto eleito como o *representante* do subconjunto e permitem as operações

`cria`(x) cria o conjunto unitário $\{x\}$, com representante x;
`busca`(x) devolve o representante do conjunto ao qual x pertence;
`uniao`(x, y) substitui os conjuntos que contêm x e y pela união desses conjuntos (e determina um representante para essa união).

Reescrevendo o algoritmo de Kruskal com as considerações feitas acima sobre estruturas de dados para conjuntos disjuntos:

Algoritmo 10: Kruskal(G)

Dado : um grafo conexo $G = (V, E, \rho)$ com pesos nas arestas.
Devolve : árvore geradora de custo mínimo.

```
1 S = ∅;
2 F ← lista das arestas de G em ordem crescente de peso;
3 para cada v ∈ V faça cria(v);
4 para cada {u, v} ∈ F faça
5     se busca(u) ≠ busca(v) então
6         insira {u, v} em S;
7         uniao(u, v);
8 retorne (V, S);
```

Entre as estruturas mais eficientes para conjuntos disjuntos destaca-se a representação por floresta com as heurísticas ***união por rank*** e ***busca com compressão de caminhos*** (Seção 4.1.3): m operações, das quais as n primeiras são `cria`, custam $O(m\,\alpha(m, n))$ Cormen et al. (2012), e $\alpha(m, n) \leq \alpha(n, n) \leq 4$ para qualquer n prático.

No Kruskal há $|V|$ operações `cria`, no máximo $2|E|$ operações `busca` e no máximo $|V| - 1$ operações `uniao`, no total menos que $2(|E| + |V|) = O(|E|)$, pois G é conexo e portanto $|V| \leq |E| + 1$. Com essa estrutura, o custo dessas operações é $O(|E|\,\alpha(|E|, |V|))$, que é dominado pelo custo da ordenação das arestas. Para um grafo simples, $|E| \leq |V|^2$, e o tempo total é $O(|E| \log |E|) = O(|E| \log |V|)$ (para $|E| \geq 2$).

Usamos um vetor C indexado por $V = \{1, \ldots, n\}$ tal que dois elementos $x, y \in V$ estão no mesmo conjunto se e somente se $C[x] = C[y]$. Escolhemos como representante o *menor elemento* do conjunto, isto

é, $C[x]$ é o menor elemento do conjunto de x (esse é o invariante mantido pelas operações abaixo).

Algoritmo 11: cria(x)

```
1 C[x] ← x;
```

Algoritmo 12: busca(x)

```
1 retorne C[x];
```

Algoritmo 13: uniao(x, y)

```
1 a ← busca(x);
2 b ← busca(y);
3 k ← min{a, b};
4 para cada i ∈ {1, ..., n} faça
5     se C[i] = a  ou  C[i] = b então  C[i] ← k ;
```

O invariante é preservado: k é o menor elemento da união, pois a e b são os menores elementos dos dois conjuntos, e o laço atribui k a $C[i]$ exatamente para os i pertencentes a esses dois conjuntos, sem alterar os demais. (É importante guardar a e b antes do laço: os valores $C[x]$ e $C[y]$ mudam durante ele.) Note que `uniao` supõe que todos os elementos já foram criados, pois o laço lê $C[i]$ para todo $i \in \{1, \ldots, n\}$.

Lema 4.1

Suponha que as n primeiras operações são `cria` (uma para cada elemento de V) e que `uniao` é sempre chamada com elementos de conjuntos distintos. Então m operações, das quais u são `uniao`, custam $O(m + nu)$. Em particular, como $u \leq n - 1$, o tempo é $O(m + n^2)$.

Demonstração. As operações `cria` e `busca` custam $O(1)$ e `uniao` custa $\Theta(n)$, pois o laço percorre sempre as n posições de C. Cada `uniao` reduz em 1 o número de conjuntos da partição, que começa com n, logo $u \leq n - 1$. Em particular, como $u \leq n - 1$, o tempo é $O(m + n^2)$. □

A cota $O(mn)$ é justa quando $m = \Theta(n)$ (por exemplo, n operações `cria` seguidas de $n - 1$ uniões encadeadas custam $\Theta(n^2)$), mas não para $m \gg n$, pois as operações `busca` extras custam $O(1)$ cada.

Corolário 4.2

Para todo grafo simples conexo G com pesos nas arestas e $|E| \geq 2$, o tempo do algoritmo Kruskal(G) com vetor para representar conjuntos disjuntos é $O(|E| \log |E| + |V|^2)$.

Demonstração. A ordenação custa $O(|E| \log |E|)$. Pelo lema, as $|V|$ operações `cria`, no máximo $2|E|$ operações `busca` e no máximo

$|V| - 1$ operações `uniao` custam $O(|V| + |E| + |V|^2)$. Como $|E| = O(|E| \log |E|)$ para $|E| \geq 2$ e $|V| \leq |V|^2$, o total é $O(|E| \log |E| + |V|^2)$. □

4.1.3 Estruturas de dados para representar conjuntos disjuntos

Nesta seção veremos estruturas mais eficientes para implementar as três operações utilizadas no algoritmo de Kruskal. Para compará-las, analisamos o tempo total gasto por uma sequência de m operações, e não o pior caso de cada operação isolada, pois este pode superestimar bastante o custo médio das operações da sequência (análise amortizada). Supomos que as n primeiras operações são `cria`, uma para cada elemento do universo $\mathcal{U} = \{1, \ldots, n\}$; em particular, $m \geq n$.

O problema é representar dinamicamente uma família de subconjuntos disjuntos $\mathcal{F} = \{A_1, A_2, \ldots, A_k\}$ tomados de um universo $\mathcal{U}$, tal que cada conjunto é identificado por um *representante*, que é um elemento do próprio conjunto. Enquanto um conjunto não for modificado por uma união, a resposta à pergunta sobre o seu representante deve ser sempre a mesma.

Essas estruturas devem comportar as operações a seguir sobre $\mathcal{F}$:

`cria`(x) para x que ainda não pertence a nenhum conjunto, acrescenta $\{x\}$ à família $\mathcal{F}$;

`busca`(x) devolve o representante do conjunto ao qual x pertence;

`uniao`(x, y) para x e y em conjuntos distintos A_x e A_y, remove A_x e A_y de $\mathcal{F}$, acrescenta $A_x \cup A_y$ e escolhe um representante para ele.

Representação usando listas ligadas

Cada conjunto é dado por uma lista ligada cujo primeiro nó é o seu representante, e um vetor $P[1..n]$ guarda, para cada x, um ponteiro para o nó de x (`nil` se x ainda não foi criado). Cada nó tem os atributos `prox` e `rep` (ponteiro para o representante); o nó representante guarda ainda o número de elementos ℓ da lista e um ponteiro `ult` para o seu último nó.

`cria`(x) cria um nó z com `rep`$[z] = z$, `prox`$[z] =$ `nil`, $\ell = 1$, `ult` $= z$, e faz $P[x] \leftarrow z$;

`busca`(x) devolve `rep`$[P[x]]$, em tempo $O(1)$;

`uniao`(x, y) para x e y em conjuntos distintos, sejam L_x e L_y as listas de x e y. Percorre-se L_y atualizando o `rep` de cada nó para o representante de L_x, liga-se `ult`$[L_x]$ ao primeiro nó de L_y e atualizam-se ℓ (soma dos dois) e `ult` (que passa a ser o antigo `ult`$[L_y]$) do representante de L_x, que passa a ser o da união. O custo é $O(1 + \ell[L_y])$. Se $L_x = L_y$ a concatenação criaria um ciclo; por isso a pré-condição.

Uma busca é feita em tempo constante e o custo de uma união é

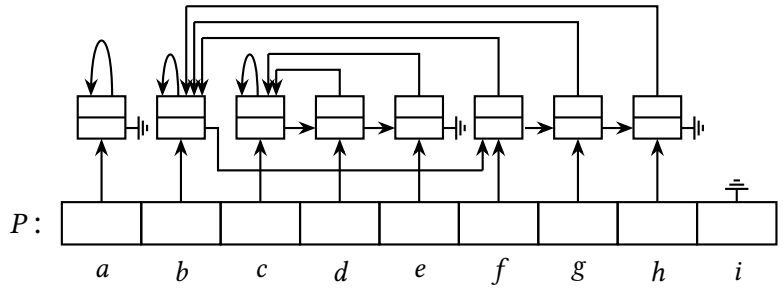


Figura 4.3: Representação com listas ligadas dos subconjuntos $\{c, d, e\}$, $\{a\}$ e $\{b, f, g, h\}$ do universo $\{a, b, \ldots, i\}$, com i ainda não criado. Os campos ℓ e `ult` dos representantes não foram representados.

basicamente o custo das atualizações. Sem outra precaução, $n - 1$ uniões podem custar $\Theta(n^2)$ (basta sempre atualizar a lista maior), o mesmo que a representação por vetor. Adotando a seguinte heurística, a complexidade melhora substancialmente:

> *atualizar sempre a menor lista*: numa união, os ponteiros `rep` são atualizados na menor das duas listas (em qualquer uma delas, se têm o mesmo tamanho), que é acrescentada ao final da maior; isto é, `uniao` compara ℓ nos dois representantes (em $O(1)$) e, se necessário, troca os papéis de L_x e L_y. Observe que a lista maior mantém o seu representante, logo, o representante de x muda se e somente se $\texttt{rep}[x]$ é atualizado. (‡)

Intuitivamente, cada vez que $\texttt{rep}[x]$ é atualizado, a lista de x pelo menos dobra de tamanho; formalizamos isso a seguir.

Proposição 4.4

Supondo (‡), em qualquer momento, se $\texttt{rep}[x]$ foi atualizado k vezes, então a lista que contém x tem pelo menos 2^k elementos.

Demonstração. Os conjuntos só crescem, logo o tamanho da lista de x não diminui com o tempo. Mostramos por indução em k que, logo após a k-ésima atualização de $\texttt{rep}[x]$, a lista de x tem pelo menos 2^k elementos; por monotonicidade isso vale também depois. Para $k = 0$, basta que a lista de x tenha pelo menos $1 = 2^0$ elemento. Seja $k \geq 1$ e seja L a lista de x imediatamente antes da k-ésima atualização; pela hipótese de indução e pela monotonicidade, $|L| \geq 2^{k-1}$. Como $\texttt{rep}[x]$ é atualizado, por (‡) a outra lista L' tem $|L'| \geq |L|$, e a lista resultante tem $|L| + |L'| \geq 2|L| \geq 2^k$ elementos. □

Corolário 4.3

Supondo (‡), o representante de um elemento qualquer muda no máximo $\lfloor \log_2 n \rfloor$ vezes.

Demonstração. Se muda k vezes, a lista de x tem pelo menos 2^k elementos e no máximo n, logo $2^k \leq n$. □

Teorema 4.5

Supondo (‡), que as n primeiras operações são `cria` e que `uniao` só é chamada com elementos de conjuntos distintos, o tempo para m operações num universo com n elementos é $O(m + n \log n)$.

Demonstração. Cada `cria` e cada `busca` custa $O(1)$, e cada `uniao` custa $O(1)$ mais o número de ponteiros `rep` atualizados. Pelo corolário, cada um dos n elementos tem `rep` atualizado no máximo $\lfloor \log_2 n \rfloor$ vezes, e portanto o total de atualizações é $O(n \log n)$. Somando, o tempo é $O(m + n \log n)$. □

Corolário 4.4

Para todo grafo simples conexo $G = (V, E, \rho)$ com $|V| \geq 2$, o tempo do algoritmo de Kruskal com listas ligadas e (‡) é $O(|E| \log |V|)$.

Demonstração. Ordenar as arestas por comparações custa $O(|E| \log |E|)$, e como $|E| \leq |V|^2$ isso é $O(|E| \log |V|)$. Como G é conexo e $|V| \geq 2$, temos $|E| \geq |V| - 1 \geq 1$, logo $|V| \leq |E| + 1 \leq 2|E|$. O algoritmo faz $|V|$ operações `cria`, no máximo $2|E|$ operações `busca` e no máximo $|V| - 1$ operações `uniao` (sempre em conjuntos distintos, por causa do teste $\texttt{busca}(u) \neq \texttt{busca}(v)$), ou seja, $m \leq 2|V| + 2|E| = O(|E|)$ operações, das quais as $|V|$ primeiras são `cria`. Pelo teorema com $n = |V|$, elas custam $O(|E| + |V| \log |V|)$. Como $|V| \leq 2|E|$ e $\log_2 |V| \geq 1$, vale $|E| \leq |E| \log |V|$ e $|V| \log |V| \leq 2|E| \log |V|$, logo o total é $O(|E| \log |V|)$. □

Representação por floresta

A ideia é que cada conjunto seja representado por uma árvore, em que cada elemento é um nó. Cada nó aponta para o seu pai, a raiz é o representante e o pai da raiz é ela mesma (Figura 4.4). Assim, `cria` cria em tempo constante uma árvore com um só nó; $\texttt{busca}(x)$ percorre o caminho de x até a raiz, em tempo proporcional à profundidade de x, e devolve a raiz; e a união de duas árvores é feita em tempo constante *a partir das duas raízes*, pendurando uma na outra (Figura 4.5). Portanto, $\texttt{uniao}(x, y)$ custa duas buscas mais $O(1)$.

Essa estrutura pode ser muito ruim: se pendurarmos sempre a raiz da árvore maior numa árvore unitária, formamos uma cadeia com n nós, em que cada busca no nó mais profundo custa $\Theta(n)$. Com uma heurística simples o resultado melhora substancialmente. O *tamanho* de uma árvore é o seu número de nós.

União por tamanho: *a raiz da árvore de menor tamanho é pendurada na raiz da árvore de maior tamanho (em caso de empate, em qualquer uma).*

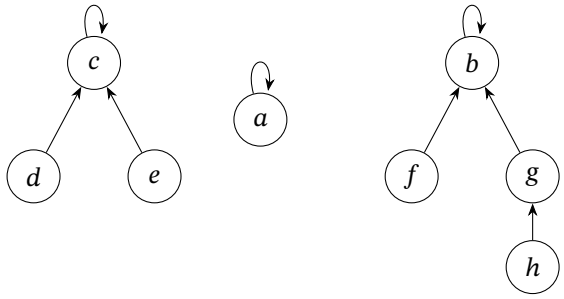


Figura 4.4: Uma representação com árvores dos subconjuntos $\{c, d, e\}$, $\{a\}$ e $\{b, f, g, h\}$.

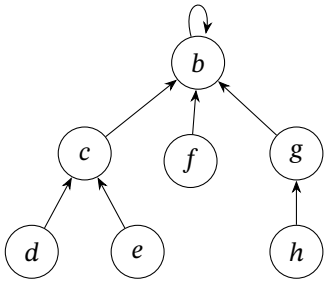


Figura 4.5: Resultado de Uniao(e, f) pendurando-se a raiz c, da árvore menor (tamanho 3), na raiz b, da árvore maior (tamanho 4).

Cada nó tem um atributo pai$[x]$ e, nas raízes, um atributo $t[x]$ que armazena o número de elementos na árvore enraizada em x. Os algoritmos ficam da seguinte forma:

Algoritmo 14: cria(x)

```
1 pai[x] ← x;
2 t[x] ← 1;
```

Algoritmo 15: busca(x)

```
1 enquanto x ≠ pai[x] faça
2     x ← pai[x];
3 retorne x;
```

O procedimento `link` a seguir pressupõe que x e y são raízes distintas, e `uniao` que x e y estão em árvores distintas.

Algoritmo 16: link(x, y)

```
1 t[x] ← t[x] + t[y];
2 pai[y] ← x;
```

Algoritmo 17: uniao(x, y)

```
1 r_x ← busca(x);
2 r_y ← busca(y);
3 se t[r_x] ≥ t[r_y] então
    link(r_x, r_y);
4 senão link(r_y, r_x);
```

Definimos a *altura de uma árvore* como a maior distância de um nó até a raiz e temos a seguinte relação entre tamanho e altura resultantes de uma sequência de uniões com a heurística de união por tamanho.

Proposição 4.5

Usando união por tamanho, e supondo que as n primeiras operações são `cria`, em qualquer momento cada árvore tem pelo menos 2^h nós, onde h é a sua altura.

Demonstração. Por indução no número k de operações `uniao` já realizadas. Para $k = 0$, as únicas operações realizadas até então que

alteram a estrutura são `cria` (as buscas não a alteram), e portanto toda árvore tem um nó e altura 0, com $1 \geq 2^0$. Seja $k \geq 1$ e considere a k-ésima união. Como as n primeiras operações são `cria`, todas as criações já foram feitas antes dela e nenhuma outra ocorre depois; assim, nada além das uniões altera a estrutura. A k-ésima união liga duas árvores distintas A e B, com tamanhos t_A, t_B e alturas h_A, h_B; pela hipótese de indução $t_A \geq 2^{h_A}$ e $t_B \geq 2^{h_B}$. As demais árvores não mudam. Sem perda de generalidade $t_A \geq t_B$, e então a raiz de B é pendurada na raiz de A. A árvore resultante tem $t' = t_A + t_B$ nós e altura $h' = \max\{h_A,\ h_B + 1\}$. Se $h' = h_A$, então $t' \geq t_A \geq 2^{h_A} = 2^{h'}$. Se $h' = h_B + 1$, então $t' = t_A + t_B \geq 2t_B \geq 2 \cdot 2^{h_B} = 2^{h'}$. Em ambos os casos o resultado vale após a k-ésima união. □

Corolário 4.5

Usando união por tamanho, e supondo que as n primeiras operações são `cria`, o tempo de m operações num universo com n elementos é $O(m(1 + \log n))$.

Demonstração. Pela proposição, uma árvore de altura h tem $2^h \leq n$ nós, logo $h \leq \log_2 n$ e cada `busca` custa $O(1 + \log n)$. Cada `cria` custa $O(1)$ e cada `uniao` custa duas buscas mais $O(1)$, isto é, $O(1 + \log n)$. O total é $O(n + m(1 + \log n)) = O(m(1 + \log n))$, pois $m \geq n$. □

Como o tempo de `busca` é proporcional à altura da árvore, parece mais natural uma heurística baseada na altura e não no tamanho.

União por altura: *a raiz da árvore mais baixa é pendurada na raiz da árvore mais alta (em caso de empate, em qualquer uma).*

Cada raiz r guarda $h[r]$, a altura da árvore enraizada em r, que é iniciada com 0.

Algoritmo 18: cria(x)

```
1 pai[x] ← x;
2 h[x] ← 0;
```

Algoritmo 19: link(x, y)

```
1 se h[x] = h[y] então
2     h[x] ← h[x] + 1
3 pai[y] ← x;
```

Algoritmo 20: uniao(x, y)

```
1 r_x ← busca(x);
2 r_y ← busca(y);
3 se h[r_x] ≥ h[r_y] então link(r_x, r_y);
4 senão link(r_y, r_x);
```

O procedimento `link`(x, y) pressupõe que x e y são raízes distintas com $h[x] \geq h[y]$; `busca` é o mesmo. Só é preciso incrementar $h[x]$ quando as árvores têm a mesma altura. O valor $h[r]$ é a altura exata da árvore de raiz r: se $h_A \geq h_B$ e a raiz de B é pendurada na de

A, a nova altura é $\max\{h_A, h_B + 1\}$, que é h_A se $h_A > h_B$ e $h_A + 1$ se $h_A = h_B$.

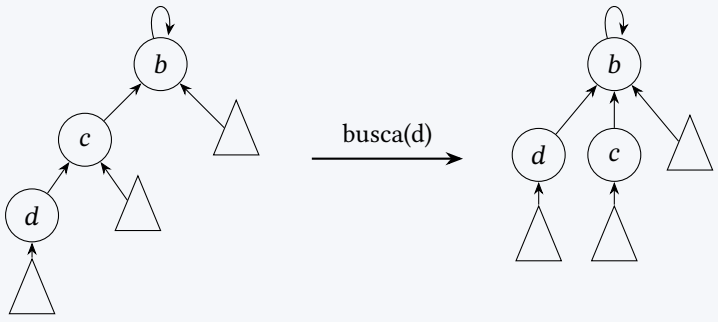
A Proposição 4.5 pode ser reescrita para as duas heurísticas ("usando união por tamanho ou por altura, cada árvore tem pelo menos 2^h nós"), pois também vale para a união por altura, com a seguinte prova para o passo de indução. Sejam A e B as árvores unidas, com $h_A \geq h_B$ (a raiz de B é pendurada na de A). Se $h_A > h_B$, a nova árvore tem altura $h' = h_A$ e $t' \geq t_A \geq 2^{h_A} = 2^{h'}$. Se $h_A = h_B$, então $h' = h_A + 1$ e $t' = t_A + t_B \geq 2 \cdot 2^{h_A} = 2^{h'}$. O corolário segue como antes.

Observação 4.2

Comparando as cotas $O(m + n \log n)$ (listas ligadas) e $O(m \log n + n)$ (árvores), ambas com união por tamanho, as árvores *não* são melhores, e podem ser pior. Com $n = 2^k$ elementos, façam-se $n - 1$ uniões de árvores de mesma altura, obtendo uma árvore de altura $\log_2 n$, e depois $m - 2n$ buscas sobre um nó de profundidade $\log_2 n$; o custo é $\Omega(m \log n)$ para $m \geq 4n$, enquanto as listas gastam $O(m + n \log n)$. A vantagem das árvores vem da heurística de *compressão de caminhos*, vista a seguir.

Busca com compressão de caminhos e união por rank: *quando fazemos um* `busca`(x) *aproveitamos o percurso de x até a raiz da árvore e desviamos os ponteiros* `pai`$[y]$*, de todo y nesse caminho, para a raiz; na união penduramos a árvore de menor rank na raiz da de maior rank.*

Exemplo 4.1: Esquema de uma busca com compressão de caminhos



Com compressão de caminhos a altura de uma árvore pode diminuir, mas atualizá-la a cada busca seria caro. Mantemos então, em cada nó x, rank$[x]$, iniciado em 0 e alterado apenas por `link`; ele é uma cota superior da altura da subárvore de x (e igual a ela quando não houve compressão). Na união, a raiz de menor rank é pendurada na de maior rank, e, em caso de empate, o rank da raiz que recebe aumenta de 1. A compressão não muda o conjunto de nós de uma árvore nem

o rank das raízes, logo a Proposição 4.5 vale com *rank* no lugar de altura (mesma prova da união por altura): toda árvore de raiz r tem pelo menos $2^{\text{rank}[r]}$ nós e, portanto, rank $\leq \lfloor \log_2 n \rfloor$.

O algoritmo para `busca` com compressão de caminhos fica

Algoritmo 21: cria(x)

```
1 pai[x] ← x;
2 rank[x] ← 0;
```

Algoritmo 22: uniao(x, y)

```
1 r_x ← busca(x);
2 r_y ← busca(y);
3 se rank[r_x] ≥ rank[r_y] então link(r_x, r_y);
4 senão link(r_y, r_x);
```

Algoritmo 23: busca(x) com compressão

```
1 se x ≠ pai[x] então pai[x] ← busca(pai[x]);
2 retorne pai[x]
```

Algoritmo 24: link(x, y)

```
1 se rank[x] = rank[y] então rank[x] ← rank[x] + 1;
2 pai[y] ← x;
```

Essas duas heurísticas juntas, a união por *rank* e a compressão de caminhos é bastante eficiente. O próximo resultado, sobre a complexidade de m operações num universo de tamanho n, envolve uma função $\alpha(m, n)$ que cresce muito vagarosamente; na prática podemos assumir que $\alpha = 4$. Não demonstraremos esse teorema (veja Cormen et al. (2012)), o resultado que mostraremos a seguir é um pouco menos preciso.

Teorema 4.6: (Tarjan, 1975)

Com união por rank e compressão de caminhos, o tempo gasto com m operações num universo de tamanho n, com $m \geq n$ (as n primeiras sendo `cria`), é $O(m\,\alpha(m, n))$.

(Demonstração em Cormen et al. (2012).)

Observação 4.3

A função de Ackermann (na versão de Cormen et al. (2012)) é dada, para $i, j \geq 1$, por

$$A(i,j) = \begin{cases} 2^j & \text{se } i = 1, \\ A(i-1,2) & \text{se } i \geq 2 \text{ e } j = 1, \\ A\big(i-1, A(i,j-1)\big) & \text{se } i \geq 2 \text{ e } j \geq 2, \end{cases}$$

e cresce muito rapidamente: $A(2, j)$ é uma torre de exponenciais com $j+1$ dois, $A(3,1) = 16$ e $A(4,1) = A(2,16)$ é uma torre de 17 dois, imensamente maior que 10^{80} (estimativa do número de átomos do universo observável). Para $m \geq n$ define-se

$$\alpha(m,n) = \min\{i \geq 1 : A(i, \lfloor m/n \rfloor) > \log_2 n\},$$

como A é crescente em j, $\alpha(m,n) \leq \alpha(n,n)$, e portanto $\alpha(m,n) \leq 4$ para todo $n < 2^{A(4,1)}$. Por exemplo,

n	$\alpha(n,n)$
1 – 3	1
4 – 15	2
16 – 65535	3
$65536 - 2^{A(4,1)} - 1$	4

Considere uma sequência de m operações, das quais as n primeiras são `cria`, com união por *rank* e compressão de caminhos. Sejam $b_0 = 1$ e $b_g = 2^{b_{g-1}}$ (isto é, $1, 2, 4, 16, 65536, \ldots$). Particionamos os elementos de $\mathcal{U}$ em *grupos* segundo o *rank* final: o Grupo 0 tem os de *rank* 0 e 1 e, para $g \geq 1$, o Grupo g tem os de *rank* em $(b_{g-1}, b_g] = (K, 2^K]$ com $K = b_{g-1}$. O *rank* satisfaz:

(i) o *rank* de um nó só muda em `link`, quando ele é a raiz que recebe a outra; logo, depois que deixa de ser raiz, não muda mais;

(ii) $\text{rank}[x] < \text{rank}[\text{pai}[x]]$ para todo x que não é raiz (vale na criação; em `link`, se os ranks são iguais o da raiz que recebe aumenta; e a compressão aponta x para um ancestral, de rank maior). Em particular, $\text{rank}[\text{pai}[x]]$ só cresce ao longo do tempo;

(iii) para cada x, seja $D(x)$ a árvore de x no instante em que x deixa de ser raiz (ou no fim, se isso nunca ocorre). Então $|D(x)| \geq 2^{\text{rank}[x]}$ (Proposição 4.5 com *rank*); e se $x \neq y$ têm o mesmo *rank* final, então $D(x) \cap D(y) = \varnothing$;

(iv) para todo k, há no máximo $n/2^k$ nós de *rank* maior que k; em particular, o Grupo g tem no máximo $n/2^{b_{g-1}}$ nós;

(v) há no máximo $\lg^* n$ grupos.

Justificativas. (iii) A árvore de x no instante em que deixa de ser raiz tem pelo menos $2^{\text{rank}[x]}$ nós, e o seu *rank* já é o final por (i). Suponha $z \in D(x) \cap D(y)$, com $x \neq y$ do mesmo rank r e, sem perda de generalidade, x deixando de ser raiz antes de y (ou y sendo raiz no fim). Nesse instante x é ligado a uma raiz R de rank $\geq r$, e após o `link` a raiz da árvore de z passa a ter rank $> r$ (se $\text{rank}[R] = r$, ele aumenta), e esse rank só cresce. Quando y deixa de ser raiz, y é a raiz da árvore de z com rank r, uma contradição. (iv) Para cada r, os $D(x)$ de rank r são disjuntos e têm $\geq 2^r$ elementos, logo há

$\leq n/2^r$ nós de rank r; some sobre $r > k$. (v) O rank é no máximo $\lfloor \lg n \rfloor$ (Proposição 4.5). Se g_0 é o menor g com $b_g \geq \lfloor \lg n \rfloor$, há $g_0 + 1$ grupos, e para $g_0 \geq 1$ vale $b_{g_0-1} < \lg n$, isto é, $n > b_{g_0}$; aplicando lg repetidamente, $\lg^{(g_0)} n > b_0 = 1$, logo $\lg^* n \geq g_0 + 1$.

O custo total é $O(m)$ (cada `cria` e cada `link` custa $O(1)$) mais o custo das $f \leq 2m$ buscas, e o custo de uma busca é o número de nós do caminho $x = y_0, \ldots, y_k$ até a raiz, cujos apontadores são redirecionados. Classificamos os nós y do caminho em dois tipos:

1. Tipo 1: y é a raiz ou filho da raiz, ou pai$[y]$ está em grupo diferente do de y;
2. Tipo 2: os demais, isto é, y não é a raiz nem filho da raiz e pai$[y]$ está no mesmo grupo que y.

Tipo 1. Ao longo do caminho o rank cresce estritamente, logo o grupo não decresce e muda no máximo $\lg^* n - 1$ vezes; com a raiz e o filho da raiz, cada busca tem no máximo $\lg^* n + 1$ nós do Tipo 1. Em f buscas são $O(m \lg^* n)$.

Tipo 2. Seja y no Grupo g, com ranks em $(K, 2^K]$. Cada vez que y é do Tipo 2 em uma busca, pai$[y]$ passa a ser a raiz, cujo rank é maior que o do pai anterior (pois este não era a raiz); como o rank do pai só cresce e o de y é fixo por (i), o rank de pai$[y]$ assume valores distintos em $(\text{rank}[y], 2^K]$ a cada ocorrência, e depois que sai do grupo y nunca mais é do Tipo 2. Logo y é do Tipo 2 no máximo 2^K vezes. Por (iv) há no máximo $n/2^K$ tais nós, e o Grupo g contribui com no máximo

$$\frac{n}{2^K} \cdot 2^K = n$$

e, somando sobre os $\leq \lg^* n$ grupos, $O(n \lg^* n)$.

Teorema 4.7

Com união por *rank* e compressão de caminhos, o tempo gasto com m operações `cria`/`busca`/`uniao` num universo com n elementos, das quais as n primeiras são `cria`, é $O((m + n) \lg^* n)$. □

Exercícios

Exercício 74. Seja $G = (V, E, \rho)$ um grafo conexo, $U \subsetneq V$ não-vazio e e uma aresta de peso mínimo em $\nabla_G(U)$. Mostre que e pertence a alguma árvore geradora mínima de G.

Exercício 75. Seja $G = (V, E, \rho)$ um grafo conexo e seja C um circuito de G. Mostre que se e é uma aresta de peso máximo entre as arestas de C, então $G - e$ é conexo e existe uma árvore geradora mínima de $G - e$ que também é uma árvore geradora mínima de G. Em particular, isso vale para toda aresta de peso máximo em G que pertence a algum circuito.

Exercício 76. Seja $G = (V, E, \rho)$ um grafo conexo e seja $U \subsetneq V$ não-vazio. Mostre que se e é a *única* aresta de peso mínimo em $\nabla_G(U)$, então e pertence a toda árvore geradora mínima de G. Mostre, com um exemplo, que a hipótese de unicidade não pode ser omitida.

Exercício 77. Seja $G = (V, E, \rho)$ um grafo conexo e seja T uma árvore geradora de G. Mostre que T é uma árvore geradora mínima se e somente se, para toda $e \in E(G) \setminus E(T)$, toda aresta do único circuito C de $T + e$ tem peso no máximo $\rho(e)$.

Exercício 78. Seja $G = (V, E, \rho)$ um grafo conexo. Mostre que se, para todo corte $\nabla_G(U)$ com $U \subsetneq V$ não-vazio, existe uma única aresta de peso mínimo no corte, então G tem exatamente uma árvore geradora mínima. A recíproca dessa afirmação vale? Justifique.

Exercício 79. Seja $G = (V, E, \rho)$ um grafo conexo com pesos positivos nas arestas. Para toda árvore geradora mínima T de G e todo caminho $P \subseteq T$, o caminho P é um caminho de peso mínimo em G entre as suas extremidades? Justifique.

Exercício 80. Mostre que para toda árvore geradora mínima T de G existe uma ordenação $e_1, \ldots, e_{|E|}$ das arestas de G, com $\rho(e_1) \leq \cdots \leq \rho(e_{|E|})$, tal que o algoritmo de Kruskal, examinando as arestas nessa ordem, devolve T.

Exercício 81. Seja $G = (V, E, \rho)$ um grafo conexo com pesos positivos nas arestas. Mostre que todo conjunto $F \subseteq E$ tal que (V, F) é conexo e $\rho(F) = \sum_{e \in F} \rho(e)$ é mínimo entre os conjuntos com essa propriedade é tal que (V, F) é uma árvore. Dê um exemplo com pesos não-positivos em que essa conclusão não vale.

Exercício 82. Considere a seguinte estratégia genérica para computar uma árvore geradora mínima (AGM). Seja $G = (V, E, \rho)$ um grafo conexo com pesos nas arestas. Dado $S \subseteq E$, dizemos que uma aresta $e \in E \setminus S$ é **boa para** S se $S \cup \{e\} \subseteq E(T)$ para alguma AGM T de G.

Algoritmo 25: AGM(G)

Dado : um grafo conexo $G = (V, E, \rho)$.
Devolve : árvore geradora mínima.

```
1 S ← ∅;
2 enquanto (V, S) não é árvore geradora faça
3 |   escolha uma aresta f ∈ E \ S boa para S;
4 |   S ← S ∪ {f};
5 retorne (V, S);
```

1. Suponha que, sempre que (V, S) não é árvore geradora, existe uma aresta boa para S, e que um *oráculo* a entrega ao algoritmo na linha 3. Prove que o laço termina e que AGM(G) devolve uma AGM de G. (Dica: determine e prove um invariante para o laço; para a terminação, observe que $|S|$ cresce a cada iteração.)

2. Sejam T uma AGM de G, $S \subseteq E(T)$ e $U \subsetneq V$ não-vazio tal que $\nabla_G(U) \cap S = \emptyset$. Prove que toda aresta e de peso mínimo em $\nabla_G(U)$ é boa para S. (Dica: considere os casos $e \in E(T)$ e $e \notin E(T)$; no segundo, o circuito de $T + e$ cruza o corte em pelo menos duas arestas, e a aresta que se remove de T pertence ao corte e, portanto, não pertence a S.)
3. Conclua que, se $S \subseteq E(T)$ para alguma AGM T de G e (V, S) não é árvore geradora, então existe uma aresta boa para S. Logo a hipótese do item 1 se verifica em toda iteração: pelo invariante desse item, S permanece contida em alguma AGM. (Dica: (V, S) é acíclico e, se não é árvore geradora, não é conexo; tome U como o conjunto de vértices de uma componente de (V, S) e use que $\nabla_G(U) \neq \emptyset$, pois G é conexo.)
4. Use o item 2 para provar a correção dos algoritmos de Dijkstra–Jarník–Prim e Kruskal: mostre que, em cada iteração em que uma aresta é inserida em S, ela é boa para o S corrente (suponha, por indução, que o S corrente está contido em alguma AGM; no início, $S = \emptyset$), e conclua, pelo item 1, que o algoritmo devolve uma AGM. (Dicas: no Prim, tome U como o conjunto de vértices da árvore corrente; nenhuma aresta de S está em $\nabla_G(U)$ e a aresta inserida tem peso mínimo nesse corte. No Kruskal, tome U como o conjunto de vértices da componente de uma das pontas da aresta em (V, S) e use a ordem em que as arestas são examinadas para mostrar que toda aresta de $\nabla_G(U)$ é examinada depois da inserida.)

5 Grafos dirigidos

Formalmente, um ***grafo dirigido*** é um par (V, E) onde V é um conjunto finito (vértices) e $E \subseteq V \times V$ é o conjunto das arestas, ou ***arcos*** do grafo dirigido. Quando vale a restrição $(v, v) \notin E$, para todo $v \in V$, chamamos de ***grafo dirigido simples*** (podem existir (u, v) e (v, u) simultaneamente).

Dizemos que a aresta, ou ***arco***, $(v, u) \in E$ ***sai de*** v e ***chega em*** u. Também, definimos os subconjuntos $N^+(v)$, $N^-(v) \subseteq V$, de *vizinhos* de cada $v \in V$

$$N^+(v) \coloneqq \{u \in V : (v, u) \in E\} \quad \text{e} \quad N^-(v) \coloneqq \{u \in V : (u, v) \in E\}.$$

Definimos, para cada $v \in V$, o ***grau de saída***, $\mathrm{d}^+(v)$, e o ***grau de entrada***, $\mathrm{d}^-(v)$, respectivamente, por

$$\mathrm{d}^+(v) \coloneqq |N^+(v)| \quad \text{e} \quad \mathrm{d}^-(v) \coloneqq |N^-(v)|$$

e é fácil de verificar que vale

$$\sum_{v \in V} \mathrm{d}^+(v) = \sum_{v \in V} \mathrm{d}^-(v) = |E|,$$

pois cada arco (u, v) é contado uma vez em $\mathrm{d}^+(u)$ e uma vez em $\mathrm{d}^-(v)$.

O ***grafo subjacente*** ao grafo dirigido $G = (V, E)$ é o grafo SG dado por

$$SG = \left(V, \bigcup_{\substack{(u,v) \in E \\ u \neq v}} \{\{u, v\}\}\right)$$

5.1 Representação computacional e percurso

De modo análogo ao caso não dirigido, assumiremos que nos problemas computacionais os vértices de G são $\{1, 2, \ldots, n\}$ (caso contrário construímos um isomorfismo).

Como antes, um grafo dirigido (V, E) pode ser representado por uma matriz A quadrada $|V| \times |V|$ definida por

$$A(i, j) = \begin{cases} 1, & \text{se } (i, j) \in E \\ 0, & \text{caso contrário,} \end{cases}$$

ou seja, é uma matriz com entradas 0 e 1 dada por $a_{i,j} = 1$ se e só se (i, j) é arco. Notemos que a matriz agora pode não ser simétrica. Para o grafo G representado na Figura 5.1, a matriz de adjacência é

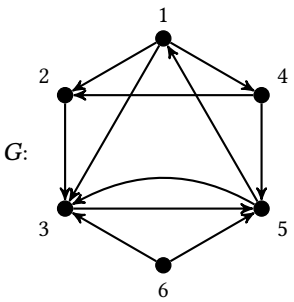


Figura 5.1: Grafo dirigido representado na matriz (5.2) e na lista de adjacências da Figura 5.2.

$$A = \begin{bmatrix} 0 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 & 0 \end{bmatrix}. \tag{5.2}$$

Também podemos representar um grafo por sua ***lista de adjacência dirigida***, dada por um vetor $N^+[\ ]$ de listas ligadas. A lista $N^+[i]$ contém os vizinhos $N^+(i)$ do vértice i e cada nó dessa lista é composto por uma variável que armazena um vértice e um ponteiro que aponta para o próximo nó da lista.

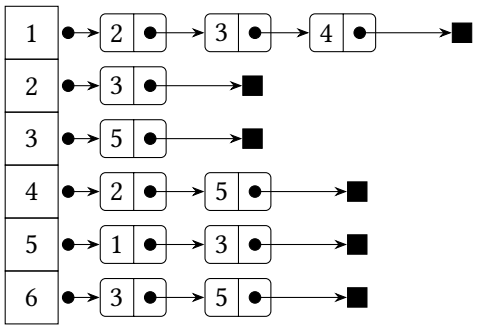


Figura 5.2: Lista de adjacência de um grafo dirigido com 6 vértices.

5.1.1 Busca em profundidade

Uma busca em profundidade rotulada, algoritmo 3 na página 32 reproduzido parcialmente na margem ao lado, executada num grafo dirigido resulta numa classificação dos arcos em quatro tipos, ao invés dos dois tipos (árvore/retorno) que ocorrem no caso não dirigido.

```
1 cont ← cont + 1
2 chega[w] ← cont
3 para cada u ∈ N⁺(w) faça
4     se chega[u] = 0 então
5         p[u] ← w
6         BP(G, u)
7 cont ← cont + 1
8 sai[w] ← cont.
```

Considere uma execução completa da busca em profundidade rotulada, que recomeça em um vértice ainda não visitado até visitar todos. Seja $p[v]$ o pai de v, com $p[v] = \texttt{nil}$ se v é raiz de uma árvore da busca, e defina $p^{(0)}[v] = v$ e $p^{(t)}[v] = p[p^{(t-1)}[v]]$ enquanto o resultado está definido. Dizemos que u é ***ancestral próprio*** de v (e v ***descendente próprio*** de u) se $p^{(t)}[v] = u$ para algum $t \geq 1$. Se $p[u] = v$, dizemos que u é ***filho*** de v e que v é ***pai*** de u.

Proposição 5.1: Classificação dos arcos

Após uma busca em profundidade rotulada, todo arco $(u, v) \in E$ satisfaz exatamente uma das condições

1. *arco pai–filho*: $p[v] = u$;
2. *retorno ascendente*: v é ancestral próprio de u, caso em que $[\text{chega}[u], \text{sai}[u]] \subset [\text{chega}[v], \text{sai}[v]]$;
3. *retorno descendente*: v é descendente próprio de u e $p[v] \neq u$, caso em que $[\text{chega}[v], \text{sai}[v]] \subset [\text{chega}[u], \text{sai}[u]]$;

4. *arco transversal*: chega[v] < sai[v] < chega[u] < sai[u].

(A prova usa que, para $x \neq y$, os intervalos [chega, sai] são disjuntos ou um contém o outro, e que chega[u] < chega[v] implica v descendente de u se v é branco quando u é explorado; veja Cormen et al. (2012).)

A figura abaixo esquematiza uma busca em profundidade rotulada no grafo dirigido do exemplo 5.2.

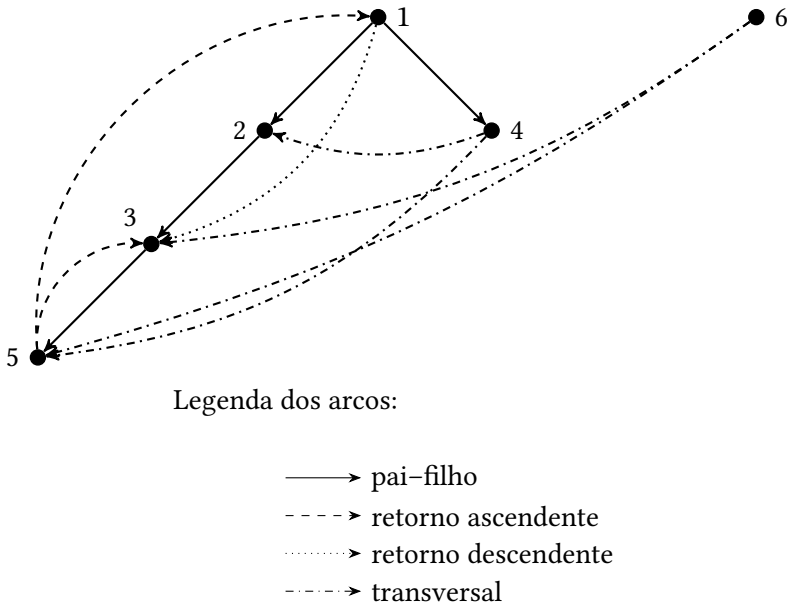


Figura 5.3: Busca em profundidade rotulada.

Exercício 83. A Figura 5.5 é o resultado de uma busca em profundidade rotulada no grafo dirigido da Figura 5.4.

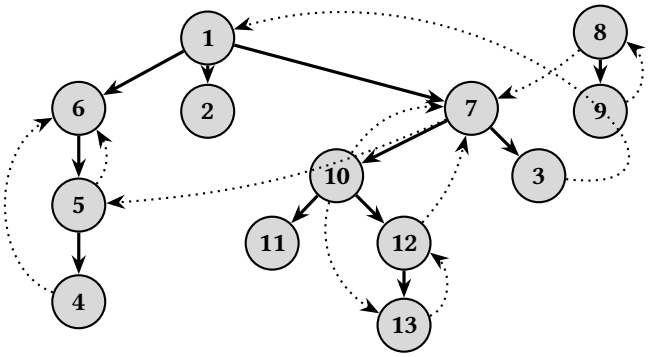


Figura 5.5: Esquema de uma busca rotula em 5.4

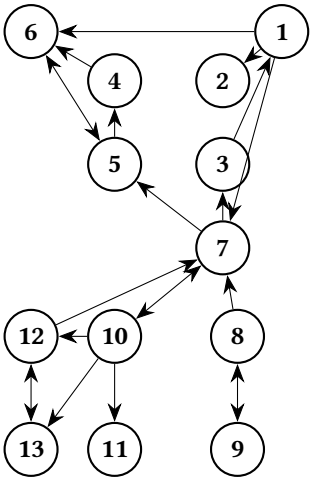


Figura 5.4: Exemplo de BP em grafo dirigido

Classifique os arcos do grafo dirigido após a busca em profundidade.

5.2 Grafos dirigidos acíclicos e ordenação topológica

Nesta seção todos os grafos dirigidos são simples, isto é, sem laços.

Em muitos problemas os arcos de um grafo dirigido representam relações de precedência: um arco (u, v) indica que u deve vir antes de v. É o caso de tarefas de um projeto em que algumas só podem começar depois que outras terminam, de disciplinas com pré-requisitos, ou de arquivos de um programa em que a compilação de um depende de outro. Nessas situações queremos uma sequência que respeite todas as precedências. Isso é impossível se as precedências formam um circuito: u antes de v, v antes de w e w antes de u. Nesta seção mostramos que essa é a única obstrução e como encontrar uma tal sequência em tempo linear.

Definição 5.1: Caminho e circuito orientados

Um ***caminho orientado*** em G é uma sequência de vértices distintos $x_1, \dots, x_k$, com $k \geq 1$ e $(x_i, x_{i+1}) \in E(G)$ para todo $i \in \{1, \dots, k-1\}$.
Um ***circuito orientado*** é uma sequência $x_1, \dots, x_k, x_1$, em que $x_1, \dots, x_k$ é um caminho orientado com $k \geq 2$ e $(x_k, x_1) \in E(G)$. (Como não há laços, $k = 1$ não ocorre; para $k = 2$ o circuito usa os arcos (x_1, x_2) e (x_2, x_1).)

Definição 5.2: grafo dirigido acíclico

Um ***grafo dirigido acíclico*** (GDA, em inglês *DAG*) é um grafo dirigido que não contém circuito orientado.

Exemplo 5.1

Considere cinco disciplinas a, b, c, d, e com os seguintes pré-requisitos: a é pré-requisito de b e de c; b e c são pré-requisitos de d; c é pré-requisito de e. O grafo dirigido G dado por $V(G) = \{a, b, c, d, e\}$ e

$$E(G) = \{(a, b), (a, c), (b, d), (c, d), (c, e)\}$$

modela essa situação e é acíclico (Figura 5.6). A sequência a, b, c, d, e respeita os pré-requisitos, assim como a, c, e, b, d. Já a sequência a, b, d, c, e não serve, pois c é pré-requisito de d mas vem depois dele.

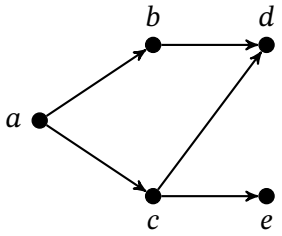


Figura 5.6: grafo dirigido do Exemplo 5.1.

Definição 5.3: ordenação topológica

Seja G um grafo dirigido com n vértices. Uma ***ordenação topológica*** de G é uma ordenação $v_1, v_2, \ldots, v_n$ dos vértices de G, cada vértice aparecendo exatamente uma vez, tal que

$$(v_i, v_j) \in E(G) \implies i < j.$$

Escrevendo $u \prec v$ quando u aparece antes de v na ordenação, a condição é que $u \prec v$ para todo arco (u, v) de G. O ***problema da ordenação topológica*** é: dado um grafo dirigido G, determinar uma ordenação topológica de G ou concluir que não existe.

Se dispomos os vértices em linha, na ordem da ordenação, todos os arcos apontam para a direita, como no diagrama da Figura 5.8. As ordenações do Exemplo 5.1 são exatamente as sequências de disciplinas que respeitam os pré-requisitos.

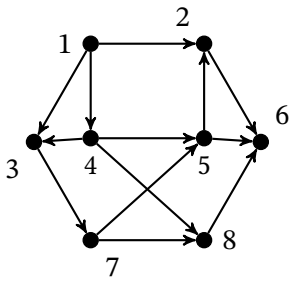


Figura 5.7: Um grafo dirigido e sua ordenação topológica mostrada na Figura 5.8.

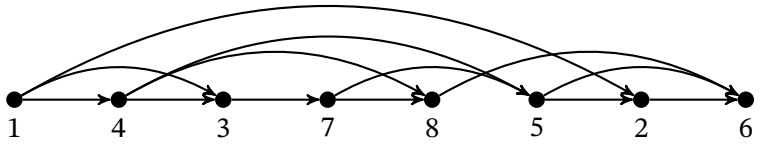


Figura 5.8: Uma ordenação topológica dos vértices do grafo da Figura 5.7: $1 \prec 4 \prec 3 \prec 7 \prec 8 \prec 5 \prec 2 \prec 6$.

Dizemos que um vértice v é uma ***fonte*** se $\mathrm{d}^-(v) = 0$, isto é, nenhum arco chega em v, e que v é um ***sumidouro*** se $\mathrm{d}^+(v) = 0$, isto é, nenhum arco sai de v. Em toda ordenação topológica o primeiro vértice é uma fonte e o último é um sumidouro: um arco que chegasse no primeiro, ou saísse do último, teria o outro extremo antes do primeiro ou depois do último vértice. Em um GDA sempre existem uma fonte e um sumidouro, e isso é o ponto de partida para provar a existência de ordenações topológicas.

Proposição 5.2: Fontes e sumidouros

Todo grafo dirigido acíclico com pelo menos um vértice tem uma fonte e um sumidouro.

Demonstração. Seja G um GDA com pelo menos um vértice e tome um caminho orientado $P = x_1, x_2, \ldots, x_k$ com $k \geq 1$ máximo.

Suponha que existe um arco $(u, x_1) \in E(G)$. Como G não tem laços, $u \neq x_1$. Se $u \notin \{x_1, \ldots, x_k\}$ então $u, x_1, \ldots, x_k$ é um caminho orientado com $k+1$ vértices, contradizendo a maximalidade de P. Se $u = x_i$ para algum $i \in \{2, \ldots, k\}$ então $x_1, \ldots, x_i, x_1$ é um circuito orientado em G, contradizendo que G é acíclico. Logo nenhum arco chega em x_1, isto é, $\mathrm{d}^-(x_1) = 0$ e x_1 é uma fonte.

Analogamente, suponha que existe um arco $(x_k, w) \in E(G)$; en-

tão $w \neq x_k$, pois não há laços. Se $w \notin \{x_1, \ldots, x_k\}$, o caminho $x_1, \ldots, x_k, w$ contradiz a maximalidade de P; se $w = x_i$ com $i \in \{1, \ldots, k-1\}$, então $x_i, \ldots, x_k, x_i$ é um circuito orientado em G, uma contradição. Portanto $\mathrm{d}^+(x_k) = 0$ e x_k é um sumidouro. □

Teorema 5.1: Existência de ordenação topológica

Um grafo dirigido G admite uma ordenação topológica se e somente se G é acíclico.

Demonstração. (⇒) Seja $v_1, \ldots, v_n$ uma ordenação topológica de G e denote por $\mathrm{pos}(v)$ o índice de v nessa ordenação. Suponha, por absurdo, que $C = x_1, \ldots, x_k, x_1$ é um circuito orientado em G. Os arcos (x_i, x_{i+1}) dão

$$\mathrm{pos}(x_1) < \mathrm{pos}(x_2) < \cdots < \mathrm{pos}(x_k),$$

e o arco (x_k, x_1) dá $\mathrm{pos}(x_k) < \mathrm{pos}(x_1)$. Juntos, $\mathrm{pos}(x_1) < \mathrm{pos}(x_k) < \mathrm{pos}(x_1)$, o que é absurdo.

(⇐) Por indução em $n = |V(G)|$. Se $n = 1$ não há arcos, pois não há laços, e a única ordenação do vértice é uma ordenação topológica. Seja $n \geq 2$ e suponha o resultado válido para GDAs com $n-1$ vértices. Pela Proposição 5.2, G tem uma fonte s. O grafo $G' \coloneqq G\big[V(G) \setminus \{s\}\big]$ tem $n-1$ vértices e é acíclico, pois todo circuito orientado de G' é um circuito orientado de G. Pela hipótese de indução, G' tem uma ordenação topológica $w_1, \ldots, w_{n-1}$. Afirmamos que $s, w_1, \ldots, w_{n-1}$ é uma ordenação topológica de G. De fato, os arcos de G que não incidem em s são arcos de G' e respeitam a ordem de $w_1, \ldots, w_{n-1}$. Como $\mathrm{d}^-(s) = 0$, todo arco que incide em s é da forma (s, w_j), e s vem antes de w_j. □

A demonstração sugere um algoritmo: enquanto restarem vértices, remover uma fonte do grafo restante e colocá-la na próxima posição. Se em algum momento o grafo restante, que é não-vazio, não tem fonte, então, pela Proposição 5.2, ele não é acíclico, e portanto contém um circuito orientado, que também é um circuito de G (o restante é um subgrafo induzido de G). Assim o algoritmo decide se G é acíclico e, em caso afirmativo, devolve uma ordenação topológica. O Exercício 85 pede para implementá-lo em tempo $O(|V| + |E|)$. Vejamos outro algoritmo, baseado na busca em profundidade rotulada, que também decide se G é acíclico.

5.2.1 Ordenação topológica por busca em profundidade

A ideia é a seguinte. Empilhar u corresponde a *começar* a explorar u, e desempilhar u corresponde a *terminar* de explorar u. Enquanto u está na pilha, a busca está "dentro" de u, e u só é desempilhado depois que todos os caminhos que partem dele foram esgotados. Assim, se

$u \to v$, então v faz parte do que u precisa terminar de explorar, e portanto v termina antes de u, isto é, sai$(v) <$ sai(u). Listando os vértices em ordem decrescente de sai, cada u aparece antes de todos os seus sucessores, que é exatamente o que uma ordenação topológica exige.

Para ver isso com mais cuidado, considere o instante em que a busca examina uma aresta $u \to v$. Nesse momento u está na pilha e v está em um de três estados:

- v ainda não foi visitado: ele é empilhado por cima de u e, como a pilha é LIFO, sai antes de u;
- v já foi visitado e já saiu da pilha: então v saiu antes de u, pois u ainda está na pilha;
- v já foi visitado e ainda está na pilha: então v é um ancestral de u na busca, ou seja, existe um caminho de v até u. Junto com a aresta $u \to v$ isso formaria um ciclo, o que é impossível num DAG.

Portanto, nos dois casos possíveis, sai$(v) <$ sai(u). A hipótese de G ser acíclico é usada apenas no último caso. De fato, em um grafo com ciclo a aresta de retorno $u \to v$ satisfaz sai$(v) >$ sai(u), e a ordem por sai decrescente viola essa aresta.

Em uma frase: *ao terminar u, todos os seus sucessores já terminaram, pois ou já estavam terminados, ou foram explorados dentro da chamada de u; um sucessor ainda em andamento seria ancestral de u, o que daria um ciclo.*

Exemplo 5.2

Executando OrdTop(G) no grafo do Exemplo 5.1, com os vértices percorridos em ordem alfabética (em $V(G)$ e em cada $N^+(v)$), obtemos

v	a	b	c	d	e
chega[v]	1	2	6	3	7
sai[v]	10	5	9	4	8

e a lista ord no final é a, c, e, b, d, que lista os vértices em ordem decrescente de sai[ ] e é uma das ordenações do Exemplo 5.1. O arco (c, d) é transversal: é examinado quando d já terminou e, nesse caso, sai$[d] <$ sai$[c]$.

Para obter a lista sem ordenar, inserimos cada vértice no início de uma lista ord no instante em que ele termina. Como o último vértice a terminar é inserido por último, ele fica na frente: ao final, ord lista os vértices em ordem decrescente de sai[ ].

As variáveis cont, chega, sai, p, ord e circ são globais; o vetor p não é usado pelo algoritmo, apenas pela análise.

Algoritmo 26: VisitaOT(G, w).

Dado : um grafo dirigido G e um vértice w com chega[w] = 0.
Devolve : visita w e os vértices ainda não visitados alcançáveis a partir de w.

```
1  cont ← cont + 1;
2  chega[w] ← cont;
3  para cada u ∈ N⁺(w) faça
4      se chega[u] = 0 então
5          p[u] ← w;
6          VisitaOT(G, u);
7      senão
8          se sai[u] = 0 então circ ← verdadeiro;
9  cont ← cont + 1;
10 sai[w] ← cont;
11 insira w no início da lista ord;
```

Algoritmo 27: OrdTop(G).

Dado : um grafo dirigido G.
Devolve : uma ordenação topológica de G, se G é acíclico; caso contrário, a informação de que G contém circuito orientado.

```
1  para cada v ∈ V(G) faça
2      chega[v] ← 0;
3      sai[v] ← 0;
4      p[v] ← nil;
5  cont ← 0;
6  ord ← lista vazia;
7  circ ← falso;
8  para cada v ∈ V(G) faça
9      se chega[v] = 0 então VisitaOT(G, v);
10 se circ então
11     retorne G contém circuito orientado
12 retorne ord
```

Análise do algoritmo

Fixe um grafo dirigido G e considere uma execução de OrdTop(G). Pela estrutura recursiva do algoritmo valem as seguintes observações, que usaremos sem mais comentários.

(a) Para todo $v \in V(G)$ a chamada VisitaOT(G, v) é feita exatamente uma vez. De fato, ela só é feita se chega[v] = 0 e seus dois primeiros comandos tornam chega[v] $\neq$ 0; além disso, ao

final do laço principal todo vértice tem chega $\neq 0$. Em particular, cada arco (w, u) é examinado exatamente uma vez, no laço da chamada de w.

(b) O contador cont aumenta de uma unidade antes de cada atribuição a chega[ ] ou a sai[ ]. Logo esses $2|V(G)|$ valores são inteiros distintos e chega[v] < sai[v] para todo v.

(c) As chamadas recursivas são aninhadas: a chamada de u feita na chamada de w termina antes de w prosseguir. Em cada instante, os vértices com chega $\neq 0$ e sai $= 0$ (chamadas *ativas*) são o vértice z cuja chamada está em execução e seus ancestrais na floresta definida por p[ ].

(d) Se chega[w] < chega[x] < sai[w], então x foi descoberto durante a chamada de w, portanto por uma chamada aninhada nela; logo x é descendente próprio de w e sai[x] < sai[w].

(e) Para todo arco $(w, u) \in E(G)$, ao final do laço da chamada de w temos chega[u] $\neq 0$: ou já era diferente de zero ao examinar o arco, ou a chamada recursiva o tornou. Como sai[w] é atribuído depois do laço, (b) dá chega[u] < sai[w].

(As relações entre os intervalos [chega, sai] e ancestralidade são as da Proposição 3.2 e do Exercício 58.)

Lema 5.1: Exame de um arco

Considere o instante em que um arco $(w, u) \in E(G)$ é examinado no laço da linha 3, durante a chamada de w. Então vale exatamente uma das situações:

1. chega[u] $= 0$; então u é visitado em seguida, com p[u] $= w$, e sai[u] < sai[w];
2. chega[u] $\neq 0$ e sai[u] $\neq 0$; então sai[u] < sai[w];
3. chega[u] $\neq 0$ e sai[u] $= 0$; então u é ancestral próprio de w (isto é, (w, u) é um arco de retorno ascendente) e sai[w] < sai[u].

Reciprocamente, se u é ancestral próprio de w, o arco (w, u) cai na situação 3.

Demonstração. Os três casos são mutuamente exclusivos e esgotam as possibilidades, pois sai[u] $\neq 0$ implica chega[u] $\neq 0$ por (b).

Caso 1. O algoritmo faz p[u] $\leftarrow w$ e chama VisitaOT(G, u), que termina, e atribui sai[u], antes de w prosseguir. Como sai[w] ainda não foi atribuído, por (b) temos sai[u] < sai[w].

Caso 2. O valor sai[u] já foi atribuído, enquanto sai[w] ainda não, pois a chamada de w está em execução. Por (b), sai[u] < sai[w].

Caso 3. A chamada de u está ativa. No instante do exame do arco, a chamada em execução é a de w, logo, por (c), os vértices ativos são w e seus ancestrais. Como G não tem laços, $u \neq w$, portanto u é ancestral próprio de w. Sendo a chamada de w aninhada na de u, ela termina

antes, isto é, $\text{sai}[w] < \text{sai}[u]$.

Reciprocamente, se u é ancestral próprio de w então, durante a chamada de w, a chamada de u está ativa por (c), o que é a situação 3. □

Proposição 5.3: Circuitos e retornos ascendentes

Em uma execução de OrdTop(G) ocorre algum arco de retorno ascendente se e somente se G contém circuito orientado.

Demonstração. (⇒) Seja (w, u) um arco de retorno ascendente, isto é, u é ancestral próprio de w. Então existe $t \geq 1$ com $\text{p}^{(t)}[w] = u$ e, definindo $y_j := \text{p}^{(t-j)}[w]$ para $0 \leq j \leq t$, temos $y_0 = u$, $y_t = w$ e $\text{p}[y_j] = y_{j-1}$ para $1 \leq j \leq t$. Como $\text{p}[y_j]$ só é atribuído ao examinar o arco $(\text{p}[y_j], y_j)$, temos $(y_{j-1}, y_j) \in E(G)$. Os vértices $y_0, \ldots, y_t$ são distintos, pois $\text{chega}[\text{p}[v]] < \text{chega}[v]$ para todo v (o pai é descoberto antes do filho) e, com o arco $(w, u) = (y_t, y_0)$, formam o circuito orientado $y_0, y_1, \ldots, y_t, y_0$.

(⇐) Seja $C = x_1, \ldots, x_k, x_1$ um circuito orientado em G. Trocando a numeração por uma rotação cíclica, podemos supor que $\text{chega}[x_1]$ é o menor valor de chega entre os vértices de C. Mostramos por indução em i, para $i = 2, \ldots, k$, que

$$\text{chega}[x_1] < \text{chega}[x_i] < \text{sai}[x_1].$$

Em particular, por (d), cada x_i é descendente próprio de x_1 e $\text{sai}[x_i] < \text{sai}[x_1]$.

Para $i = 2$, por (e) aplicado a (x_1, x_2) temos $\text{chega}[x_2] < \text{sai}[x_1]$, e $\text{chega}[x_2] > \text{chega}[x_1]$ pela escolha de x_1, pois $x_2 \neq x_1$. Isso mostra Q_2.

Suponha Q_i para algum $i < k$. Por (d), x_i é descendente próprio de x_1 e $\text{sai}[x_i] < \text{sai}[x_1]$; por (e) aplicado a (x_i, x_{i+1}),

$$\text{chega}[x_{i+1}] < \text{sai}[x_i] < \text{sai}[x_1],$$

e $\text{chega}[x_{i+1}] > \text{chega}[x_1]$ pela escolha de x_1. Isso mostra Q_{i+1}.

Assim x_k é descendente próprio de x_1, isto é, x_1 é ancestral próprio de x_k, e $(x_k, x_1) \in E(G)$ é um arco de retorno ascendente. □

Teorema 5.2: Corretude

Ao final de OrdTop(G) o indicador circ é verdadeiro se e somente se G contém circuito orientado. Se G é acíclico, a lista ord é uma ordenação topológica de G.

Demonstração. O indicador circ é atribuído na linha 8 precisamente quando um arco é examinado na situação 3 do Lema 5.1, isto é, quando

ocorre um arco de retorno ascendente. Pela Proposição 5.3, isso acontece se e somente se G contém circuito orientado.

Suponha G acíclico. Pelo que acabamos de provar, nenhum arco é examinado na situação 3, portanto todo arco (w, u) cai na situação 1 ou 2 e satisfaz

$$\text{sai}[u] < \text{sai}[w].$$

Por (a), cada vértice é inserido em ord exatamente uma vez, na linha 11, logo após receber seu valor de sai[ ]. Como as inserções são no início da lista e os valores de sai[ ] são atribuídos em ordem crescente, ord contém os vértices em ordem decrescente de sai[ ]. Assim, para todo arco (w, u), o vértice w aparece antes de u em ord, e ord é uma ordenação topológica de G. □

Teorema 5.3: Complexidade

Se G é dado por listas de adjacência $N^+[\ ]$ e a inserção no início de ord custa $O(1)$, então OrdTop(G) tem complexidade de tempo $O(|V| + |E|)$.

Demonstração. A inicialização custa $O(|V|)$ e o laço principal faz $|V|$ testes de custo $O(1)$, fora as chamadas. Por (a), cada vértice w dispara uma única chamada de VisitaOT, que, descontadas as chamadas recursivas, executa $O(1)$ operações fora do laço da linha 3 (incluindo a inserção na linha 11) e $\mathrm{d}^+(w)$ iterações do laço, cada uma de custo $O(1)$. O custo total é, portanto,

$$O(|V|) + \sum_{w \in V} O\big(1 + \mathrm{d}^+(w)\big) = O(|V| + |E|),$$

pois $\sum_{w \in V} \mathrm{d}^+(w) = |E|$. □

Se G é dado por sua matriz de adjacência, percorrer $N^+(w)$ custa $\Theta(|V|)$ para cada w e o algoritmo tem complexidade $\Theta(|V|^2)$. A pilha de recursão tem profundidade até $|V|$, e o espaço total é $O(|V|)$ além da representação de G.

5.3 Componentes fortemente conexas

Definição 5.4: componente fortemente conexa

Uma ***componente fortemente conexa*** num grafo dirigido G é um subconjunto de vértices $C \subseteq V(G)$ maximal com respeito a seguinte propriedade: para quaisquer $u, v \in C$ existem um $(u \to v)$-caminho orientado e um $(v \to u)$-caminho orientado, ambos contidos em $G[C]$.

Equivalentemente, a relação $\leftrightsquigarrow$ sobre V dada por $u \leftrightsquigarrow$ se e somente se existem $(u \to v)$-caminho e $(v \to u)$-caminho orientados em G

é de equivalência. As classes de equivalência são as componentes fortemente conexas do grafo dirigido G.

O diagrama da Figura 5.9 abaixo mostra um grafo dirigido, cada um dos quatro círculos identifica uma componente fortemente conexa do grafo.

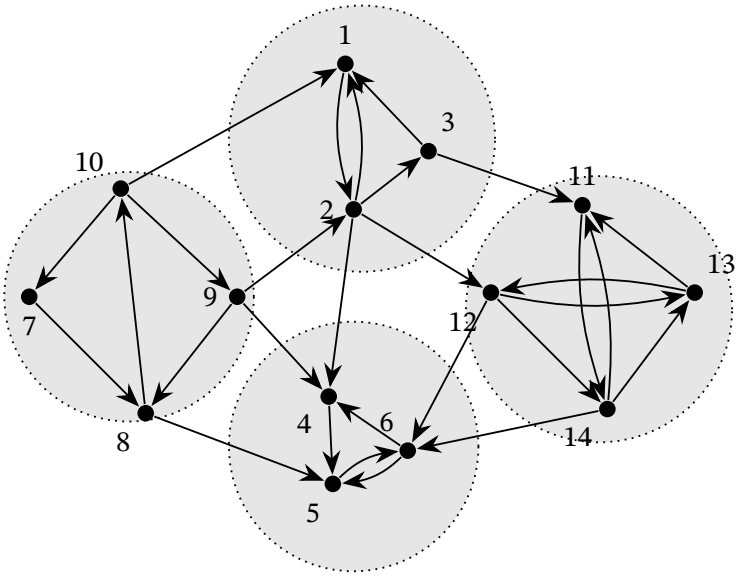


Figura 5.9: Um grafo dirigido e suas componentes fortemente conexas, destacadas em cinza.

Observemos que se um caminho orientado deixa uma componente fortemente conexa C, então esse caminho não pode retornar a C pois, caso contrário, todos os vértices de C deveriam pertencer a C. Em particular, cale o seguinte resultado.

Proposição 5.4

Sejam C e C' componentes fortemente conexas distintas de um grafo dirigido simples G. Então todas as arestas entre C e C' têm o mesmo sentido: ou todas vão de C para C', ou todas vão de C' para C.

Demonstração. Suponha, por absurdo, que existam arestas $u \to v$ e $y \to w$ com $u, w \in C$ e $v, y \in C'$.

Como C é fortemente conexa, existe um caminho de w até u dentro de C. Como C' é fortemente conexa, existe um caminho de v até y dentro de C'. Concatenando, obtemos um caminho de v até u

$$v \rightsquigarrow y \to w \rightsquigarrow u.$$

Junto com a aresta $u \to v$, isso mostra que u e v são alcançáveis um a partir do outro, isto é, u e v pertencem à mesma componente fortemente conexa. Mas $u \in C$, $v \in C'$ e $C \neq C'$, o que é uma contradição. □

Em vista disso que acabamos de provar, fica bem definido o grafo das componentes que passamos a definir em seguida.

Definição 5.5: Grafo G^* das cfc

O ***grafo das componentes*** de G, denotado por G^* é o grafo orientado com um vértice para cada componente fortemente conexa de G e (c_1, c_2) é um arco em G^* se, e somente se, existe um arco que sai de um vértice de c_1 e chega num vértice de c_2 (veja um exemplo na Figura 5.10(a) abaixo).

Proposição 5.5: G^* é G.D.A.

Seja G um grafo dirigido e seja G^* o grafo das componentes fortemente conexas de G. Então G^* é acíclico.

Demonstração. Suponha, por absurdo, que G^* tenha um ciclo dirigido

$$C_1 \to C_2 \to \cdots \to C_k \to C_1,$$

com $k \geq 2$ e as componentes $C_1, \ldots, C_k$ duas a duas distintas (basta tomar um ciclo de comprimento mínimo; $k \geq 2$ pois G^* não tem laços).

Para cada i, pela definição das arestas de G^{SCC}, existe em G uma aresta $u_i \to v_{i+1}$ com $u_i \in C_i$ e $v_{i+1} \in C_{i+1}$ (os índices são tomados módulo k). Como C_i é fortemente conexa, existe um caminho em G de v_i até u_i dentro de C_i. Assim, para cada i há um caminho de v_i até v_{i+1}:

$$v_i \rightsquigarrow u_i \to v_{i+1}.$$

Concatenando esses caminhos para $i = 1, \ldots, k$, obtemos um caminho de v_1 até $v_{k+1} = v_1$ passando por $v_2, \ldots, v_k$. Em particular, existe um caminho de v_2 até v_1 (seguindo $v_2 \rightsquigarrow v_3 \rightsquigarrow \cdots \rightsquigarrow v_k \rightsquigarrow v_1$) e um caminho de v_1 até v_2. Logo v_1 e v_2 são mutuamente alcançáveis e pertencem à mesma componente fortemente conexa. Mas $v_1 \in C_1$, $v_2 \in C_2$ e $C_1 \neq C_2$, uma contradição. □

Corolário 5.1

Os vértices de G^* admitem uma ordenação topológica. □

Na figura abaixo mostramos (a) o grafo das componentes fortemente conexas do grafo dirigido mostrado na Figura 5.9 e também (b) uma ordenação topológica dos vértices do grafo das componentes.

Agora, vamos descrever um algoritmo que, dado um grafo dirigido G, determina as componentes fortemente conexas de G. Para isso, definimos, ainda, o ***grafo transposto***: denotado por $G^\intercal$ é o grafo dirigido obtido invertendo-se o sentido dos arcos em $E(G)$: $G^\intercal = \big(V(G), \{(u, v) \in V \times V : (v, u) \in E(G)\}\big)$.

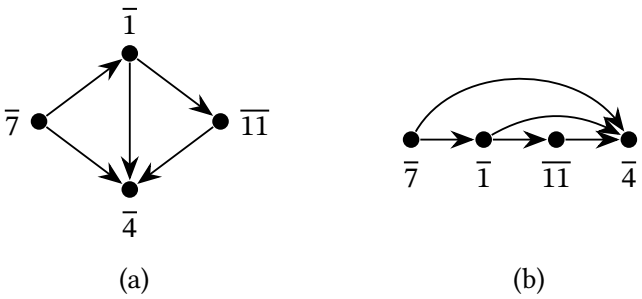


Figura 5.10: O grafo das componentes fortemente conexas (a) e uma ordenação topológica (b).

Observação 5.1: Transposição não altera componentes

Os grafos G e $G^\intercal$ têm as mesmas componentes fortemente conexas. De fato, se u e v estão no mesmo componente C de G então o $(u \to v)$-caminho em G com todos os vértices em C é um $(v \to u)$-caminho em $G^\intercal$ com todos os vértices em C e, analogamente, o $(v \to u)$-caminho em G é um $(u \to v)$--caminho em $G^\intercal$, portanto, u e v estão no componente C de $G^\intercal$.

Observação 5.2: Transposição inverte G^*

Para componentes distintas C e C', existe aresta $C \to C'$ em G^* se e somente se existe aresta $C' \to C$ em $(G^\intercal)^*$. Isto é, $(G^\intercal)^* = (G^*)^\intercal$. De fato, uma aresta $u \to v$ de G com $u \in C$ e $v \in C'$ corresponde à aresta $v \to u$ de $G^\intercal$, de C' para C, e vice-versa.

Intuição

Se contraímos cada componente fortemente conexa de G a um vértice, obtemos o G.D.A. G^*. Uma componente *fonte* de G^* é como uma "ilha de partida": dela se chega a outras componentes, mas nenhuma componente chega a ela. Se começássemos uma busca em um vértice de uma componente *sumidouro* de G^*, a busca visitaria exatamente essa componente e pararia, pois não há como sair dela. Bastaria então repetir: remover essa componente e buscar de novo.

O problema é que não sabemos de antemão quais vértices estão em componentes sumidouro. Dois truques resolvem isso.

- A primeira busca, em G, produz os valores de sai, e o vértice com maior sai está sempre em uma componente *fonte* de G^*: componentes "mais à montante", mais próximas das fontes, terminam depois.
- Ao transpor G, todas as arestas entre componentes se invertem, e uma componente fonte vira um *sumidouro*. A segunda busca, em $G^\intercal$, escolhendo raízes em ordem decrescente de sai, começa então sempre em um sumidouro do que ainda não foi visitado, e extrai uma componente de cada vez, sem "vazar" para as outras.

Em linhas gerais, a estratégia para computar as componentes fortemente conexas de um grafo dirigido G é

Algoritmo 28: CFC(G)

1 BP(G): busca em profundidade rotulada em G para determinar sai[v] para todo $v \in V(G)$;
2 Compute $G^\intercal$;
3 BP($G^\intercal$): busca em profundidade em $G^\intercal$ com o laço principal modificado para escolher os vértices em ordem decrescente dos valores de *sai*[ ] computado no passo 1;
4 Devolva cada árvore da segunda busca como uma componente fortemente conexa de G.

Notação e fatos preliminares

No que segue sempre que nos referirmos aos valores de chega[ ] e sai[ ] trata-se do conteúdo desses vetores após a primeira busca em profundidade, salvo menção em contrário.

Seja $G = (V, E)$ um grafo dirigido. Para $C \subseteq V$ definimos

$$\text{chega}(C) := \min\{\text{chega}[v] : v \in C\}, \quad \text{sai}(C) := \max\{\text{sai}[v] : v \in C\}.$$

Proposição 5.6: o primeiro vértice de C tem o maior sai

Seja C uma componente fortemente conexa e seja $x \in C$ o vértice com $\text{chega}(x) = \text{chega}(C)$. Então $\text{sai}(x) = \text{sai}(C)$, e $\text{sai}(x) > \text{sai}(v)$ para todo $v \in C \setminus \{x\}$.

Demonstração. No instante chega[x], todo vértice de C diferente de x é não visitado, pois x é o primeiro de C a ser descoberto. Como C é fortemente conexa, para cada $v \in C$ existe um caminho de x a v dentro de C, portanto formado só por vértices não visitados (além de x).

Observação 5.3: Teorema do caminho branco

Em uma busca em profundidade, um vértice y é descendente de x na floresta de busca se e somente se, no instante chega[x], existe um caminho de x a y cujos vértices são todos ainda não visitados.

Pelo fato acima, todo $v \in C \setminus \{x\}$ é descendente de x, e portanto termina antes de x: $\text{sai}[v] < \text{sai}[x]$. □

Os seguintes resultados ajudam a entender o funcionamento do algoritmo.

Lema 5.2

Sejam C e C' duas componentes fortemente conexas de G. Suponha que existe um arco $(u, v) \in E(G)$ tal que $u \in C$ e $v \in C'$. Então $\text{sai}(C) > \text{sai}(C')$.

Intuitivamente: se C aponta para C', então C' é "mais ao sumidouro" e termina antes.

Demonstração. Como G^* é acíclico, não existe caminho em G de nenhum vértice de C' a nenhum vértice de C. (Caso contrário, juntando-o à aresta de C para C' e a caminhos dentro das componentes, teríamos um ciclo passando por C e C' em G^*.) Consideramos dois casos.

Caso 1: $\text{chega}(C) < \text{chega}(C')$. Seja $x \in C$ com $\text{chega}(x) = \text{chega}(C)$. No instante $\text{chega}[x]$, todos os vértices de $C \cup C'$ diferentes de x são brancos: os de C porque x é o primeiro de C a ser descoberto, e os de C' porque $\text{chega}(C') > \text{chega}[x]$. Seja $u \to v$ uma aresta de G com $u \in C$ e $v \in C'$. Para qualquer $w \in C'$, existe um caminho

$$x \rightsquigarrow u \to v \rightsquigarrow w,$$

com o primeiro trecho dentro de C, o último dentro de C', e todos os vértices (exceto x) não visitados. Portanto, pelo teorema do caminho branco (Obs. 5.3) todo vértice de C' é descendente de x, e portanto $\text{sai}[w] < \text{sai}[x]$ para todo $w \in C'$. Pela Proposição 5.6, $\text{sai}[x] = \text{sai}(C)$, logo $\text{sai}(C') < \text{sai}(C)$.

Caso 2: $\text{chega}(C') < \text{chega}(C)$. Seja $y \in C'$ com $\text{chega}[y] = \text{chega}(C')$. Pela Proposição 5.6, $\text{sai}(C') = \text{sai}[y]$. Todo descendente de y é alcançável a partir de y; como nenhum vértice de C é alcançável a partir de um vértice de C', nenhum vértice de C é descendente de y. Logo nenhum vértice de C é descoberto durante o intervalo $[\text{chega}[y], \text{sai}[y]]$. Como $\text{chega}(C) > \text{chega}(C') = \text{chega}[y]$, segue que $\text{chega}(C) > \text{sai}[y] = \text{sai}(C')$. Portanto

$$\text{sai}(C) > \text{chega}(C) > \text{sai}(C').$$

Nos dois casos, $\text{sai}(C) > \text{sai}(C')$. □

Observação 5.4

Seja $\mathcal{U}$ um conjunto de componentes e seja $C \in \mathcal{U}$ com $\text{sai}(C)$ máximo em $\mathcal{U}$. Então não existe aresta $C' \to C$ em G^* com $C' \in \mathcal{U}$, $C' \neq C$. Com efeito, pelo Lema 5.2 teríamos $\text{sai}(C') > \text{sai}(C)$, contrariando a maximalidade de C.

Corolário 5.2

Sejam C e C' duas componentes fortemente conexas de G. Suponha que existe um arco $(u, v) \in E(G^\intercal)$ tal que $u \in C$ e $v \in C'$. Então

$$\text{sai}(C) < \text{sai}(C').$$

Demonstração. Imediato da definição de $G^\intercal$ e do resultado acima pois G e $G^\intercal$ têm as mesmas componentes fortemente conexas. □

Intuitivamente, ao transpor o grafo, a componente que era fonte (entre as não visitadas) passa a ser sumidouro, e a busca não consegue escapar dela.

Análise e correção do algoritmo CFC

Teorema 5.4: Corretude de CFC

Cada árvore da floresta produzida pela segunda busca (em $G^\intercal$, com raízes escolhidas em ordem decrescente do valor sai calculado na primeira busca) é exatamente o conjunto de vértices de uma componente fortemente conexa de G, e toda componente aparece como uma árvore.

Demonstração. Sejam $r_1, r_2, \ldots$ as raízes da segunda busca, na ordem em que são escolhidas. Mostramos, por indução no número de árvores já construídas, que vale o seguinte invariante:

> (I) *imediatamente antes da escolha de cada raiz, o conjunto S dos vértices já visitados pela segunda busca é uma união de componentes fortemente conexas de G, e cada árvore construída até então é exatamente uma dessas componentes.*

Para a base, $S = \varnothing$. Para o passo, suponha que (I) vale antes da escolha de uma raiz r e seja C a componente fortemente conexa de G que contém r. Seja $\mathcal{U}$ o conjunto das componentes contidas em $\overline{S}$; por (I), $\overline{S}$ é a união das componentes de $\mathcal{U}$. Como $r \notin S$, temos $C \in \mathcal{U}$, ou seja, nenhum vértice de C foi visitado até este instante. Além disso, os vértices ainda não visitados pela segunda busca são exatamente os de $\overline{S}$.

Afirmação 1: $\text{sai}(C) \geq \text{sai}(D)$ *para toda* $D \in \mathcal{U}$. Pela regra de escolha, r é o vértice de $\overline{S}$ com maior valor de sai. Como $r \in C \subseteq \overline{S}$, temos $\text{sai}(C) = \text{sai}[r]$. Por outro lado, para cada $D \in \mathcal{U}$, o valor $\text{sai}(D)$ é atingido por um vértice de $D \subseteq \overline{S}$, logo $\text{sai}(D) \leq \text{sai}[r] = \text{sai}(C)$.

Afirmação 2: nenhum arco de $G^\intercal$ *vai de* C *a um vértice de* $\overline{S}$ *fora de* C. Suponha que $(u, v) \in E(G^\intercal)$ com $u \in C$, $v \in \overline{S}$ e $v \notin C$, e seja D a componente de v. Então $D \in \mathcal{U} \setminus \{C\}$ e, pelo Corolário 5.2, $\text{sai}(C) < \text{sai}(D)$, o que contradiz a Afirmação 1.

Afirmação 3: o conjunto de vértices da árvore T_r da segunda busca com raiz r é exatamente C. No instante em que r é descoberto, os vértices ainda não visitados são exatamente os de $\overline{S}$. Assim:

- ($\supseteq$) Seja $v \in C$. Pela Observação 5.1, C é fortemente conexa em $G^\intercal$, logo existe em $G^\intercal$ um caminho de r a v com todos os vértices em $C \subseteq \overline{S}$, isto é, todos ainda não visitados. Pelo teorema do caminho de vértices não visitados (Obs. 5.3), v é descendente de r.
- ($\subseteq$) Seja z um descendente de r. Pelo mesmo teorema, existe em $G^\intercal$ um caminho de r a z cujos vértices pertencem todos a $\overline{S}$. Se $z \notin C$, a primeira aresta desse caminho que sai de C é um arco de $G^\intercal$ de C a um vértice de $\overline{S}$ fora de C, o que contradiz a Afirmação 2. Logo $z \in C$.

Pela Afirmação 3, a árvore de r é exatamente a componente C e, portanto, ao final de sua construção, o conjunto de vértices visitados é $S \cup C$, uma união de componentes: o invariante (I) é preservado.

Como o laço principal só termina quando todos os vértices foram visitados, cada vértice pertence a alguma árvore, e cada árvore é a componente de sua raiz. Logo cada árvore é uma componente fortemente conexa de G e toda componente aparece como uma (única) árvore. □

Uma implementação

O Algoritmo 30 usa o vetor cfc[ ] para identificar, no final da execução, as componentes fortemente conexas de G. Os vetores chega, sai e cfc e o contador *cont* são variáveis globais. A busca em profundidade utilizada é o Algoritmo 29:

Algoritmo 29: BP(G, v, cc)

Dado : um grafo dirigido G, um vértice v e um rótulo cc.

Devolve : os vetores chega[ ], sai[ ] e cfc[ ] atualizados para os vértices alcançados a partir de v.

```
1 cfc[v] ← cc;
2 cont ← cont + 1;
3 chega[v] ← cont;
4 para cada u ∈ N⁺_G(v) faça
5     se chega[u] = 0 então BP(G, u, cc) ;
6 cont ← cont + 1;
7 sai[v] ← cont.
```

O parâmetro *cc* rotula os vértices visitados pela busca com o índice da componente fortemente conexa à qual pertencem: todos os vértices alcançados por uma chamada de BP a partir da raiz v recebem cfc[·] = *cc*. Na primeira busca, feita em G, as componentes ainda não são

conhecidas e o rótulo não é usado; por isso passamos $cc = 0$, que significa "sem rótulo". Os rótulos $1, 2, \ldots$ só são atribuídos na segunda busca, feita em $G^\intercal$, e o algoritmo CFC fica

Algoritmo 30: CFC(G)

Dado : um grafo dirigido G.
Devolve : vetor cfc[ ] das componentes fortemente conexas.

```
1  cont ← 0;
2  para cada k ∈ V(G) faça
3  |   sai[k] ← 0;
4  |   chega[k] ← 0;
5  para cada k ∈ V(G) faça
6  |   se chega[k] = 0 então BP(G, k, 0) ;
7  L ← lista ordenada de V(G) por ordem decrescente
     de sai[ ];
8  compute G^T;
9  cont ← 0;
10 c ← 0;
11 para cada k ∈ V(G) faça
12 |   sai[k] ← 0;
13 |   chega[k] ← 0;
14 para cada k ∈ L faça
15 |   se chega[k] = 0 então
16 |   |   c ← c + 1;
17 |   |   BP(G^T, k, c);
18 retorne cfc
```

Complexidade

A primeira busca custa $O(|V| + |E|)$, a construção de $G^\intercal$ custa $O(|V| + |E|)$, a ordenação (*counting sort*) e a segunda busca custam $O(|V|+|E|)$, desde que os vértices sejam percorridos na ordem decrescente de sai (basta guardar os vértices em uma lista, na ordem de término, durante a primeira busca, e percorrê-la ao contrário). O custo total é $O(|V| + |E|)$.

Exercícios

Exercício 84 (***Fecho transitivo***). Se $D = (V, E)$ é um grafo dirigido então o fecho transitivo de D é o grafo $D^* = (V, E^*)$ onde

$$E^* = \{(i, j) \in V \times V : \text{ existe um } (i \to j)\text{-caminho em } D\}.$$

Escreva um algoritmo de tempo $O(|V|^3)$ para determinar o fecho transitivo de um grafo dirigido.

Exercício 85. Escreva um algoritmo que determina uma ordenação topológica removendo repetidamente uma fonte, como na demonstração do Teorema 5.1. Mantenha um vetor com os graus de entrada $d^-[\,]$ e uma fila com as fontes atuais para obter tempo $O(|V| + |E|)$. Como decidir, ao fim, se G é acíclico?

Exercício 86. Quantas ordenações topológicas tem o grafo do Exemplo 5.1?

Exercício 87. Prove que um grafo dirigido acíclico com n vértices tem no máximo $\binom{n}{2}$ arcos e que existe um GDA com exatamente $\binom{n}{2}$ arcos.

Exercício 88. Prove que um grafo dirigido acíclico tem uma única ordenação topológica se e somente se contém um caminho orientado que passa por todos os vértices.

Exercício 89. Modifique o algoritmo OrdTop para que, quando G contém circuito orientado, ele devolva um circuito orientado de G. (Dica: use o vetor p[ ] e a demonstração da Proposição 5.3.)

Exercício 90. Mostre que listar os vértices em ordem *crescente* de chega[ ] não dá, em geral, uma ordenação topológica de um GDA. (Dica: considere o grafo com $V = \{1, 2, 3\}$ e arcos $(1, 3)$ e $(2, 3)$.)

Exercício 91. Mostre que um grafo dirigido é acíclico se e somente se todas as suas componentes fortemente conexas (Seção 5.3) têm um único vértice.

Exercício 92. Se ao invés de usarmos uma busca em profundidade em $G^\intercal$ por ordem decrescente de *sai*[ ] no algoritmo 30 usarmos uma busca em profundidade em G por ordem crescente de *chega*[ ] o algoritmo determinaria as componentes fortemente conexas do grafo?

Exercício 93. Se ao invés de usarmos uma busca em profundidade em $G^\intercal$ por ordem decrescente de *sai*[ ] no algoritmo 30 usarmos uma busca em profundidade em G por ordem crescente de *sai*[ ] o algoritmo determinaria as componentes fortemente conexas do grafo?

Exercício 94. Seja G um grafo dirigido e u e w vértices de G.

1. Escrevemos $u \rightsquigarrow w$ se existe $(u \to w)$-caminho em G. Prove que $\rightsquigarrow$ é uma relação simétrica e transitiva em $V(G)$.
2. Escrevemos $u \leftrightsquigarrow w$ se existem $(u \to w)$- e $(w \to u)$-caminhos em G. Prove que $\leftrightsquigarrow$ é uma relação de equivalência em $V(G)$. Nesse caso, quem são as classes de equivalência?

Exercício 95. Escreva um algoritmo para determinar se um grafo dirigido G é ***semiconexo***: para $u, v \in V(G)$ ou $u \rightsquigarrow v$ ou $v \rightsquigarrow u$. Determine a complexidade do algoritmo.

Exercício 96. Escreva um algoritmo de complexidade $O(|V|+|E|)$ para computar o grafo G^* das componentes fortemente conexas de G.

Exercício 97. Escreva um algoritmo de complexidade $O(|V|+|E|)$ para determinar o grafo transposto de um grafo dado por uma lista de adjacências.

Caminhos mínimos 6

Além da orientação nos arcos, os grafos desta seção têm peso nas arcos, ou seja, esses grafos são definidos por uma tripla (V, E, ρ) com $\rho : E \to \mathbb{R}_+$ (escrevemos $\rho(u,t)$ em vez de $\rho\big((u,t)\big)$). O ***comprimento*** de um caminho orientado $P = x_1, x_2, \dots, x_k$ é definido, naturalmente, como a soma dos pesos (comprimentos) das arcos nesse caminho

$$c(P) = \sum_{i=1}^{k-1} \rho(x_i, x_{i+1}),$$

e a ***distância*** entre dois vértices é o comprimento do menor caminho orientado que os liga,

$$\operatorname{dist}_G(u,v) = \min\big\{c(P) : P \text{ é um } (u \to v)\text{-caminho}\big\}$$

quando existe algum caminho, caso contrário convencionamos que $\operatorname{dist}_G(u,v) = \infty$ com as convenções adotadas na seção 1.3. Um $(u \to v)$-caminho de comprimento $\operatorname{dist}_G(u,v)$ é chamado de ***caminho mínimo***.

O seguinte diagrama representa um grafo dirigido com pesos nas arcos

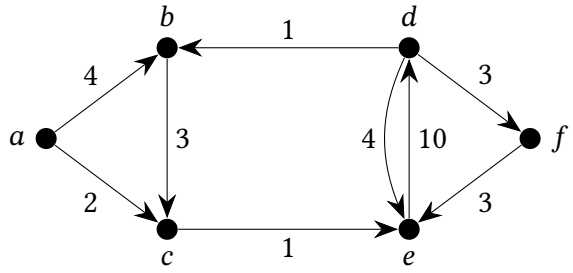


Figura 6.1: Diagrama de um grafo dirigido com peso nas arcos. Note que $\operatorname{dist}(c,e) \neq \operatorname{dist}(e,c)$, $\operatorname{dist}(a,f) = 10$.

Em um grafo dirigido G podemos ter

$$\operatorname{dist}_G(u,v) \neq \operatorname{dist}_G(v,u).$$

Para um grafo dirigido G qualquer, com pesos nos arcos, e s um vértice de G as seguintes propriedades são satisfeitas:

***desigualdade triangular*:** vale a seguinte variante da desigualdade triangular (1.6): para todo arco $(v,u) \in E(G)$

$$\operatorname{dist}_G(s,u) \leq \operatorname{dist}_G(s,v) + \rho(v,u). \tag{6.1}$$

***subestrutura ótima*:** Se $P = v_1, v_2, \dots, v_k$ é um caminho mínimo num grafo G então $P_{i,j} = v_i, \dots, v_j$ é um caminho mínimo com extremos v_i e v_j em G para quaisquer i, j com $1 \leq i < j \leq k$.

6.1 Algoritmo de Dijkstra

O ***algoritmo de Dijkstra*** foi concebido por Edsger W. Dijkstra em 1956 e publicado em 1959 na nota *A note on two problems in connexion with graphs*, na *Numerische Mathematik*. Segundo o próprio Dijkstra, ele o inventou em cerca de vinte minutos, sem papel e lápis, pensando num exemplo para demonstrar o computador ARMAC[1] a um público sem formação técnica: o menor caminho entre duas cidades de um mapa rodoviário. A mesma nota trata traz o algoritmo de Prim que estudamos na Seção 9.

Há registros de que outros autores chegaram a ideias próximas na mesma época.

1: *Automatic Relay and Magnetic-drum Automatic Calculator*, foi um computador Holandês construído em 1956 no Centro Matemático de Amsterdã, atual *Centrum Wiskunde & Informatica* (CWI), prestigioso centro de pesquisa na área de matemática e ciência da computação teórica.

São dados um grafo dirigido $G = (V, E)$, pesos $\rho : E \to \mathbb{R}$ e um vértice $s \in V$. Queremos determinar $\text{dist}_G(s, v)$ para todo $v \in V$. Para isso, usaremos um vetor d[ ] indexado pelos vértices, um vetor p[ ] de predecessores e um subconjunto $S \subseteq V$ de modo que, em qualquer instante da execução do algoritmo,

S: o conjunto S contém somente os vértices $v \in V$ que já têm a distância $\text{dist}_G(s, v)$ computada corretamente;

d[ ]: para todo $v \in V$,

- se $v \in S$, então $\text{d}[v] = \text{dist}_G(s, v)$;
- se $v \in \overline{S} = V \setminus S$ e $\text{d}[v] < \infty$, então $\text{d}[v] = c(P)$ para algum $(s \to v)$-caminho orientado P;
- se $v \in \overline{S}$ e v ainda não foi alcançado, então $\text{d}[v] = \infty$.

O algoritmo que descrevemos a seguir recebe G e s como acima e funciona da seguinte forma.

1. De início $\text{d}[s] = 0$, pois s está a distância 0 dele mesmo, $S = \varnothing$ e $\text{d}[v] = \infty$ para todo $v \neq s$. Assim, na primeira iteração o vértice escolhido é s, o único com d finito.
2. Enquanto $\overline{S} \neq \varnothing$, retiramos de $\overline{S}$ o vértice u de menor valor de d[ ] e o colocamos em S. Provaremos a seguir que u é o vértice de $\overline{S}$ mais próximo de s e que, portanto, $\text{d}[u] = \text{dist}_G(s, u)$.
3. Em seguida, visitamos cada sucessor $t \in N_G^+(u)$. Como $\text{d}[u] = \text{dist}_G(s, u)$, a desigualdade triangular diz que $\text{dist}_G(s, t) \leq \text{d}[u] + \rho(u, t)$; testamos então se um caminho de custo menor até t, passando por u, foi encontrado e, em caso afirmativo, atualizamos $\text{d}[t]$ e $\text{p}[t]$. Essa etapa é chamada de ***relaxação*** do arco (u, t):

Algoritmo 31: Relaxação(u, t).

Dado : um arco $(u, t) \in E(G)$.
Devolve : atualiza $\text{d}[t]$ e $\text{p}[t]$ se um caminho de menor custo até t foi encontrado passando por u.

```
1 se d[t] > d[u] + ρ(u, t) então
2     d[t] ← d[u] + ρ(u, t);
3     p[t] ← u;
```

Notemos que a condição na Relaxação(u,t) é sempre falsa para $t \in S$: se u acabou de entrar em S, então $\mathrm{d}[t] = \mathrm{dist}_G(s,t) \leq \mathrm{dist}_G(s,u) + \rho(u,t) = \mathrm{d}[u] + \rho(u,t)$. O algoritmo prossegue até que $S = V$.

O seguinte algoritmo é conhecido como ***algoritmo de Dijkstra***.

Algoritmo 32: Dijkstra(G,s).

Dado : um grafo dirigido G, pesos $\rho(e) \geq 0$ nos arcos e um vértice $s \in V(G)$.

Devolve : $\mathrm{d}[v] = \mathrm{dist}_G(s,v)$ para todo $v \in V(G)$ e um vetor p de predecessores.

```
para cada v ∈ V(G) faça
    d[v] ← ∞;
    p[v] ← nil;
d[s] ← 0;
S ← ∅;
enquanto S̄ ≠ ∅ faça
    seja u tal que d[u] = min{d[v] : v ∈ S̄};
    insira u em S;
    para cada t ∈ N⁺_G(u) faça
        Relaxação(u, t);
retorne d, p
```

Uma demonstração formal de que o algoritmo funciona, isto é, no final $\mathrm{d}[v] = \mathrm{dist}_G(s,v)$ para todo $v \in V$, e a complexidade do algoritmo são determinadas a seguir.

Análise e correção do algoritmo de Dijkstra

Vamos mostrar que o algoritmo funciona corretamente.

Proposição 6.1

Em qualquer momento da execução de Dijkstra(G,s) valem os seguintes fatos.

(a) $\mathrm{d}[v] \geq \mathrm{dist}_G(s,v)$ para todo $v \in V$ e, se $\mathrm{d}[v] = \mathrm{dist}_G(s,v)$, o valor de $\mathrm{d}[v]$ não muda.
(b) Se $\mathrm{dist}_G(s,v) = \infty$ então $\mathrm{d}[v] = \infty$.

Demonstração. Para provar (a) vamos supor o contrário e assumir que $w \in V$ é o primeiro vértice, em ordem temporal, para o qual ocorreu

$$\mathrm{d}[w] < \mathrm{dist}_G(s,w) \tag{6.2}$$

durante uma execução, e seja $(u,w) \in E$ o arco cuja relaxação causou a mudança no valor de $\mathrm{d}[w]$. Assim a relaxação faz

$$\mathrm{d}[w] = \mathrm{d}[u] + \rho(u,w) \tag{6.3}$$

e, como w foi o primeiro caso de (6.2), nesse momento $\mathrm{d}[u] \geq \mathrm{dist}_G(s,u)$; logo

$$\mathrm{d}[u] + \rho(u,w) \geq \mathrm{dist}_G(s,u) + \rho(u,w). \tag{6.4}$$

Das três relações (6.2), (6.3) e (6.4) deduzimos

$$\begin{aligned} \mathrm{dist}_G(s,w) &> \mathrm{d}[u] + \rho(u,w) \\ &\geq \mathrm{dist}_G(s,u) + \rho(u,w), \end{aligned}$$

contradizendo a desigualdade triangular. Ademais, uma relaxação nunca aumenta o valor $\mathrm{d}[v]$; logo, uma vez que $\mathrm{d}[v] = \mathrm{dist}_G(s,v)$, esse valor não muda, pois não pode diminuir.

Para provar (b) notemos que, por (a), $\mathrm{d}[v] \geq \mathrm{dist}_G(s,v) = \infty$; portanto nunca será atribuído um valor real a $\mathrm{d}[v]$. □

Proposição 6.2

Seja $P = x_0, x_1, \ldots, x_k, v$ um caminho mínimo de $x_0 = s$ a v em G, com $k \geq 0$. Se $\mathrm{d}[x_k] = \mathrm{dist}_G(s,x_k)$ então após Relaxação(x_k, v) teremos $\mathrm{d}[v] = \mathrm{dist}_G(s,v)$.

Demonstração. Após Relaxação(x_k, v) temos

$$\mathrm{d}[v] \leq \mathrm{d}[x_k] + \rho(x_k, v)$$

e por hipótese

$$\mathrm{d}[x_k] + \rho(x_k, v) = \mathrm{dist}_G(s,x_k) + \rho(x_k, v),$$

logo

$$\mathrm{d}[v] \leq \mathrm{dist}_G(s,x_k) + \rho(x_k, v).$$

Pela subestrutura ótima, $x_0, \ldots, x_k$ é um caminho mínimo até x_k e

$$\mathrm{dist}_G(s,x_k) + \rho(x_k, v) = \mathrm{dist}_G(s,v),$$

portanto $\mathrm{d}[v] \leq \mathrm{dist}_G(s,v)$. Pela Proposição 6.1(a), $\mathrm{d}[v] \geq \mathrm{dist}_G(s,v)$, portanto $\mathrm{d}[v] = \mathrm{dist}_G(s,v)$. □

A correção do algoritmo segue do próximo resultado. Provaremos um *invariante* do laço enquanto. Esse invariante é uma propriedade que permanece válida a cada iteração do laço, independentemente do número de vezes que é executado.

Teorema 6.1

Antes de cada teste da condição do laço **enquanto** na linha 6 (inclusive o último, quando o laço termina) vale que

$$\mathrm{d}[v] = \mathrm{dist}_G(s,v) \text{ para todo } v \in S.$$

Demonstração. Suponha que o invariante falha em algum momento e seja r o menor número de iterações completas após as quais existe $v \in S$ com $\mathrm{d}[v] \neq \mathrm{dist}_G(s, v)$. Para $r = 0$ temos $S = \emptyset$ e após a primeira iteração temos $S = \{s\}$ e $\mathrm{d}[s] = 0 = \mathrm{dist}_G(s, s)$; logo $r \geq 2$.

Seja u o vértice inserido em S na r-ésima iteração. Pela minimalidade de r, os vértices de S antes dessa iteração têm d correto e, pela Proposição 6.1(a), esse valor não muda; a relaxação de arcos que saem de u só altera d de sucessores de u, e u não tem laços. Logo $\mathrm{d}[u] \neq \mathrm{dist}_G(s, u)$ no instante em que u é escolhido na linha 7, e nesse instante, como $r \geq 2$, temos $s \in S$ (o primeiro vértice inserido é s, o único com d finito) e $u \in \overline{S}$. Por 6.1(a),

$$\mathrm{d}[u] > \mathrm{dist}_G(s, u). \tag{6.5}$$

Por 6.1(b), $\mathrm{dist}_G(s, u) < \infty$, e existe um caminho mínimo, digamos que $P = x_0, x_1, \ldots, x_m, x_{m+1}$ em G com $x_0 = s$ e $x_{m+1} = u$.

Como $x_0 \in S$ e $x_{m+1} \notin S$, seja i o menor índice, $0 \leq i \leq m$, tal que $x_{i+1} \notin S$; então $x_0, \ldots, x_i \in S$ e (x_i, x_{i+1}) é o primeiro arco de P que sai de S. Notemos que é possível termos $x_i = s$ e $x_{i+1} = u$.

Quando x_i entrou em S, em uma iteração anterior à r-ésima, tínhamos $\mathrm{d}[x_i] = \mathrm{dist}_G(s, x_i)$ e, logo em seguida, ocorreu Relaxação(x_i, x_{i+1}). Como $x_0, \ldots, x_i, x_{i+1}$ é um caminho mínimo, a Proposição 6.2 dá $\mathrm{d}[x_{i+1}] = \mathrm{dist}_G(s, x_{i+1})$ após essa relaxação, e por 6.1(a) esse valor não muda. Além disso, $\mathrm{dist}_G(s, x_{i+1}) \leq \mathrm{dist}_G(s, u)$, pois os pesos são não-negativos. Usando (6.5),

$$\mathrm{d}[x_{i+1}] = \mathrm{dist}_G(s, x_{i+1}) \leq \mathrm{dist}_G(s, u) < \mathrm{d}[u].$$

Mas x_{i+1} e u pertencem a $\overline{S}$ e u foi escolhido com $\mathrm{d}[u]$ mínimo em $\overline{S}$, de modo que $\mathrm{d}[u] \leq \mathrm{d}[x_{i+1}]$, uma contradição. □

A correção do algoritmo de Dijkstra segue facilmente do teorema acima: cada iteração remove um vértice de $\overline{S}$ e, portanto, após $|V|$ iterações $\overline{S} = \emptyset$, ou seja $S = V(G)$, e o teorema, aplicado ao teste final do laço, garante que $\mathrm{d}[v] = \mathrm{dist}_G(s, v)$ para todo $v \in V(G)$. Os vértices não alcançáveis a partir de s terminam com $\mathrm{d}[v] = \infty$ pela Proposição 6.1(b).

Agora, vamos analisar a complexidade do algoritmo de Dijkstra. As linhas de inicialização tomam tempo $O(|V|)$ e cada Relaxação(u, t) toma tempo constante. A complexidade de tempo do laço depende de como é determinado o mínimo na linha 7. O laço da linha 6 é executado uma vez para cada $u \in V(G)$; a sua contribuição para a complexidade do algoritmo é

$$\sum_{u \in V} \Big(T_7(u) + T_8(u) + T_{9,10}(u) \Big)$$

onde $T_i(u)$ é o tempo gasto na linha i na iteração em que u é escolhido

na linha 7. Se o mínimo é determinado fazendo comparações, então em cada rodada o custo é $|\overline{S}|-1$ e $|\overline{S}|$ assume os valores $|V|, |V|-1, \ldots, 1$; portanto,

$$\sum_{u\in V} T_7(u) = \sum_{i=1}^{|V|}(i-1) = \frac{|V|(|V|-1)}{2} = O(|V|^2).$$

Ademais $\sum_{u\in V} T_8(u) = O(|V|)$ e, como cada arco (u,t) é relaxado uma única vez, na iteração em que u é escolhido,

$$\sum_{u\in V} T_{9,10}(u) = \sum_{u\in V}\sum_{t\in N_G^+(u)} O(1) = O(|E|).$$

Logo o tempo total, incluindo a inicialização, é

$$O(|V|^2 + |E|),$$

para grafos dirigidos simples.

Outra possibilidade é usar uma fila de prioridades para determinar o mínimo

Mantendo os vértices de $\overline{S}$ em uma heap binária com chaves d[ ] e o vetor auxiliar pos[ ], como no algoritmo de Dijkstra–Jarník–Prim com heap (Seção 9), cada escolha do mínimo na linha 7 é uma operação `extrair-min` de custo $O(\log|V|)$ e cada relaxação que melhora d[t] é uma operação `diminuir-chave` de custo $O(\log|V|)$. Como há $|V|$ extrações, no máximo $|E|$ relaxações bem-sucedidas e a construção inicial da heap custa $O(|V|)$, o algoritmo de Dijkstra tem complexidade de tempo $O\big((|V|+|E|)\log|V|\big)$.

6.2 Algoritmo de Bellman–Ford

O ***algoritmo de Bellman–Ford***, ou ***algoritmo de Bellman–Ford–Moore*** leva o nome dos matemáticos Richard Bellman, Lester R. Ford Jr. e Edward F. Moore, que o publicaram em 1958, 1956 e 1959, respectivamente, embora já tenha sido proposto em 1955 por Alfonso Shimbel.

É um algoritmo anterior e mai lentos que o algoritmo de Dijkstra, porém permite pesos negativos nos arcos. Como cada vértice só precisa dos valores de seus vizinhos, o algoritmo tem uma versão distribuída, que está na base dos protocolos de roteamento por vetor de distâncias, como o RIP (*Routing Information Protocol*), usados em redes de computadores.

Seja $G = (V, E, \rho)$ um grafo dirigido com pesos nos arcos, que podem ser negativos. Um ***circuito negativo*** em G é um circuito orientado em que a soma dos pesos dos arcos é negativa. Num circuito negativo podemos ficar dando voltas e a cada volta o comprimento diminui; assim, uma tentativa ingênua de computar distâncias em G

Bellman foi um dos criadores do conceito de *programação dinâmica*, um paradigma para algoritmos de otimização e também cunhou o termo "maldição da dimensionalidade", muito em voga atualmente com os avanços do aprendizado de máquina. Ford, especialista em fluxo em redes, escreveu junto com D.R. Fulkerson uma solução sobre o problema do fluxo máximo e o algoritmo de Ford-Fulkerson, publicado como um relatório técnico em 1954 e em um periódico em 1956, estabeleceu o teorema do fluxo máximo e do corte mínimo; ambos, trabalhavam na *RAND Corporation*. Moore foi inventor da máquina de estados finitos de Moore e um dos pioneiros da vida artificial.

poderia facilmente entrar em um laço infinito.

O algoritmo de Bellman–Ford recebe G e um vértice s e devolve *falso* se G tem um circuito negativo alcançável a partir de s, e *verdadeiro* caso contrário. Neste último caso, os valores $\mathrm{d}[v]$ calculados são as distâncias $\mathrm{dist}_G(s, v)$, com ∞ para os vértices não alcançáveis a partir de s.

A ideia do algoritmo é a seguinte. Suponha que não há circuito negativo alcançável a partir de s e seja v um vértice alcançável. Então existe um caminho mínimo

$$P = x_0, x_1, \ldots, x_k, \qquad x_0 = s,\ x_k = v,$$

e, como um caminho não repete vértices, $k \leq |V| - 1$. Repetimos $|V| - 1$ vezes a Relaxação (Algoritmo 31) de todos os arcos de G, em qualquer ordem. Na primeira rodada o arco (x_0, x_1) é relaxado e, depois dela, $\mathrm{d}[x_1] = \mathrm{dist}_G(s, x_1)$; na segunda rodada o arco (x_1, x_2) é relaxado e, depois dela, $\mathrm{d}[x_2] = \mathrm{dist}_G(s, x_2)$; e assim por diante: depois da rodada i temos $\mathrm{d}[x_i] = \mathrm{dist}_G(s, x_i)$. Como todo caminho tem no máximo $|V| - 1$ arcos, ao final das repetições as distâncias estão determinadas.

Para detectar um circuito negativo alcançável basta testar se há algum arco $(u, w) \in E(G)$ com $\mathrm{d}[w] > \mathrm{d}[u] + \rho(u, w)$ depois das $|V| - 1$ rodadas.

Algoritmo 33: Bellman–Ford(G, s).

Dado : $G = (V, E, \rho)$ dirigido com pesos ρ e $s \in V(G)$.

Devolve : *falso* se G tem um circuito negativo alcançável a partir de s; caso contrário, *verdadeiro*, e então $\mathrm{d}[v] = \mathrm{dist}_G(s, v)$ para todo $v \in V(G)$ e p é um vetor de predecessores.

```
1  para cada v ∈ V(G) faça
2      d[v] ← ∞;
3      p[v] ← nil;
4  d[s] ← 0;
5  cont ← 1;
6  enquanto cont ≤ |V(G)| − 1 faça
7      para cada (u, w) ∈ E(G) faça
8          Relaxação(u, w);
9      cont ← cont + 1;
10 para cada (u, w) ∈ E(G) faça
11     se d[w] > d[u] + ρ(u, w) então
12         retorne falso;
13 retorne verdadeiro, d, p
```

Análise e correção do algoritmo de Bellman–Ford

O resultado a seguir prova que o algoritmo devolve corretamente o valor booleano prometido e que, no caso *verdadeiro*, os valores $\mathrm{d}[v]$ são as distâncias.

Lema 6.1

Seja $G = (V, E, \rho)$ um grafo dirigido com pesos e $s \in V$. *Bellman–Ford*(G, s) devolve *falso* se, e somente se, G tem um circuito negativo alcançável a partir de s. Além disso, se devolve *verdadeiro*, então $\mathrm{d}[v] = \mathrm{dist}_G(s, v)$ para todo $v \in V$.

Demonstração. Valem duas afirmações, quaisquer que sejam os pesos.

(a) Em todo momento, $\mathrm{d}[v] = \infty$ ou $\mathrm{d}[v]$ é o peso de algum passeio de s a v. De fato, isso vale no início e a Relaxação(u, w) só atribui $\mathrm{d}[w] \leftarrow \mathrm{d}[u] + \rho(u, w)$, que é o peso do passeio de s a u seguido do arco (u, w).

(b) Seja $P = x_0, x_1, \ldots, x_k$ um caminho com $x_0 = s$ e $k \leq |V| - 1$. Ao final da rodada i (a i-ésima execução do laço da linha 6), $\mathrm{d}[x_i] \leq \rho(x_0, \ldots, x_i)$, o peso do trecho de P de x_0 até x_i. Provamos por indução em i. Para $i = 0$, $\mathrm{d}[s] \leq 0$ pois $\mathrm{d}[s]$ é inicializado com 0. Os valores d nunca aumentam. Assim, quando (x_{i-1}, x_i) é relaxado na rodada i, temos $\mathrm{d}[x_{i-1}] \leq \rho(x_0, \ldots, x_{i-1})$ (hipótese de indução) e, depois da relaxação, $\mathrm{d}[x_i] \leq \mathrm{d}[x_{i-1}] + \rho(x_{i-1}, x_i) \leq \rho(x_0, \ldots, x_i)$. Em particular, $\mathrm{d}[v] < \infty$ para todo v alcançável a partir de s, pois existe um caminho de s a v e ele tem no máximo $|V| - 1$ arcos.

Se há um circuito negativo alcançável, o algoritmo devolve falso. Seja $C = v_0, v_1, \ldots, v_{k-1}, v_0$ um circuito negativo alcançável a partir de s e $v_k := v_0$. Todos os v_i são alcançáveis, logo $\mathrm{d}[v_i] < \infty$ por (b). Suponha que o algoritmo devolve *verdadeiro*. Então $\mathrm{d}[v_i] \leq \mathrm{d}[v_{i-1}] + \rho(v_{i-1}, v_i)$ para $i = 1, \ldots, k$ e, somando sobre os k arcos de C,

$$\sum_{i=1}^{k} \mathrm{d}[v_i] \leq \sum_{i=1}^{k} \mathrm{d}[v_{i-1}] + \sum_{i=1}^{k} \rho(v_{i-1}, v_i).$$

Como $v_k = v_0$, as duas somas de d são iguais e finitas; logo $0 \leq \sum_{i=1}^{k} \rho(v_{i-1}, v_i)$, contradizendo o fato de C ser negativo.

Se não há circuito negativo alcançável, o algoritmo devolve verdadeiro e $\mathrm{d}[v] = \mathrm{dist}_G(s, v)$. Todo passeio de s a v é a união de um caminho de s a v com circuitos alcançáveis a partir de s, todos de peso ≥ 0; assim, seu peso é $\geq \mathrm{dist}_G(s, v)$. Por (a), $\mathrm{d}[v] \geq \mathrm{dist}_G(s, v)$ sempre. Por outro lado, se v é alcançável, existe um caminho mínimo de s a v, com $k \leq |V| - 1$ arcos, e por (b) com $i = k$ temos $\mathrm{d}[v] \leq \mathrm{dist}_G(s, v)$ ao final das $|V| - 1$ rodadas. Se v não é alcançável, $\mathrm{d}[v] = \infty = \mathrm{dist}_G(s, v)$ por (a). Portanto $\mathrm{d}[v] = \mathrm{dist}_G(s, v)$ para todo $v \in V$. Finalmente, para cada arco (u, w) com u alcançável, $\mathrm{d}[w] = \mathrm{dist}_G(s, w) \leq \mathrm{dist}_G(s, u) + \rho(u, w) = \mathrm{d}[u] + \rho(u, w)$, pois

um caminho mínimo até u seguido de (u, w) é um passeio, de peso $\geq \text{dist}_G(s, w)$; e se u não é alcançável, $\text{d}[u] = \infty$. Em ambos os casos o teste do último laço falha e o algoritmo devolve *verdadeiro*. □

Para a complexidade, a inicialização custa $\Theta(|V|)$, as $|V|-1$ rodadas das linhas 6–7 fazem $|E|$ relaxações de tempo $O(1)$ cada, e a verificação final custa $O(|E|)$. O tempo total é $O(|V| + |V||E|)$, que é $O(|V||E|)$ se $|E| \geq 1$.

6.3 Algoritmo de Floyd–Warshall

Os algoritmos de Dijkstra e de Bellman–Ford calculam as distâncias a partir de *um* vértice s. Para obter as distâncias entre *todos os pares* de vértices poderíamos executar um deles $|V|$ vezes, uma para cada origem. O ***algoritmo de Floyd–Warshall*** faz isso de forma direta, com três laços aninhados e uma matriz.

O algoritmo foi publicado por Robert W. Floyd em 1962, na seção de algoritmos da *Communications of the ACM* (***Algorithm 97: Shortest path***), a partir de um teorema de Stephen Warshall, do mesmo ano, sobre o fecho transitivo de matrizes booleanas, isto é, sobre a alcançabilidade em grafos dirigidos (Exercício 113). A mesma estrutura de três laços já aparecia em trabalhos anteriores, como os de Kleene (1956), sobre expressões regulares, e de Bernard Roy (1959), sobre fecho transitivo, e por isso o método também é chamado de algoritmo de Roy–Floyd–Warshall ou, simplesmente, algoritmo de Floyd. Usa o paradigma de programação dinâmica.

Seja $G = (V, E, \rho)$ um grafo dirigido simples, com $V = \{1, 2, \ldots, n\}$ e pesos ρ nos arcos, que podem ser negativos. Como antes, $\text{dist}_G(i, j)$ é o menor peso de um $(i \to j)$-caminho orientado, com $\text{dist}_G(i, i) = 0$ e $\text{dist}_G(i, j) = \infty$ se j não é alcançável a partir de i. Queremos determinar $\text{dist}_G(i, j)$ para todo par $(i, j) \in V \times V$ ou descobrir que G tem um circuito negativo.

Os ***vértices internos*** de um caminho $x_0, \ldots, x_m$ são $x_1, \ldots, x_{m-1}$ (um arco não tem vértices internos). A ideia do algoritmo é considerar, para $k = 0, 1, \ldots, n$, só os caminhos cujos vértices internos pertencem a $\{1, \ldots, k\}$. Seja P um $(i \to j)$-caminho mínimo entre esses. Há duas possibilidades:

- k não é vértice interno de P; então os vértices internos de P estão em $\{1, \ldots, k-1\}$;
- k é vértice interno de P; como P é um caminho, passa por k uma única vez, e P é a união de um $(i \to k)$-caminho e de um $(k \to j)$-caminho, ambos com vértices internos em $\{1, \ldots, k-1\}$, e cada um deles é mínimo com essa propriedade (subestrutura ótima, como em (F2) na página 86).

Assim, se $\text{d}_k[i, j]$ denota o peso de um tal caminho mínimo (entre os que têm vértices internos em $\{1, \ldots, k\}$), então, na ausência de circuitos negativos,

$$\text{d}_k[i, j] = \min\{\text{d}_{k-1}[i, j],\ \text{d}_{k-1}[i, k] + \text{d}_{k-1}[k, j]\},$$

com $\text{d}_0[i, j] = \rho(i, j)$ se $(i, j) \in E$, $\text{d}_0[i, i] = 0$ e $\text{d}_0[i, j] = \infty$ nos demais casos; e $\text{dist}_G(i, j) = \text{d}_n[i, j]$. O algoritmo a seguir mantém uma única matriz d[,], que ao final da iteração k contém d_k, e uma matriz p[,] de predecessores: p$[i, j]$ é o penúltimo vértice do caminho encontrado de i a j.

Algoritmo 34: Floyd–Warshall(G).

Dado : $G = (V, E, \rho)$ um grafo dirigido simples e pesos ρ pesos nos arcos.

Devolve : *falso* se G tem um circuito negativo; caso contrário, *verdadeiro*, e então $\mathrm{d}[i, j] = \mathrm{dist}_G(i, j)$ para todo $i, j \in V$ e p é a matriz de predecessores.

```
1  para cada i ∈ V(G) faça
2      para cada j ∈ V(G) faça
3          d[i, j] ← ∞;
4          p[i, j] ← nil;
5      d[i, i] ← 0;
6  para cada (i, j) ∈ E(G) faça
7      d[i, j] ← ρ(i, j);
8      p[i, j] ← i;
9  para cada k = 1, 2, ... , n, nessa ordem faça
10     para cada i ∈ V(G) faça
11         para cada j ∈ V(G) faça
12             se d[i, k] + d[k, j] < d[i, j] então
13                 d[i, j] ← d[i, k] + d[k, j];
14                 p[i, j] ← p[k, j];
15 para cada i ∈ V(G) faça
16     se d[i, i] < 0 então
17         retorne falso;
18 retorne verdadeiro, d, p
```

Notemos que o teste da linha 12 é falso se $\mathrm{d}[i, k] = \infty$ ou $\mathrm{d}[k, j] = \infty$, pela convenção acima; e que, na iteração k, a atualização de $\mathrm{d}[i, j]$ usa valores de $\mathrm{d}[i, k]$ e $\mathrm{d}[k, j]$ que podem ter sido atualizados na própria iteração k. A prova a seguir mostra que isso não prejudica a correção.

Análise e correção do algoritmo de Floyd–Warshall

Lema 6.2

Floyd–Warshall(G) devolve *falso* se, e somente se, G tem um circuito negativo. Além disso, se devolve *verdadeiro*, então $\mathrm{d}[i, j] = \mathrm{dist}_G(i, j)$ para todo $i, j \in V$.

Demonstração. Valem duas afirmações, quaisquer que sejam os pesos.

(a) Em todo momento durante e depois da iteração k, $\mathrm{d}[i, j] = \infty$ ou $\mathrm{d}[i, j]$ é o peso de algum $(i \to j)$-passeio com vértices internos em $\{1, \dots, k\}$. De fato, isso vale antes da primeira iteração ($k = 0$),

pois os valores iniciais são pesos de passeios triviais ou de arcos. E a atribuição $\mathrm{d}[i,j] \leftarrow \mathrm{d}[i,k] + \mathrm{d}[k,j]$ é o peso de um $(i \to k)$-passeio seguido de um $(k \to j)$-passeio, que é um $(i \to j)$-passeio cujos vértices internos estão em $\{1, \ldots, k\}$.

(b) Ao final da iteração k, $\mathrm{d}[i,j] \leq c(P)$ para todo par (i,j) e todo $(i \to j)$-caminho P cujos vértices internos estão em $\{1, \ldots, k\}$. Provamos por indução em k. Para $k = 0$, P é um arco, ou o caminho trivial, e o valor inicial de $\mathrm{d}[i,j]$ é $\rho(i,j)$ ou 0. Para $k \geq 1$, lembre que os valores de d nunca aumentam. Se k não é vértice interno de P, então vale a hipótese de indução. Senão, P passa por k uma única vez e é a união de um $(i \to k)$-caminho P_1 e de um $(k \to j)$--caminho P_2 com vértices internos em $\{1, \ldots, k-1\}$. Pela hipótese de indução, no início da iteração k temos $\mathrm{d}[i,k] \leq c(P_1)$ e $\mathrm{d}[k,j] \leq c(P_2)$, e esses valores não aumentam; logo, depois do teste do par (i,j), $\mathrm{d}[i,j] \leq \mathrm{d}[i,k] + \mathrm{d}[k,j] \leq c(P)$.

Se G não tem circuito negativo, todo passeio fechado tem peso ≥ 0 e todo $(i \to j)$-passeio é a união de um $(i \to j)$-caminho com circuitos, portanto tem peso $\geq \mathrm{dist}_G(i,j)$. Por (a), $\mathrm{d}[i,j] \geq \mathrm{dist}_G(i,j)$ ao final; em particular $\mathrm{d}[i,i] \geq 0$, e como $\mathrm{d}[i,i]$ começa em 0 e não aumenta, vale $\mathrm{d}[i,i] = 0$ e o algoritmo devolve *verdadeiro*. Por (b) com $k = n$, todo caminho de i a j tem peso $\geq \mathrm{d}[i,j]$, logo $\mathrm{d}[i,j] \leq \mathrm{dist}_G(i,j)$ (trivial se $\mathrm{dist}_G(i,j) = \infty$). Portanto $\mathrm{d}[i,j] = \mathrm{dist}_G(i,j)$.

Se G tem um circuito negativo, seja C um circuito negativo. Como G não tem laços, C tem pelo menos dois vértices. Seja k o maior vértice de C e $j \neq k$ outro vértice de C. Então C é a união de um $(j \to k)$-caminho P_1 e de um $(k \to j)$-caminho P_2, ambos com vértices internos em $\{1, \ldots, k-1\}$. Por (b) (iteração $k-1$), no início da iteração k temos $\mathrm{d}[j,k] \leq c(P_1)$ e $\mathrm{d}[k,j] \leq c(P_2)$, logo, após o teste do par (j,j) na iteração k, $\mathrm{d}[j,j] \leq c(P_1) + c(P_2) = c(C) < 0$. Como $\mathrm{d}[j,j]$ não aumenta, o último laço encontra $\mathrm{d}[j,j] < 0$ e o algoritmo devolve *falso*. Isso prova as duas implicações, pois se G não tem circuito negativo o algoritmo devolve *verdadeiro*, como visto acima. □

Para a complexidade, os três laços aninhados das linhas 9–11 executam n^3 vezes um teste e, possivelmente, duas atribuições de tempo $O(1)$; a inicialização e o último laço custam $O(n^2 + |E|) = O(n^2)$. O tempo total é $\Theta(n^3)$ e o espaço usado pelas matrizes é $\Theta(n^2)$. Em comparação, executar o algoritmo de Dijkstra $|V|$ vezes, quando $\rho \geq 0$, custa $O(|V|^3)$ com a versão simples e $O(|V||E| \log |V|)$ com *heap* (que é melhor para grafos esparsos), e executar Bellman–Ford $|V|$ vezes custa $O(|V|^2|E|)$, que é $O(|V|^4)$ para grafos densos. O algoritmo de Floyd–Warshall aceita pesos negativos, detecta circuitos negativos e é muito simples de implementar.

Exercícios

Exercício 98. Mostre que o vetor p[ ] devolvido por Dijkstra(G, s) permite reconstruir caminhos mínimos: para todo v alcançável a partir de s, a sequência v, p[v], p[p[v]], ... termina em s e, lida de trás para frente, é um $(s \to v)$-caminho orientado de custo $\mathrm{dist}_G(s, v)$.

Exercício 99. Mostre, com um exemplo, que o algoritmo de Dijkstra pode devolver valores incorretos se G tem arcos de peso negativo. (Dica: G com arcos (s, a), (s, b), (b, a), (a, c) de pesos $2, 3, -2, 1$.) Em que ponto exato da demonstração do Teorema 6.1 a hipótese $\rho \geq 0$ é usada?

Exercício 100. Seja G um grafo dirigido com pesos $\rho(e) \geq 0$ nos arcos. Suponha que $v_1, v_2, \ldots, v_k$ é um caminho orientado de custo mínimo de v_1 a v_k em G. Prove que $v_i, \ldots, v_j$ é um caminho orientado de custo mínimo de v_i a v_j em G para quaisquer i, j com $1 \leq i < j \leq k$. (Dica: se existisse um caminho mais barato Q de v_i a v_j, trocar o trecho por Q daria um passeio de v_1 a v_k de custo menor; retire os circuitos.) Mostre que o resultado vale também se G tem arcos de peso negativo, mas nenhum circuito orientado de custo negativo.

Exercício 101. Seja G um grafo dirigido com pesos $\rho(e) \geq 0$ nos arcos e $s \in V(G)$. Prove que, para todo arco $(u, w) \in E(G)$,

$$\mathrm{dist}_G(s, w) \leq \mathrm{dist}_G(s, u) + \rho(u, w)$$

(equação (6.1)). Onde a hipótese $\rho \geq 0$ é usada? Mostre que a desigualdade pode falhar se G tem um circuito orientado de custo negativo. (Dica: arcos (s, w), (w, u), (u, w) de pesos $1, -3, 1$.)

Exercício 102. Construa um grafo dirigido, com pesos ρ nos arcos, alguns negativos mas sem circuito orientado de custo negativo, no qual o algoritmo Dijkstra(G, s) devolve $\mathrm{d}[v] \neq \mathrm{dist}_G(s, v)$ para algum v. Em que ponto exato da demonstração do Teorema 6.1 a hipótese $\rho \geq 0$ é usada? (Dica: quatro vértices bastam; com três, o algoritmo desta seção, que relaxa também os arcos que chegam em S, ainda acerta. Tente os arcos (s, a), (s, b), (b, a), (a, c) de pesos $2, 3, -2, 1$.)

Exercício 103. Suponha que o teste da linha 6 do algoritmo de Dijkstra seja trocado por $|\overline{S}| > 1$, o que faria o laço ser executado $|V| - 1$ vezes em vez de $|V|$. O algoritmo continua correto? Justifique.

Exercício 104. Seja $D = (V, E, \rho)$ um grafo dirigido com $\rho : E \to [0, 1]$, em que $\rho(e)$ é a probabilidade de o arco e *não* falhar, e suponha que as falhas dos arcos são independentes. A probabilidade de que um caminho orientado P não falhe é $\prod_{e \in E(P)} \rho(e)$. Dados $s, t \in V$, escreva um algoritmo que determina um $(s \to t)$-caminho orientado com a maior probabilidade de não falhar e prove que ele está correto. (Dica: troque cada peso $\rho(e) > 0$ por $-\log \rho(e) \geq 0$ e descarte os arcos com $\rho(e) = 0$.)

Exercício 105. Seja $D = (V, E, \rho)$ um grafo dirigido com pesos $\rho : E \to \{0, 1, \dots, m\}$ nos arcos, $m \in \mathbb{N}$. Descreva uma fila de prioridades para o algoritmo de Dijkstra de modo que a complexidade de tempo para computar as distâncias seja $O(m|V| + |E|)$. (Dica: toda distância finita é no máximo $m(|V| - 1)$; use $m|V|$ listas duplamente ligadas, uma para cada valor de d, e um índice que só cresce; explique por que o índice nunca precisa voltar.)

Exercício 106. Dados um grafo dirigido G com pesos $\rho(e) \geq 0$ nos arcos e um vértice s, considere o algoritmo que inicia com $\mathrm{d}[v] \leftarrow \infty$ para todo vértice v e $\mathrm{d}[s] \leftarrow 0$ e, em seguida, repete $|V|$ vezes a instrução

para cada $(u, t) \in E(G)$ **faça** Relaxação(u, t).

Esse algoritmo determina $\mathrm{dist}_G(s, v)$ corretamente para todo v? Quantas repetições bastam? Qual a complexidade? O resultado vale se G tem arcos de peso negativo, mas nenhum circuito orientado de custo negativo? (Dica: indução no número de arcos de um caminho mínimo.)

Exercício 107. Mostre que, se numa rodada nenhum valor d[ ] é alterado, então nenhuma rodada seguinte altera valores e o laço da linha 6 pode ser interrompido. Que consequência isso tem para o teste de circuito negativo?

Exercício 108 (Propriedades da Relaxação). Seja $G = (V, E, \rho)$ um grafo dirigido com pesos e $s \in V$, e suponha que não há circuito negativo alcançável a partir de s. Suponha que d e p foram inicializados como no Algoritmo 33 (antes do laço da linha 6) e que depois disso foram feitas apenas execuções de Relaxação, em qualquer número e ordem. Prove que sempre valem:

(a) $\mathrm{d}[v] = \infty$ ou $\mathrm{d}[v]$ é o peso de um passeio de s a v, para todo $v \in V$;
(b) $\mathrm{d}[v] \geq \mathrm{dist}_G(s, v)$ para todo $v \in V$, e uma vez que vale a igualdade o valor de $\mathrm{d}[v]$ não muda mais;
(c) se $\mathrm{dist}_G(s, v) = \infty$, então $\mathrm{d}[v] = \infty$;
(d) se $s, \dots, u, w$ é um $(s \to w)$-caminho mínimo e $\mathrm{d}[u] = \mathrm{dist}_G(s, u)$, então depois da execução de Relaxação(u, w) tem-se $\mathrm{d}[w] = \mathrm{dist}_G(s, w)$;
(e) seja $s = v_0, v_1, \dots, v_k = w$ um caminho mínimo. Se ocorrem as relaxações de $(v_0, v_1), (v_1, v_2), \dots, (v_{k-1}, v_k)$, nessa ordem e possivelmente com outras relaxações intercaladas, então ao final da sequência $\mathrm{d}[w] = \mathrm{dist}_G(s, w)$.

Mostre com um exemplo que (b) pode falhar se há um circuito negativo alcançável a partir de s. Use (a)–(e) para provar o Lema 6.1 sem usar a Afirmação (b) da sua prova.

Exercício 109. Para $v \in V$, seja $\delta(s, v)$ o ínfimo dos pesos dos passeios de s a v; assim $\delta(s, v) = -\infty$ se, e somente se, existe um passeio de

s a v que passa por um circuito negativo, e $\delta(s,v) = \text{dist}_G(s,v)$ caso contrário. Modifique o algoritmo de Bellman–Ford para que, ao final, $\text{d}[v] = \delta(s,v)$ para todo $v \in V$. (*Dica:* depois das $|V|-1$ rodadas, faça mais $|V|$ rodadas em que $\text{d}[w] \leftarrow -\infty$ sempre que $\text{d}[u] = -\infty$ ou $\text{d}[w] > \text{d}[u] + \rho(u,w)$, para $(u,w) \in E$. Prove que isso funciona.)

Exercício 110. Suponha que existe um algoritmo $\mathcal{A}(G,u,v)$ que, em tempo polinomial, recebe um grafo dirigido G com pesos reais nos arcos (possivelmente negativos, inclusive com circuitos negativos) e devolve um $(u \to v)$-caminho de custo mínimo, ou informa que não existe caminho. Mostre como usar $\mathcal{A}$ para determinar em tempo polinomial se G tem um circuito orientado hamiltoniano[2]. (*Dica:* atribua peso -1 a todos os arcos e, para cada $(v,u) \in E(G)$, compute $\mathcal{A}(G,u,v)$ e veja se o custo é $-(|V|-1)$.) Por que o algoritmo de Bellman–Ford não é um algoritmo $\mathcal{A}$?

2: Circuito orientado hamiltoniano é um circuito orientado que passa por todos os vértices do grafo dirigido.

Como o problema de decidir se um grafo dirigido tem circuito hamiltoniano é NP-completo, a existência de $\mathcal{A}$ implicaria P = NP.

Exercício 111 (Exibindo um circuito negativo). Suponha que G tem um circuito negativo alcançável a partir de s, de modo que Bellman–Ford(G,s) devolve *falso*. Mostre como encontrar, em tempo $O(|V|)$ adicional, um circuito negativo. (*Dica:* seja (u,w) um arco que falha no teste final; execute Relaxação(u,w), siga o vetor p a partir de w por $|V|$ passos e mostre que o vértice atingido pertence a um circuito de p cujo peso é negativo.)

Exercício 112 (Grafos dirigidos acíclicos). Seja G um GDA com pesos arbitrários nos arcos e $s \in V(G)$. Mostre que, se os arcos são relaxados uma única vez, ordenados pela ordem topológica dos seus vértices de saída (todos os arcos que saem de um vértice antes dos arcos que saem de um vértice posterior), então ao final $\text{d}[v] = \text{dist}_G(s,v)$ para todo $v \in V(G)$. Qual o tempo total, incluindo o cálculo da ordem topológica (Seção 5.2)? (*Dica:* item (e) do Exercício 108.)

Exercício 113 (Fecho transitivo). Seja $G = (V,E)$ um grafo dirigido com $V = \{1,\ldots,n\}$. Considere a variante do Algoritmo 34 em que $\text{d}[i,j]$ é um valor booleano, inicializado com *verdadeiro* se $i = j$ ou $(i,j) \in E$, e *falso* caso contrário, e a atualização é $\text{d}[i,j] \leftarrow \text{d}[i,j] \vee (\text{d}[i,k] \wedge \text{d}[k,j])$. Prove que, ao final, $\text{d}[i,j]$ é *verdadeiro* se, e somente se, existe um $(i \to j)$-caminho orientado em G. Compare o tempo com o de calcular as componentes fortemente conexas (Seção 5.3).

Exercício 114 (Reconstrução dos caminhos). Suponha que G não tem circuito negativo. Prove que, para todo $i \neq j$ com $\text{d}[i,j] < \infty$, a sequência

$$j, \quad \text{p}[i,j], \quad \text{p}\big[i,\text{p}[i,j]\big], \ \ldots$$

chega a i sem repetir vértices e, lida de trás para frente, é um $(i \to j)$-caminho de peso $\text{d}[i,j]$. Escreva um algoritmo que imprime esse caminho em tempo $O(n)$.

Exercício 115 (A ordem dos laços importa). Mostre que, se trocarmos os laços de modo que k seja o laço mais interno, isto é, para cada i, para cada j, para cada k, o algoritmo pode calcular distâncias erradas mesmo sem circuitos negativos. (*Dica:* G com 4 vértices e os arcos $(2,3)$, $(3,4)$ e $(4,1)$, todos de peso 1.) Onde, na prova do Lema 6.2, é usado que k é o laço mais externo?

Exercício 116 (Quando usar cada algoritmo). Para grafos dirigidos com $|V| = n$ vértices, $|E| = \Theta(n)$ arcos e pesos $\rho \geq 0$, qual é mais rápido para calcular todas as distâncias: Floyd–Warshall ou n execuções de Dijkstra com *heap*? E para $|E| = \Theta(n^2)$? E se há arcos de peso negativo, mas nenhum circuito negativo? (*Dica:* execute Bellman–Ford uma vez a partir de um novo vértice s^* com arcos de peso 0 para todos os vértices e use os valores $\mathrm{d}[v]$ para trocar $\rho(u,v)$ por $\rho(u,v) + \mathrm{d}[u] - \mathrm{d}[v]$; mostre que os novos pesos são não negativos e que os caminhos mínimos não mudam.)

Exercício 117 (Diâmetro). Seja G um grafo dirigido fortemente conexo com pesos $\rho \geq 0$. A ***excentricidade*** de i é $\max_j \mathrm{dist}_G(i,j)$, o ***diâmetro*** de G é a maior excentricidade e o ***raio*** é a menor. Mostre como calcular o diâmetro, o raio e um vértice de excentricidade mínima em tempo $O(n^3)$, e que depois de obtida a matriz d isso custa só $O(n^2)$.

Índice

$d^+(v)$, 66
(V, E, ρ), 8
A_G, 19
C^k, 5
$E(G)$, 1
$G + xy$, 40
$G - e$, 39
$G[M]$, 3
$G[U]$, 3
$G \simeq H$, 14
$G^{\complement}$, 9
$H \subseteq G$, 2
K^n, 10
K^n, 15
$N^+(v)$, $N^-(v)$, 66
$N_G(v)$, 2
P^k, 4
$V(G)$, 1
$d^-(v)$, 66
$d_G(v)$, 2
$\mathrm{dist}_G(u, v)$, 6
$\mathrm{tr}(A)$, 27
$\Delta(G)$, 10
$\alpha(G)$, 11
$\chi(G)$, 10
$\delta(G)$, 5
$\omega(G)$, 11
$\nabla_G(L)$, 37
$d(G)$, 10
SG, 66
k-cubo, 12
$p^{(t)}$, 36
$G := (V, E)$, 1
Grafo de Petersen, 15

acíclico, 37, 39
adjacentes, 1
alcança, 7
algoritmo de Bellman–Ford, 91
algoritmo de Bellman–Ford–Moore, 91
algoritmo de Dijkstra, 87, 88
algoritmo de Floyd–Warshall, 94
ancestral próprio, 67
arco, 66
arcos, 8, 66
aresta-transitivo, 26
arestas, 1
arestas adjacentes, 1
assintoticamente **muito** menor, 21
assintoticamente menor, 22
automorfismo, 26

caminho, 4
caminho mínimo, 86
caminho orientado, 69
chega em u, 66
cintura, 13
circuito, 4
circuitos fundamentais de G com respeito a T, 43
circuito negativo, 91
circuito orientado, 69
clique, 11
complemento, 9
completo, 10
complexidade de tempo polinomial, 21
componentes conexas, 7
componente fortemente conexa, 76
comprimento, 5, 6, 86
concatenação, 12
conexo, 7
conjunto estável, 11
conjunto independente, 11
corte de arestas, 37
cortes fundamentais de G com respeito a T, 43
custo, 45

descendente próprio, 67
desigualdade triangular, 86
diagrama, 1
distância, 6, 86
diâmetro, 13, 100

espaço de cortes, 44
excentricidade, 100
extremos, 1, 4

Fecho transitivo, 84
filho, 67
floresta, 39
folha, 39
fonte, 70

G - v, 39
G∩H, 10
G∪H, 9
grafo, 1
grafo com peso nas arestas, 8
grafo das componentes, 78
grafo de Fibonacci, 13
grafo dirigido, 66
grafo dirigido ou digrafo, 8
grafo dirigido simples, 66
grafo orientado, 8
grafo subjacente, 8, 66
grafo transposto, 78
grafo dirigido acíclico, 69
grau de v, 2
grau de entrada, 66
grau de saída, 66
grau máximo, 10
grau médio, 10
grau mínimo, 5

incide, 2
intersecção, 10
isomorfismo, 14
isomorfos, 14

lista de adjacência dirigida, 67

multigrafo, 8

operação elementar, 43
ordenação topológica, 70

pai, 67
passeio, 3
passeio fechado, 4
ponte, 43
problema da ordenação topológica, 70

raio, 100
regular, 11
relaxação, 87

sai de v, 66
semiconexo, 85
subestrutura ótima, 86
subgrafo, 2
subgrafo gerador, 3
subgrafo induzido por M, 3
subgrafo induzido por U, 3
sumidouro, 70

T + uv - a, 43
Termo, 8
traço, 27

união, 9

vizinhança, 2
vértice-transitivo, 26
vértices, 1
vértices internos, 4, 94

árvore, 39
árvore geradora, 41
árvore geradora mínima, 45

Bibliografia

BONDY, J. A.; MURTY, U. S. R. **Graph Theory**. New York: Springer, 2008. (Graduate Texts in Mathematics, v. 244).

BRASSARD, Gilles; BRATLEY, Paul. **Fundamentals of Algorithmics**. Englewood Cliffs: Prentice Hall, 1996.

BRUNO, J.; WEINBERG, L. A constructive graph-theoretic solution of the Shannon switching game. **IEEE Transactions on Circuit Theory**, v. 17, n. 1, p. 74–80, 1970.

CORMEN, Thomas H.; LEISERSON, Charles E.; RIVEST, Ronald L.; STEIN, Clifford. **Algoritmos**: teoria e prática. 3. ed. Rio de Janeiro: Elsevier, 2012.

DASGUPTA, Sanjoy; PAPADIMITRIOU, Christos H.; VAZIRANI, Umesh V. **Algorithms**. Boston: McGraw-Hill, 2008.

DIESTEL, Reinhard. **Graph Theory**. 6. ed. Heidelberg: Springer, 2025. (Graduate Texts in Mathematics, v. 173).

ERICKSON, Jeff. **Algorithms**. 1. ed. [S. l.: s. n.], 2019. Capítulos 5–11. Disponível em: https://jeffe.cs.illinois.edu/teaching/algorithms/. Acesso em: 01/02/2026.

JOYNER, David; NGUYEN, Minh Van; COHEN, Nathann. **Algorithmic Graph Theory**. Version 0.7. [S. l.: s. n.], 2013. Disponível em: https://static.latexstudio.net/wp-content/uploads/2013/03/book.pdf. Acesso em: 01/02/2026.

LEHMAN, A. A solution of the Shannon switching game. **Journal of the Society for Industrial and Applied Mathematics**, v. 12, n. 4, p. 687–725, 1964.

MANBER, Udi. **Introduction to Algorithms**: a creative approach. [S. l.]: Addison-Wesley, 1998.

ROUGHGARDEN, Tim. **Algorithms Illuminated**. Part 2: Graph Algorithms and Data Structures. New York: Soundlikeyourself Publishing, 2018. Disponível em: https://www.rexresearch1.com/AlgorithmLibrary/AlgorithmsIlluminated2Roughgarden.pdf. Acesso em: 01/02/2026.

SEDGEWICK, Robert. **Algorithms in C**. Part 5: Graph Algorithms. 3. ed. Reading: Addison-Wesley, 2002. 482 p.